DELTA-V REPOSITORY BOOK - AUDIO EDITION

Project Documentation

A listener-friendly rewrite of the canonical documentation, produced for text-to-speech reading (for example, Speechify) while walking. Code blocks, tables, and file paths have been replaced with plain-English prose.

This edition is lossy by design. For the faithful reference, use the standard documentation book.

CHAPTERS	WORDS
27	51,724
BUILT	**SOURCE**
19 April 2026 at 20:13	delta-v

Contents

Chapters mirror the standard documentation book. Content has been rewritten for linear listening: code is described rather than quoted, tables are narrated, and forward references are inlined.

Part I. Orientation

Start here for the product overview, contributor workflow, and the main architectural frame of the codebase.

1. Chapter 1. 🚀 Delta-V
   README.md
2. Chapter 2. Contributing
   docs/CONTRIBUTING.md
3. Chapter 3. Delta-V Architecture & Design Document
   docs/ARCHITECTURE.md

Part II. Design Patterns and Contracts

These chapters explain the recurring engineering choices in the system: boundaries, state flow, persistence, validation, scenarios, and testing.

1. Chapter 4. Delta-V Coding Standards
   docs/CODING_STANDARDS.md
2. Chapter 5. Delta-V Pattern Catalogue
   patterns/README.md
3. Chapter 6. Engine & Architecture Patterns
   patterns/engine-and-architecture.md
4. Chapter 7. Protocol & Persistence Patterns
   patterns/protocol-and-persistence.md
5. Chapter 8. Client Patterns
   patterns/client.md
6. Chapter 9. Type System & Validation Patterns
   patterns/type-system-and-validation.md
7. Chapter 10. Scenarios & Config Patterns
   patterns/scenarios-and-config.md
8. Chapter 11. Testing Patterns
   patterns/testing.md
9. Chapter 12. Delta-V Protocol & Data Contracts
   docs/PROTOCOL.md

Part III. Game Design and World

This section covers the canonical rules, scenarios, and the visual direction of the setting and ship roster.

1. Chapter 13. Delta-V Game Rules
   docs/SPEC.md
2. Chapter 14. Delta-V Lore & Visual Direction
   docs/LORE.md

Part IV. Quality, Operations, and Release

Operational guidance for testing, accessibility, observability, security, privacy, reviews, and coordinated releases.

1. Chapter 15. Simulation and Load Testing
   docs/SIMULATION_TESTING.md
2. Chapter 16. Manual Test Plan
   docs/MANUAL_TEST_PLAN.md
3. Chapter 17. Exploratory Testing
   docs/EXPLORATORY_TESTING.md
4. Chapter 18. Accessibility (DOM) - audit guide
   docs/A11Y.md
5. Chapter 19. Observability
   docs/OBSERVABILITY.md
6. Chapter 20. Security & Competitive Integrity Review
   docs/SECURITY.md
7. Chapter 21. Privacy — technical summary
   docs/PRIVACY_TECHNICAL.md
8. Chapter 22. Cross-Cutting Review Plan
   docs/REVIEW_PLAN.md
9. Chapter 23. Coordinated release checklist
   docs/COORDINATED_RELEASE_CHECKLIST.md

Part V. Agents and Integrations

Everything needed to understand the agent-facing APIs, MCP integration, and the deeper agent protocol model.

1. Chapter 24. Delta-V Agents: Practical Guide
   docs/AGENTS.md
2. Chapter 25. Delta-V MCP Reference

docs/DELTA_V_MCP.md

3. Chapter 26. Delta-V Agent Spec
   AGENT_SPEC.md

Part VI. Current Project State

The current backlog closes the book with the remaining open work and priority ordering.

1. Chapter 27. Delta-V Backlog
   docs/BACKLOG.md

How To Listen To This Book

- Open the PDF in Speechify (or any TTS reader). The text is already cleaned up for listening.
- The standard documentation book is still the source of truth. Treat this edition as a walk-companion, not a reference.
- If a passage sounds wrong, check the standard edition before assuming the code disagrees.

PART 1

Orientation

Start here for the product overview, contributor workflow, and the main architectural frame of the codebase.

🚀 Delta-V

Delta-V is an online turn-based multiplayer tactical space combat and racing game featuring realistic vector movement and orbital gravity mechanics across the inner Solar System.

Command your fleet, master astrogation trajectories, slingshot around celestial bodies, and engage in high-stakes combat where positioning and velocity matter as much as firepower.

Features

Delta-V's core gameplay rests on several interlocking systems. Vector physics spaceflight means that velocity persists between turns — you burn fuel to alter your course, and gravity deflects you one turn later. Orbital mechanics let you exploit planetary gravity wells for slingshot maneuvers, while weak gravity at moons is an optional player choice.

Tactical combat uses odds-based dice resolution modified by range and relative velocity. Your arsenal can include mines, torpedoes, and nukes, though per-scenario rules govern which weapons are available.

The game ships with nine scenarios: Bi-Planetary, Escape, Lunar Evacuation, Convoy, Duel, Blockade Runner, Fleet Action, Interplanetary War, and Grand Tour. You can play against a local artificial intelligence at three difficulty levels, join an online multiplayer game via a five-character room code, jump into the quick-match queue to be paired with any waiting opponent, watch as a spectator, or connect as a machine-native agent through the agent API. A public leaderboard using Glicko-2 ratings runs as a unified ladder for humans and agents; no login is required to participate.

Under the hood, the renderer paints a smooth space view using the HTML5 Canvas API while the game engine operates on a strict axial hex grid.

Architecture

The codebase is divided into three main areas. The shared engine is side-effect-free and contains no input or output operations. The server layer runs on Cloudflare Workers and Durable Objects, acting as the authoritative game room. The client layer handles the Canvas renderer, the DOM user interface, and reactive session state. A separate scripts area holds simulation, load testing, agent tooling, and Model Context Protocol (MCP) support.

The server is authoritative and event-sourced. The architecture documentation covers module inventories, data flow, and the Durable Object model in detail.

Documentation

Each topic has one owner document to keep decisions from drifting.

If you are new to the codebase, read in this order. Start with the contributing guide, which covers setup, pre-commit hooks, and verification. Next, read the first two sections of the architecture document for a layer overview and an introduction to Durable Objects. After that, read only the Core Principles section of the coding standards document. Then pick one pattern chapter that matches the area you will touch — options cover the client, engine, protocol, testing, scenarios, and types. Finally, consult the game specification document if you are touching game rules, or the protocol document if you are touching the wire format.

The documentation set covers the following purposes. The game specification holds the canonical game rules and scenario definitions. The protocol document covers the wire format, state shapes, hex math, and HTTP and WebSocket routes. The architecture document describes the module inventory, data flow, Durable Object design, and replay and recovery mechanisms. The coding standards document covers conventions and refactoring guidance. The patterns directory is a design-pattern walk-through explaining why the code looks the way it does. The contributing guide covers the contributor workflow, pre-commit hooks, and verification. The security document covers integrity, abuse controls, rate limits, and data retention. The observability document covers telemetry events, database queries, and incident triage. The accessibility document is a DOM accessibility audit checklist. The privacy technical document explains what the stack stores. There are also documents for the manual test plan, simulation testing, an exploratory-testing playbook for open-ended discovery passes, a recurring review checklist, a coordinated release checklist, an open backlog of prioritised work, ship aesthetics and visual direction, a practical guide for building Delta-V agents, an MCP tool reference, and a deep agent protocol and design reference.

Glossary

A Room is a game lobby identified by a five-character code. One room can host multiple matches through the rematch flow.

A Match is a single game session within a room. Each match has a stable identifier such as "ROOM1 dash m2".

A Seat is a slot for a player, numbered zero or one. It is protected by a player token that allows reconnection.

A Session means two different things depending on context. On the server it refers to the authoritative game Durable Object instance. On the client it refers to the client session aggregate, which manages signals, planning state, and transport.

A Phase is the authoritative game state's current phase. The possible phases are fleet building, astrogation, ordnance, combat, logistics, and game over.

Client State is the finer-grained UI-layer state, which includes substates such as "playing movement animation" derived from the authoritative phase.

An Event is an append-only domain fact in the match stream, called an engine event. This is not a browser DOM event.

A Checkpoint is a full game state snapshot saved at turn boundaries to speed up projection.

Projection is the process of reconstructing game state from a checkpoint plus the event tail that follows it. It is used for reconnects, parity checks, and replay.

An Agent is any non-browser player — a script, a large language model, a reinforcement learning system — connected via MCP, a bridge, or a raw WebSocket. Agents are identified by a player key prefixed with "agent underscore".

A Burn is a fuel-costing course shift during astrogation. Normally a burn shifts you by one hex; a warship overload burn shifts you by two hexes.

An Overload is a two-fuel warship course shift that can be used once between maintenance stopovers.

Quick Start

Setting up a local development environment takes five steps.

First, switch to the Node version pinned by the project's version file — the continuous integration workflow uses the same version. Second, install the project's dependencies using the package manager. Third, install Playwright's Chromium browser, which is used by pre-commit hooks and the continuous integration suite for browser smoke tests. Fourth, start the development server. Fifth, open a browser to the local server address, create a game in one tab, copy the room link or five-character code, and join from a second tab.

CLI Commands

Several npm scripts are available. The dev script starts local development using the Wrangler bundler and esbuild. The build script produces the client bundle. The lint script runs Biome for linting and format checking. The typecheck script checks types in the source directory, and the typecheck-all variant also covers scripts, end-to-end tests, and configuration files.

For testing, the test script runs all unit tests using Vitest. The test coverage script runs tests with coverage reporting and enforces thresholds on the shared engine code. The end-to-end test script runs Playwright browser smoke tests against a local Wrangler server. The accessibility end-to-end test script adds an axe accessibility baseline to that suite. A headed variant of the smoke suite runs the same tests with a visible browser. The watch script runs Vitest in watch mode.

The verify script runs a full pre-release sweep covering lint, type checking, coverage, build, end-to-end tests, accessibility, and simulation. The simulate script runs headless AI-versus-AI matches; you pass a scenario name and iteration count as arguments. The load test script runs a WebSocket load and chaos harness; you pass the number of games and concurrency level as options. Finally, the deploy script deploys to Cloudflare Workers.

Test Strategy

Three complementary layers keep the regression net cheap to run.

Vitest covers engine, protocol, client-helper, and server-logic unit and property tests. This is the main regression net. Headless AI simulation runs scenario-wide engine stability and balance sweeps, which is much cheaper than running a full browser. Playwright provides an intentionally thin browser smoke suite for boot, core multiplayer, and accessibility baselines.

When deciding where a new test belongs: rules, combat, and protocol assertions go in Vitest; broad scenario behavior across many turns goes in simulation; anything that requires a real browser, multiple pages, storage, or WebSocket wiring goes in Playwright. The simulation testing document covers simulation in detail, and the contributing guide covers the verification flow.

Game Rules

The canonical ruleset — movement edge cases, damage tables, and scenario-specific rules — lives in the game specification document. Open engineering work lives in the backlog document.

External References

The project builds on Cloudflare Workers, Durable Objects, and the WebSocket Hibernation API on the server side, and the HTML5 Canvas API and the Service Worker API on the client side. TypeScript narrowing patterns are also used extensively throughout the codebase.

License

All rights reserved.

Contributing

This document covers the contributor workflow only. The readme handles onboarding, the architecture document covers system design, and the coding standards document covers conventions.

Pre-commit (Husky)

The pre-commit hook runs a series of checks in order before any commit is accepted.

First, it runs the lint check. Next, it performs a set of boundary checks using text search — these will fail the commit if any violations are found. Specifically, it checks for direct inner-HTML assignment outside the dedicated DOM helper module (which requires using the set-trusted-HTML helper instead), for use of the built-in random number generator inside the shared engine directory (which requires using an injected random number generator instead), and for any console log, warn, or error calls inside the shared layer (which must remain side-effect free).

After the boundary checks, it runs a full typecheck across all packages. Then it applies any pending local database migrations. Next, it clears the coverage directory and runs the full test suite with coverage reporting. It then runs Playwright browser smoke tests with end-to-end testing enabled, followed by Playwright accessibility baseline tests using the axe tool. Finally, it runs a headless simulation sweep across all nine scenarios for sixty iterations.

Continuous integration runs the same list, except for the local database setup step.

Coverage

The coverage test run uses a flag to disable file parallelism so that the coverage file merger does not encounter race conditions on its temporary files. If coverage fails unexpectedly, removing the coverage directory and retrying is the recommended fix.

Both the standard test command and the coverage test command set a Node options environment variable to silence experimental web-storage warnings that appear in Node version 25 and later.

Playwright ports

The default port for Playwright end-to-end tests is 8787.

In continuous integration, end-to-end tests run on that default port. In the pre-commit hook, a free TCP port is chosen dynamically via Node, and a flag is set so that Playwright does not attempt to reuse an existing server — this avoids accidentally attaching to a running development server. To run end-to-end tests manually while a development server is already holding the default port, you can set the port environment variable to any other free port before running the end-to-end test command.

Windows

The pre-commit hook is a POSIX shell script. On Windows, use Git Bash, WSL, or a similar POSIX-compatible shell.

Skipping hooks (emergency only)

You can bypass the pre-commit hook by passing a no-verify flag to the git commit command. This skips all checks, which continuous integration will then catch and fail on — so prefer fixing the underlying issue instead.

Full verification

The verify command runs the full local release gate in sequence: lint, typecheck for both the application and tools, coverage, build, end-to-end tests, accessibility end-to-end tests, and a headless simulation sweep. The simulation sweep uses forty iterations when run via the verify command, compared to sixty iterations in pre-commit and continuous integration, to keep it responsive when invoked by hand.

Documentation

Each topic has a single owner document. Update documentation when behavior or architecture decisions materially change, and prefer adding anchored sections to existing files over creating new ones. The review plan document tracks the recurring review cadence, and the backlog document tracks open work.

After editing documentation, run the doc-links check command. It walks every linked reference under the readme, the agent specification document, the docs directory, and the patterns directory, verifying that files exist and that anchors match heading slugs.

CHAPTER 3 PART I. ORIENTATION DOCS/ARCHITECTURE.MD

Delta-V Architecture & Design Document

Delta-V is an online multiplayer space combat and racing game. This document describes the current architecture, core systems, major design patterns, and the highest-value follow-ups.

The authoritative server model is event-sourced: the Durable Object persists a match-scoped event stream plus checkpoints, and recovers authoritative state from the checkpoint plus event tail — not from a separate persisted game-state snapshot slot.

Document boundary: gameplay rules and protocol examples live in the spec document, coding conventions live in the coding standards document, contributor workflow lives in the contributing guide, open work lives in the backlog, and the deeper walk-through of why patterns look the way they do lives in the patterns directory.

Quick Navigation

- 1. High-Level Architecture
- 2. Core Systems Design
- 3. Data Flow
- 4. Dependency Map
- 5. Current Decisions and Planned Shifts
- 6. Client bundle and release hygiene

Deployment assumption: the client and Worker ship as a single version line, meaning one deploy updates the Worker and static assets together. Staggered old-client / new-server configurations are not supported. Breaking protocol changes require a coordinated deploy; additive JSON fields are preferred. When bumping the game-state schema version, a coordinated release checklist applies.

---

1. High-Level Architecture

Delta-V uses a full-stack TypeScript architecture built around a shared side-effect-free engine with authoritative edge sessions. The authoritative room persists an append-only match stream and derives current state from that stream plus optional checkpoints.

The codebase is divided into three top-level areas. The shared folder holds all game logic with no I/O — it is fully testable and side-effect-free. The server folder contains the Cloudflare Workers (HTTP-Requests — "Workers" is Cloudflare's term for serverless edge functions) entry point plus three Durable Object classes. The client folder holds the state machine, the Canvas renderer, and the DOM UI.

Cloudflare Durable Objects

Three Durable Object classes back the server. The first, called the game binding, runs one instance per active match and is responsible for the event stream, checkpoints, WebSocket connections, and alarms. The second, called the matchmaker binding, is a singleton running a single global instance that manages quick-match ticket queues and seat pairing. The third, called the live-registry binding, is also a singleton that powers the live-matches listing endpoint.

The flow between these components works as follows. The Cloudflare Worker handles incoming HTTP requests and routes them. Quick-match requests go to the matchmaker; room creation, joining, replay, and WebSocket upgrade requests go to the game binding; and the live-matches query goes to the live registry. Once the matchmaker pairs two players it hands off to the game binding. When a game starts or ends, the game binding notifies the live registry. Completed matches are archived to object storage, and telemetry and error data are written to a relational database.

The game Durable Object carries nearly all game state and is the focus of the rest of this document. The two support Durable Objects are small and each runs as a single instance in the cluster.

Diagrams (Mermaid)

The following paragraphs describe the runtime layers and action flows that diagrams in the source document illustrate visually.

Runtime layers: The browser contains a DOM UI with reactive signals, a Canvas renderer, and a client kernel that coordinates them. Both the UI and renderer talk to the kernel. The kernel communicates outward through a transport abstraction over a WebSocket connection. On the other side of that connection sits the Cloudflare Worker, which delegates to the game Durable Object, which in turn calls the shared engine. The Durable Object also persists data to Durable Object storage, optional object storage, and a telemetry database.

Authoritative action path (multiplayer): A client opens a WebSocket connection to a room. When the client sends a message, the Durable Object first applies a socket-level rate limit, then parses and validates the message against the client-to-server union type. The validated message is dispatched to the game-state action handler, which calls the appropriate shared-engine function such as the astrogation processor or the combat resolver. The engine returns a new state plus a list of engine events. On success the Durable Object appends versioned event envelopes, saves a checkpoint if appropriate, publishes the state change, and broadcasts a filtered state-bearing message back to clients.

Client command path: Raw input events are interpreted into game commands, which the command router sends either to the local engine (for single-player) or over the WebSocket (for multiplayer).

Engine phase state machine: The authoritative game phase follows a state machine. The game begins in a waiting state. From waiting it can move to fleet building or directly to astrogation. Fleet building can advance to astrogation or end in game over. Astrogation connects to ordnance, logistics, combat, a self-

loop for timeout-advance paths, and game over. Ordnance connects to logistics, combat, astrogation, and game over. Logistics connects to combat, astrogation, and game over. Combat connects back to astrogation or to game over.

Event-sourced recovery and replay projection: When an authoritative action runs, the engine returns the next state plus engine events. Those events are appended as event envelopes and checkpoints are saved at turn boundaries and game end. To rebuild state — for reconnection or replay — the system loads the latest checkpoint, loads the event tail that followed it, and projects those events forward to the current state.

Diagram maintenance rule: when command flow, phase transitions, or persistence and projection behavior changes, the diagrams should be updated in the same pull request.

Key Technologies

- Language: TypeScript in strict mode across the entire stack.
- Frontend: the HTML5 Canvas 2D API for rendering, and raw DOM events for UI and input. No heavy frameworks such as React or Vue are used, keeping the game loop as fast as possible.
- Backend: Cloudflare Workers (HTTP-Requests) for HTTP routing and Cloudflare Durable Objects for authoritative game state and WebSocket management.
- Build and tools: esbuild for fast client bundling, wrangler for local testing and deployment, and Vitest for unit testing.

Architectural Stance

- Side-effect-free engine. The shared folder has no I/O; the Durable Object wraps it with persistence and WebSocket plumbing.
- Event-sourced authoritative state. Match state is a projection of an event stream plus checkpoints — live play, replay, and reconnect share one code path.
- Scenario-driven. A flat scenario-rules flag bag and a partial AI-config override let scenarios vary gameplay without branching engine code.
- Narrow class usage. The only production class is the game Durable Object class. Everything else uses factory functions.
- Zero runtime UI framework. Canvas 2D rendering plus a small local signals library; no React, Vue, Immer, or similar.

Each of these stances is walked through in the patterns directory with examples and tradeoffs.

---

2. Core Systems Design

The architecture is divided into three distinct layers: Shared Logic, Server, and Client.

A. Shared Game Engine

This is the heart of the project. All game rules live in the shared folder, making the system robust and completely unit-testable.

Module Inventory

The shared layer contains the following modules. The hex math module handles axial hex distance, neighbours, line drawing, and pixel conversion — it has zero game knowledge and is fully generic. The utility module provides functional collection helpers and is also fully generic.

The types folder holds shared interfaces for domain objects, protocol messages, and scenario data. The shared protocol module handles runtime client-to-server validation and normalization — trimming chat, bounding payloads — and complements the protocol types. The replay module contains the replay timeline structure and match identity builder. The constants module defines ship stats, ordnance mass, detection ranges, and combat and movement constants. The movement module implements vector movement with gravity, fuel, takeoff and landing, and crash detection. The combat module handles gun combat tables, line-of-sight, range and velocity modifiers, heroism, and counterattack. The map-data module defines solar system bodies, gravity rings, bases, and scenario definitions.

The AI folder contains a rule-based AI with composable scoring, per-phase decision modules, and difficulty configuration. The scenario capabilities module provides a derived capability layer — defaults plus feature predicates for scenario rules. The engine barrel module re-exports the public engine API. The engine events module defines a discriminated union of 32 granular domain event types.

The event projector module performs deterministic projection from a persisted event-envelope stream plus checkpoints to a game state; it is used by both the server and tests. Additional engine phase modules cover game creation, fleet building, astrogation, movement, combat, ordnance, logistics, victory, and shared helpers. The turn-advance module handles damage recovery, player rotation, reinforcement spawning, and fleet conversion. The post-movement module covers ramming, inspection, capture, resupply, and detection.

Key Design Patterns

The game engine module is a side-effect-free state machine. It takes the current game state and player actions — astrogation orders, combat declarations — and returns a new game state along with events covering movements and combat results. It has no I/O side effects: no DOM, no network, no storage. It never mutates the caller's state.

The movement module contains the complex vector math, gravity-well logic, and collision detection. Moving a ship is resolved strictly on an axial hex grid.

The combat module evaluates line-of-sight, calculates combat odds based on velocity and range modifiers, and resolves damage. It mutates ships directly, updating lifecycle fields and heroism flags.

The types folder is the single source of truth for all data structures — game state, ships, combat results, and network message payloads — split across domain, protocol, and scenario files with a barrel re-export. This ensures the client and server never fall out of sync.

Dependency injection: engine functions accept a map and a random-number generator as parameters so they can be tested without global state or non-determinism.

Domain event emission: turn-resolution engine entry points emit engine events — 32 granular types including ship moved, ship crashed, combat attack, ordnance launched, phase changed, game over, committed command events, and logistics events — alongside state and animation data. The server reads these engine events directly; there is no server-side event derivation. Movement animation data remains separate for client rendering.

AI Strategy Design

The AI uses a config-weighted composable scoring architecture rather than a monolithic decision tree.

The AI difficulty config module defines a flat record of roughly 60 numeric weights and boolean flags. Three presets — easy, normal, and hard — tune aggression, accuracy, and capability without changing any logic. The scenario rules allow selectively overriding individual knobs per scenario; for example, the Duel scenario lengthens engagements this way. This is the Strategy pattern expressed as data rather than class hierarchies.

The AI scoring module contains composable scoring functions, each handling one concern: navigation scoring weighs distance and speed toward an objective; escape scoring weighs distance from center and velocity; race-danger scoring applies a gravity-well proximity penalty; gravity look-ahead scoring captures deferred-gravity value one turn ahead; and combat-positioning scoring evaluates engagement and interception posture. Each function takes a course candidate and a config and returns a number.

The AI orchestration module ties it together: for each AI ship it enumerates all seven burn options (six hex directions plus a null burn), computes each resulting course, sums scores across all strategies, and picks the highest. Combat and ordnance decisions follow the same evaluate-all-options-then-pick pattern.

Difficulty tuning is pure data, new scoring dimensions are pure additions, and all AI functions accept a random-number generator for deterministic testing.

Engine Mutation Model and RNG Injection

The shared engine is side-effect-free and externally immutable. Turn-resolution entry points deep-clone their input state, mutate the clone internally, and return it. A random-number generator is a mandatory parameter on all turn-resolution entry points; the server derives a per-match, per-action pseudo-random number generator from a seed persisted in storage.

B. The Server

The backend leverages Cloudflare's edge network.

Module Inventory

The server layer contains the following modules. The Worker entry module handles top-level route dispatch and static asset proxying — a generic pattern. The room routes module handles room lifecycle endpoints for creating, joining, replaying, and upgrading to WebSocket — roughly 85% of this shape is reusable across projects. The reporting module handles error and telemetry endpoints, hashing, and rate-limit helpers. The environment module defines Worker bindings and types. The protocol module covers room codes, tokens, initial-payload parsing, seat assignment, and shared-validator re-export — again about 85% game-agnostic.

The game Durable Object class composes the fetch, WebSocket, and alarm paths — roughly 70% of this is reusable multiplayer plumbing. The fetch module handles HTTP init, join, and replay endpoints plus WebSocket upgrade and welcome or reconnect messages. The WebSocket hibernation module handles incoming messages and close events, delegating parsed message handling to the socket helper module. The alarm module manages disconnect forfeit, turn timeout, and inactivity archiving. The turn-timeout module handles the timeout branch including engine outcome and state publication. The telemetry module reports engine and projection errors to the database.

The actions module wires per-action engine calls together. The broadcast module provides filtered message broadcasting and socket send helpers. The publication module runs the full state-publication pipeline: appending events, checkpointing, parity verification, archiving, timer management, and broadcasting. The HTTP handlers module handles init, join-check, and replay request handling. The socket helper module handles message rate limiting, client parsing, and an auxiliary message dispatch map.

The projection module shapes replay timelines using the event projector and applies viewer-filtered replay entries. The match module handles session initialization and rematch logic. The archive module covers match-scoped event envelopes with game ID, sequence number, timestamp, and actor fields, plus checkpoints, replay projection helpers, and match identity. The archive storage module manages chunked event stream keys in Durable Object storage. The match-archive module persistently archives completed matches to object storage plus writes metadata to the relational database — this pattern is fully generic. The message builders module constructs server-to-client message shapes. The session module manages the disconnect grace period and alarm scheduling — fully generic. The turns module handles turn-timeout auto-advance.

Key Patterns

- Event-sourced matches: a chunked event stream plus checkpoints; state is projected from the stream.
- Hibernatable WebSocket plus single-alarm scheduling: sockets are tagged with a player identifier; one alarm multiplexes disconnect grace, turn timeout, and inactivity.
- Single state-bearing outbound message per action: one server-to-client frame carries the full updated game state; a game-over message follows as a separate frame.
- Viewer-aware filtering: a filter function strips hidden state before sending; it is used for live play, reconnect, replay, and spectators.

- Single choke points: the publish-state-change function, the run-game-state-action function, and the client-side apply and clear functions each serve as the sole point where their respective side effects occur.
- Shared runtime protocol validation lives in the shared protocol module beside the protocol types — the Durable Object consumes the validate-client-message function rather than owning message shape itself.
- Rate limiting is described in the security document.
- The match-archive binding connects production deployments to object storage so completed rooms can persist replay data after the Durable Object goes inactive.

Seat Assignment and Disconnect Grace

Seat assignment uses a three-step fallback. First, a player-token match returns a returning player to their seat even if the previous socket is still open. Second, a token-less join is allowed when an open seat has no player token — this is the default guest flow. Third, if no seats are available the join is rejected.

On disconnect, the Durable Object stores a marker containing the player ID and a 30-second deadline, then schedules an alarm. If the player reconnects within 30 seconds with a valid token the marker is cleared. If the alarm fires with the marker still in place the game ends by forfeit.

C. The Client

The frontend renders the pure hex-grid state into a smooth, continuous graphical experience.

Module Inventory

The client layer is organized into four areas. The client root contains the entry point, raw input, audio, tutorial, DOM helpers, telemetry, viewport management, and the reactive signals runtime. The game subfolder holds command routing, the planning store, the game-state store, state transitions, session control, phase logic, transport, actions, HUD controller, and camera controller. The renderer subfolder handles Canvas rendering including camera, scene, entities, and effects. The UI subfolder covers DOM overlays: menu, HUD, ship list, fleet building, game log, formatters, button bindings, and screens.

Three-Layer Input Architecture

1. Raw input: mouse, touch, and keyboard events are translated into raw input events covering hex clicks and hex hovers. This layer has no game knowledge.
2. Game interpretation: raw input events combined with the current phase and state are mapped to game commands by a pure function.
3. Command dispatch: the command router sends each game command either to local state or over the network.

Client State Machine

The client state machine moves through: menu, connecting, waiting for opponent, and a family of playing substates, then game over. The playing substates are fleet building, astrogation, ordnance, logistics, combat, movement animation, and opponent turn. Input is only processed when the phase matches the

active player.

Interaction FSM (Library-free)

To prevent UI race conditions and ensure visibility is strictly synchronized with the game state, the client uses a lightweight interaction finite state machine.

The interaction mode — astrogation, animating, waiting, and so on — is derived directly from the single stored client state rather than being maintained as a second mutable field. The apply-client-state-transition function owns stored client state changes, and reactive consumers derive the interaction mode from the state signal as needed. A screen-visibility module maps these interaction modes to declarative DOM visibility states, ensuring buttons such as the confirm button or fire button are never visible during unauthorized transitions like animations.

Rendering Pipeline (per frame)

The rendering pipeline runs three layers per frame.

1. The scene layer renders in world coordinates: starfield, hex grid, gravity indicators, bodies, asteroids, and bases.
2. The entity layer renders animated elements: ship trails, velocity vectors, ships, ordnance, and combat effects.
3. The overlay layer renders in screen coordinates: ordnance guidance, combat highlights, and the minimap.

Key Design Patterns

The browser entry module handles global setup — error handlers, viewport, service worker reload — and creates the game client, then exposes it on the global window object.

The client kernel module exports the top-level client coordinator. It owns WebSocket and local-AI orchestration and delegates command dispatch, authoritative-state application and clearing, planning mutations, runtime and session fields, state-entry side effects, and session lifecycle to focused sub-modules. The runtime bootstrap exposes only the renderer, the show-toast function, and a dispose function on the global game object.

The renderer module is a highly optimized Canvas 2D renderer factory. It separates logical hex coordinates from pixel coordinates. Extracted helpers own movement-animation lifecycle and trail state. The renderer itself is the canvas shell and per-frame orchestrator, and composites a cached static scene layer — stars, grid, gravity, asteroids, and bodies — when the camera and viewport are unchanged.

The input module manages user interaction: panning, zooming, and clicking. It translates raw browser events into input event objects, while a companion module owns pointer drag, pinch, and minimap state and math. The input shell owns its DOM listener lifecycle, including outside-canvas pointer release and touch-cancel cleanup. A pure interpret-input function then maps these to game commands, ensuring the input layer never directly mutates application state.

The game subfolder handles command routing, action handlers for astrogation, combat, and ordnance, planning-state helpers, runtime and session helpers, phase derivation, game-state helpers, transition helpers, session helpers, transport abstraction, connection management, input interpretation, view-model helpers, and presentation logic. Ordnance-phase auto-selection and HUD legality are derived from shared engine rules rather than client-only heuristics.

The renderer subfolder contains Canvas drawing layers — scene, entities, vectors, effects, and overlays — plus camera, minimap, and animation management.

The UI subfolder contains screen visibility logic, HUD view building, button bindings, game log, fleet building, ship list, formatters, layout metrics, and small reactive DOM view models.

The reactive signals runtime and the UI manager together keep the client framework-free while using a small signals runtime for durable session and UI state. The client session owns reactive game state, client state, player identity, waiting and reconnect fields, and logistics references. The UI manager owns long-lived view instances and derives DOM visibility directly from the state signal through the interaction FSM, with a small scenario-active signal for the menu and scenario UI sub-state. Short-lived events such as toasts remain imperative.

The planning store owns local planning mutation methods and a revision signal that increments after local planning changes. Pure derivation modules build HUD state, submitted orders, and message handling plans from the session and planning state rather than relying on one broad helper module.

The client session is split across several files. One module defines the client session aggregate shape; another owns route and WebSocket URL helpers; a third handles HTTP create, join, and replay flows; and two more own reconnect token persistence. Stores and controllers mutate the aggregate through focused collaborators.

The audio module handles Web Audio API (Application Programming Interface) interactions.

D. Progressive Web App

Delta-V is a fully installable Progressive Web App, or PWA. A lightweight hand-written service worker provides several capabilities.

Precaching of the app shell — the main page, the client bundle, the stylesheet, and icons — enables instant repeat loads. Offline single-player works because the AI opponent runs entirely client-side, so cached assets allow full gameplay without a network connection. The service worker never intercepts non-GET requests and explicitly bypasses multiplayer and reporting routes — WebSocket connections, room creation, join validation, error reporting, and telemetry — so those stay authoritative. Static assets use a stale-while-revalidate strategy and navigation uses a network-first strategy, complementing Cloudflare's edge caching. The build script injects a content hash into the service worker cache name, so every deploy with code changes triggers an automatic service worker update and page reload.

E. Build Pipeline

The project uses minimal, fast build tooling with no heavy bundler configuration.

Client bundle: a build script produces a single ECMAScript module bundle from the client entry point. Production builds are minified; development builds include source maps. esbuild was chosen for sub-second build times. Server bundle: wrangler handles server compilation and deployment and provides local development with Durable Object simulation. Cache busting: the build script hashes the output bundle and stylesheet, then injects the hash into the service worker cache name. Every deploy with code changes triggers an automatic service worker update. Type checking: the TypeScript compiler runs separately from bundling — esbuild strips types without checking them. Linting: Biome runs as a pre-commit hook and in continuous integration.

Cloudflare bindings defined in the wrangler configuration include: the game binding, which is a Durable Object class for authoritative game rooms; the matchmaker binding, a singleton Durable Object for the quick-match queue; the live-registry binding, another singleton Durable Object for the "live now" matches registry; the database binding, a D1 relational database that holds telemetry plus the match archive, player, and match rating tables; the match-archive binding, an R2 bucket for completed match JSON archives; the assets binding for the static bundle; and four rate-limit bindings — the create limiter at five per minute that is shared by create, agent-token, quick-match, and claim-name; the telemetry limiter at one hundred twenty per minute; the error limiter at forty per minute; and the Model Context Protocol limiter at twenty per minute, keyed per agent-token hash or per hashed IP.

F. Testing Infrastructure

Testing uses Vitest with co-located test files, property-based testing via fast-check, and enforced shared-engine coverage.

Test organization: unit tests live next to the module they test, property tests do the same, and contract fixtures are JSON files in fixtures directories for protocol shape assertions. There are no separate test folders.

Next, mock patterns for Durable Objects. Focused test modules for each Durable Object concern — alarm, fetch, turn-timeout, and WebSocket — stub storage and handler dependencies with mock functions instead of full Durable Object harnesses when a narrow branch is under test. A mock storage implementation is an in-memory map with get, put, delete, and list operations matching the Durable Object storage API, including atomic multi-key put. A mock Durable Object state implementation tracks sockets via a weak map for tag-based lookup, matching the hibernatable WebSocket API surface.

Deterministic pseudo-random number generation in tests: engine tests pass a deterministic random-number function — such as one that always returns 0.5, or a seeded sequence — to reproduce exact outcomes. This is why random-number generator injection is mandatory for all turn-resolution entry points.

Property-based test generators: custom fast-check arbitraries generate valid game inputs within bounded ranges — hex coordinate generators, small velocity vector generators, and so on. Tests verify invariants that must hold across all inputs: fuel never goes negative, hex distance is symmetric, and movement preserves conservation laws.

Coverage thresholds: the shared folder has enforced coverage thresholds for statements, branches, functions, and lines via the Vitest configuration. Both the pre-commit hook and CI run the coverage check to prevent backsliding.

Library Stance

The architecture currently benefits from a narrow dependency surface. That remains the default.

Do not add framework, state-machine, or rendering stacks by default. React, Vue, Redux, Zustand, RxJS, XState, and canvas or game frameworks would blur boundaries that are currently explicit and testable. Prefer targeted libraries only when they remove a real maintenance or security burden.

Potentially good additions later: a DOM-sanitization library if any user-controlled or external HTML needs to be rendered; a schema library such as Valibot or Zod if protocol or event-envelope schemas expand enough that handwritten validators become harder to reason about.

Not worth swapping right now: the custom reactive layer. It is small, tested, and intentionally scoped. Replacing it with a library would only make sense if the project no longer wants to own reactive internals.

---

3. Data Flow

A Movement Turn

1. During the Astrogation phase, players select their burn — that is, their acceleration vectors — via the input module.
2. The client sends an astrogation WebSocket message to the server.
3. The game Durable Object gathers orders from both players.
4. When both players have submitted, or the turn timer expires, the server calls the astrogation processor in the shared engine.
5. The engine calculates the new physics vectors, resolves gravity effects, and detects crashes.
6. The Durable Object saves the new state and broadcasts a movement result to both clients.
7. The clients receive the result, pause input, and the renderer smoothly interpolates the ships along their calculated paths. Once the animation finishes, the game proceeds to the Ordnance and Combat phase.

WebSocket Protocol

Client-to-server messages cover fleet ready, astrogation, ordnance, place base, skip ordnance, begin combat, combat, single combat, end combat, skip combat, logistics, skip logistics, surrender, rematch, chat, and ping.

Server-to-client messages cover welcome, spectator welcome, match found, game start, movement result, combat result, single combat result, state update, game over, rematch pending, chat, error, pong, and opponent status.

All messages are discriminated unions validated at the protocol boundary. Chat payloads are trimmed before validation and blank post-trim messages are rejected, so non-UI clients cannot inject empty log entries. Clients never mutate authoritative state. The server persists authoritative events plus optional checkpoints, and replay and reconnect are derived from that same persisted stream.

Multiplayer Session Lifecycle

Creating a room: the client posts to the create endpoint. The Worker generates a room code and a creator token, and initializes the Durable Object. Joining a room: the client hits the join endpoint with an optional player token for a preflight validation check. Replaying a match: an authenticated replay endpoint fetches history from the checkpoint plus event stream. Opening a WebSocket: the Durable Object accepts the connection and tags the socket with the player ID. Once both unique seats are connected the game is created and a game-start message is broadcast. The game loop then runs: each client-to-server action runs through the engine, saves state and events, restarts the timer, and broadcasts a server-to-client result. On disconnect a 30-second grace period begins; the player may reconnect with a valid token, otherwise the game ends by forfeit.

Event-Sourced Match Lifecycle

At a high level, the event-sourced match lifecycle works as follows.

1. The client submits a validated command.
2. The Durable Object appends canonical, versioned domain events to a per-match stream.
3. Authoritative state is rebuilt or incrementally projected from the checkpoint plus the event tail.
4. Player and spectator or public views are derived from that projection.
5. The server broadcasts one state-bearing update plus any animation and log summaries needed by the client.

With that model established, game-state snapshots are transport payloads and optional checkpoints rather than the authoritative persisted truth.

---

4. Dependency Map

The client entry point creates the game client via the client kernel module, which serves as the composition root. The client kernel coordinates the renderer (which draws to the canvas and reads planning state by reference), the input module (which parses mouse and keyboard events into input events), the UI manager (which manages screens and accepts UI events), the command router (which maps game commands to local state mutations or network transmission), and a set of stores and controllers that apply shared runtime and session field updates, authoritative game state, and planning mutations. It also wires session lifecycle through the session controller and session API, handles server-to-client messages, manages browser event wiring and URL auto-join, coordinates UI and input and keyboard into game commands, lazily caches action handler dependencies, applies client-state entry effects and screen changes, and provides transport, phase derivation, keyboard action mapping, derived HUD and order views, and phase-specific action handlers.

The renderer module depends on a camera module for viewport transforms, a set of pure drawing modules for scene, entities, vectors, effects, and overlays, and the shared types, hex math, and constants.

The game Durable Object depends on the actions module, the archive and archive-storage modules for the event stream and checkpoints, the publication pipeline module, the broadcast module, the fetch and HTTP handler and WebSocket and socket helper modules, the projection module, the match and match-archive modules, the message builders module, the alarm and session and turns and turn-timeout modules, the telemetry module, the server protocol module, and the shared game engine.

Coupling Characteristics

Turning to how tightly different parts of the system are coupled: the boundary from input to game commands is minimal — it is a pure function with no state mutation. The boundary from the coordinator to the transport layer is also minimal — a transport abstraction hides whether the connection is a WebSocket or local, and this is wired inside the client kernel. The renderer's dependency on game state is high — it reads the full state for entity positions, damage, and so on. The renderer's dependency on planning state is also high — it reads by reference for UI overlays such as previews and selections. The UI's dependency on game state is high — the HUD needs ship stats, phase, fuel, and objective data. The client's dependency on the shared engine is medium — the local transport delegates to the shared engine and types must align. Every part of the system depends on the shared types in a very-high-coupling relationship — shared types remain the integration point and all imports use bounded modules directly.

---

5. Current Decisions and Planned Shifts

The open work lives in the backlog document. This section captures current architectural stances and why they exist.

- User accounts and authentication: adding login would create friction that hurts adoption during user testing. The current anonymous token model is sufficient. Revisit for native app store distribution or payment integration.
- N-player generalisation: Delta-V is a two-player game. A fixed two-element player array is clearer and more type-safe than a variable-length array. Generalise when a second game actually needs it.
- Generic hex engine extraction: designing a framework from a single game is premature abstraction. Fork Delta-V when game number two starts and build the framework from two concrete implementations.
- Serialisation codec: the game state is plain JSON. A codec adds overhead with zero current benefit.
- Replay architecture and event sourcing: implemented on the authoritative path. Match-scoped event streams with versioned envelopes — carrying game ID, sequence number, timestamp, and actor — along with checkpoints and parity checks are all in place. Replay is projected directly from stored events, including spectator-filtered projections and authenticated replay endpoints. Live spectating uses the same filtered game state over WebSocket. Remaining work is mostly spectator UX polish — lobby links and read-only affordances — and optional rate limits and protocol simplification.

- Public leaderboard: shipped. Glicko-2 ratings are written after each rated match by the rating-writer module. The player table tracks each claimed username and the current rating; the match-rating table holds per-match before-and-after rating snapshots. Humans claim a username through the claim-name endpoint; agents claim through the agent-token endpoint with an optional claim body. Provisional players are hidden from the default view until their rating deviation shrinks and they meet a distinct-opponents threshold. The public page is the leaderboard path; the API consists of the main leaderboard query and a per-player rank lookup.
- UI framework adoption: the DOM UI layer is still small enough to own directly. The current compromise is a tiny local signals layer for view-local state and cleanup, without paying the cost of adopting a full framework across the entire client.
- Structural sharing and Immer: reconsidered with the event-sourcing shift. Immer is not a prerequisite and should not block current work. Near-term value is in event schema stability, append ordering, explicit random-number-generation facts, and projector correctness — not a wholesale rewrite. Revisit only if projector reducers or future command handlers become materially clearer with Immer; if adopted, start at the projection layer.
- Internationalization: English-only product surface for now. No message catalogs, locales, or right-to-left support until localization is prioritized. The spec document remains the canonical English rules reference for scenarios.

---

6. Client bundle and release hygiene

Bundle baseline, re-measured in April 2026 from the current client bundle: the raw size is approximately 735 kilobytes and the gzip-compressed size is approximately 155 kilobytes. Update these figures after large renderer or dependency changes.

Supply chain: run a dependency audit before releases; update dependencies judiciously and run the verify script after bumps.

Database migrations: treat as forward-only unless Cloudflare backup and restore is used. Rolling back means redeploying the previous Worker with a compatible schema — there is no automatic down-migration. The migrations directory currently holds four files: creating the events table, creating the match archive, adding match-archive listing support, and adding the leaderboard schema.

Event retention: events rows older than the retention constant — thirty days — are deleted daily by the scheduled Worker cron, which runs at four in the morning and invokes the purge-old-events helper.

Continuous integration: Node version 25 is pinned in the CI workflow configuration, and the Node version file matches.

PART 2

Design Patterns and Contracts

These chapters explain the recurring engineering choices in the system: boundaries, state flow, persistence, validation, scenarios, and testing.

Delta-V Coding Standards

The conventions that fit this codebase today. The rules and prefixes and formatting live here; the reasoning behind them lives in the pattern catalogue.

Each item is tagged as one of three levels. Required means expected for new code and refactors unless explicitly waived. Recommended means the preferred default, with room to deviate for readability, correctness, or platform constraints. Reference means background context for alignment across modules.

Core Principles

- Required: keep docs aligned with the actual implementation.
- Prefer readability over cleverness.
- Prefer a single typed options object for any API with about five or more parameters, especially public renderer and UI helpers.
- Prefer small, testable extractions over large architectural rewrites.
- Keep the shared rules engine functional and data-oriented.
- Default to functions and factory managers. Use classes only at imperative boundaries where long-lived mutable state is natural or the platform requires it.

Project Shape

Shared engine

Required for engine and rules code: side-effect-free, meaning no input/output — no DOM, no network, no storage. Use plain typed data, testable in isolation.

Turn-resolution entry points deep-clone their input state and require a mandatory random-number-generator parameter. Avoid pushing browser, network, storage, or rendering concerns into the shared module.

Imperative boundaries

Default to plain functions, typed data, and factory managers using the create-something naming shape. The game durable object is the one module that must extend the Cloudflare durable object class — that is a platform requirement, not a stylistic choice.

Everything else uses the create-something factory shape: the game client factory, the input handler factory, the UI manager factory, the renderer factory, the camera factory, the bot client factory, and so on. Returned types are derived from the factory function rather than declared separately.

Guidance:

- If an imperative boundary binds DOM events, window events, or other long-lived event listeners, own explicit teardown via a dispose method or equivalent returned disposers.
- Prefer create-something factories for new client modules. Do not add a class unless the platform requires it or a rare case genuinely needs instance-of checking.
- When the client kernel grows, extract responsibilities into game-helper modules first. Avoid inflating the kernel with unrelated logic.

DOM helpers

A dedicated DOM helper module provides declarative DOM construction. The element helper creates an element with class, text, handlers, and children in one expression. The visibility helper accepts a boolean or a reactive signal and creates an effect in the active scope for signals. The text helper and class-toggle helper similarly accept either static values or signals. There is also a typed get-by-ID helper that throws on missing elements, a listen helper that registers for auto-cleanup in the active scope, and a render-list helper that clears and re-renders a list.

All writes of raw HTML go through two dedicated helpers — one for trusted HTML and one for clearing HTML — enforced by a pre-commit check. If user-controlled HTML ever enters the client, add a sanitizer inside the trusted-HTML helper. For plain text use the text-content property or the element helper's text prop.

State ownership

There are two key state objects. Planning state is owned by the game client factory, lives in the planning module, and serves as short-lived working memory for the current turn. The renderer reads it by reference; the input pipeline receives it read-only.

Game state receives authoritative updates through a single apply function. Other modules receive it as function arguments and never store references to it.

Refactoring Guidance

Recommended by default; apply judgment per module.

- Extract pure helper modules before introducing new patterns or libraries.
- Reduce duplication first. Do not split files only to satisfy a size target.
- Keep orchestrators focused on coordination, not business logic.
- When a file grows large, split by real responsibility boundaries.
- When a stable public entry point grows too large, keep the entry file thin and re-export narrower domain modules.

Size heuristics (not hard rules)

Pure helpers are usually five to twenty-five lines. Coordinator methods can be longer if the flow is linear. Files under two hundred lines are nice when natural. Files above five hundred lines should be reviewed for extraction. Files above a thousand lines are usually overdue for decomposition.

The thousand-line threshold applies less strictly to imperative boundary orchestrators — the client kernel, the renderer, and the game durable object — that coordinate many subsystems. For those, focus on whether responsibilities are clearly separated and helper logic has been extracted.

Testing

Required: co-location, engine coverage discipline, replay and projection parity safety. Recommended: data-driven tests and property-based tests.

- Co-locate unit tests next to source files. Property tests use a distinct naming convention. Fixture data lives near its consumers.
- Keep rules-heavy logic covered with direct unit tests. Extract pure helpers from coordinators and test them.
- Keep end-to-end browser tests focused on browser-only contracts such as boot, multi-page join, reconnect, chat, and storage and session recovery. Gameplay rules and scenario combinatorics go to unit tests and simulation.
- Use data-driven tests for tables, mappings, and input-output pairs.
- Use property-based tests — the fast-check library — for invariants on core engine functions.
- Coverage thresholds on the shared module are enforced by both pre-commit hooks and continuous integration.
- Replay and projection: changes to the event projector, archive persistence, or engine state shape must keep parity tests green and extend them when adding new persisted event types.

Constants and Configuration

Required when values cross layers or affect protocol compatibility. Recommended for readability.

- Avoid magic numbers when the value is shared between client and server.
- Promote shared gameplay or protocol constants into the shared constants module.
- Keep client UI timing aligned with server-enforced timing.

Documentation

Required: update docs when behavior or architecture decisions materially change.

There is one owner document per topic: rules in the spec, wire contracts in the protocol doc, module layout in the architecture doc, pattern rationale in the patterns folder, conventions here, recurring audits in the review plan, open work in the backlog, and contributor flow in the contributing guide.

- Do not leave roadmap items marked as future once shipped.
- When a decision is referenced from multiple places, add a short anchored subsection to the most relevant doc rather than duplicating prose.

Common Patterns

Pattern rationale — with examples and tradeoffs — lives in the patterns folder. This section lists the conventions that new code should follow.

Discriminated unions

Client variants use the field named "kind" as the discriminator. Network messages use the field named "type" as the discriminator.

Always handle every variant in a switch statement — TypeScript's exhaustive checking catches missing cases.

Derive and plan (functional core, imperative shell)

Name pure functions with the prefixes derive, build, or resolve. They return a data object, a plan. A separate apply or set-state call performs the side effect. This is the functional-core, imperative-shell approach: keep computation pure and isolate mutation.

Single choke points

High-risk side effects have one owner each. The dispatch-game-command function handles client command routing. The apply-client-state-transition function handles client state-entry effects. The apply-client-game-state function handles authoritative state application. The apply-UI-visibility function handles top-level screen toggling via the UI manager. The run-publication-pipeline function handles server event append, checkpoint, parity, archive, timer, and broadcast.

Do not route the same mutation through multiple call sites.

Contract fixtures

When a shape must stay stable across modules or over time, add fixture-style tests for validated client-to-server messages, state-bearing server-to-client messages, and replay payloads.

Guard validation

Internal validation helpers return null on success or an error object on failure. The pattern is: run validation before any mutations, fail fast on known-bad state, then proceed with known-good state.

Error returns

There are two conventions for success-or-error returns.

Engine results use a union of a success shape and an error shape. Engine entry points use this style. The caller narrows by checking whether the result contains an error field.

The generic result type from the domain module wraps either a typed value or an error. Validators, parsers, and any code returning a typed value or an error use this style. The caller narrows by checking the "ok" field.

Use the engine style for game logic with heterogeneous success shapes. Use the generic result type for parse and validate operations.

Event accumulation

Engine functions collect engine events in a local array and return them alongside the result state. Events are never emitted as side effects — they accumulate and travel with the return value.

Turn-resolution entry points such as the astrogation processor, combat processor, ordnance processor, logistics processor, and others all return engine events. The server reads the events directly from the result — no server-side event derivation.

Data-driven lookup tables

Prefer declarative record tables and indexed arrays over chains of if statements or switch trees for game data such as ship stats, damage tables, ordnance mass, and detection ranges. Add new dimensions as new records in the constants module rather than encoding them into function logic.

Composable configuration objects

When behavior varies by mode — difficulty, scenario — separate scoring and decision logic from tuning weights. Define a config type and pass it to pure scoring functions.

Scenario rules as feature flags

The scenario rules object controls behavior variation across scenarios. Defaults are permissive — omitting a field enables the feature. The engine and client UI derive from the same rule helpers to avoid drift.

String-key serialization for map lookups

JavaScript's Map and Set types use reference equality, so coordinate-style value objects need string serialization. Define a key function that serializes a coordinate to a string, and a parse function to reverse it. Use the same pattern for any value-object key. Hex coordinates follow this approach throughout the engine.

Reactive signals (UI layer)

The reactive module provides signals, computed values, effects, batching, scoping, and disposal utilities. These are used selectively in the DOM UI layer and for durable session and UI state — not as a global store.

Rules:

- Use implicit scoping. Wrap UI initialization in a scope so that effects, computed values, and listeners auto-register.
- Own effects explicitly. Any view or manager that creates computed or effect graphs owns a disposal scope and exposes a dispose method.
- Keep derivation pure. Use computed for derived values, and effect for DOM writes and other side effects.
- Batch related writes. Wrap multi-signal updates in a batch when they feed the same computed or effect.
- Clone before storing a shared object reference in a signal, or pair it with a version signal.
- Keep signals local to the view they serve. Do not thread signals through the whole client graph.
- Separate durable UI state — waiting, reconnect, game-over, replay, timer — from one-shot events such as toasts, sounds, and user commands.

Dependency injection

Pure functions take direct parameters or a small typed options object. Side-effecting functions take a dependencies object as their first parameter. Callable getter dependencies — functions of the form "get something returning type T" — ensure consumers always read fresh state and break circular initialization-order dependencies. Use direct references for stable service objects.

Managers use the factory pattern: a create-something function takes a dependencies object and returns a manager. The returned methods close over the dependencies.

When adding new side-effecting logic, prefer extending an existing dependencies interface over widening the bootstrap return value. When the client decides whether an action is legal or visible, reuse shared rule helpers from the shared engine rather than duplicating UI heuristics.

Transport adapter

Network versus local game branching is hidden by the game transport interface. Action handlers call methods on the transport — for example, submit astrogation — instead of branching on whether the game is local. The local-game flag may still appear in scheduling logic for AI-turn timing but should not appear in submission logic.

Async patterns

- The AI turn loop uses async-await with a while loop, not recursive set-timeout chains.
- Wrap callbacks in promises when an animation or timer needs to be awaited.

Screen visibility

The screen-mode signal and apply-UI-visibility function inside the UI manager factory form the single choke point for screen toggling, applying the output of a pure build-screen-visibility function. This is the one place where direct style-display assignment is acceptable. Everywhere else, use the show, hide, and visible helpers from the DOM helper module.

Library adoption

Default to no new runtime library. Add one only if it clearly does one of: removes a real security risk, removes a repeated maintenance burden already being paid, or simplifies a broad class of code without hiding control flow or ownership.

Current stance: a sanitizer library such as DOMPurify is a good candidate if user-controlled or external HTML ever enters the client. A schema validation library such as Valibot or Zod is reasonable later if protocol and event schemas grow complex. Do not add React, Vue, Redux, Zustand, RxJS, XState, Immer, or rendering frameworks by default. Do not replace the reactive module just to use a library — switch only if ownership of reactive internals is no longer wanted.

New library proposals should explain why existing code is insufficient, which modules would simplify, what bundle, runtime, and test costs are introduced, and how the library fits the existing architecture boundaries.

Naming Conventions

Files, functions, types

Files use kebab-case. Functions use camelCase. Types and interfaces use PascalCase.

Use the type keyword by default for aliases, unions, intersections, and most local object shapes. Use the interface keyword only when extensibility matters — especially for exported object contracts that may be extended or declaration-merged. Never force interface for unions; keep discriminated unions as type.

Function prefix conventions

The codebase uses consistent prefixes to signal what a function does and whether it has side effects.

Derive functions compute a view or plan from state with no side effects. Build functions construct a complex object, also with no side effects. Resolve functions interpret input and produce a structured result, again with no side effects. Process functions apply game logic and return new state, cloning their input on entry. Create functions construct a new instance or manager.

Check functions detect a condition and may sometimes mutate state. Apply functions apply a transformation to state and always have side effects. Get functions retrieve or look up values with no side effects. Predicate functions beginning with "is" or "has" return booleans with no side effects.

On the client side: present functions show a result or outcome to the user. Show functions display a UI element or feedback. Render functions paint the canvas or build and update the DOM. Draw functions paint canvas overlays, icons, trails, or toasts.

Event and sequence functions: handle functions react to an event or message. Play functions trigger an animation or sequence. Move functions relocate an entity in game state. Queue functions schedule a future action or event.

Type patterns

The codebase organizes types into bounded modules: domain types for state, ships, and phases; protocol types for client-to-server and server-to-client messages; and scenario types for scenarios and rules. A barrel module re-exports all three. Import from the specific module only when emphasizing a boundary.

Use Pick with a type and a set of keys for narrow function signatures — when a helper only needs a few fields of a large interface, Pick makes it easier to test and clearer in intent.

Entities with complex state use small string-literal unions rather than multiple booleans. For example, a ship's lifecycle is one of active, landed, or destroyed; its control status is one of own, captured, or surrendered.

Derive the public type of a factory from its return type rather than declaring a separate interface. This keeps the type in sync with the implementation automatically.

Functional Style

The shared engine is data-oriented. Lean into functional patterns.

Prefer the shared utility helpers over handwritten loops when they clarify intent. The utility module provides sum-by, min-by and max-by, count, index-by, group-by, partition, compact, filter-map, unique-by, pick-by and map-values, a conditional dispatcher called cond, pattern-matching helpers, and a clamp function and random-choice function, among others.

Next, for choosing between conditional dispatch forms: use cond for multiple independent boolean conditions. Use the pattern-match-against-predicate form for the same comparison pattern applied against successive values. Use the strict-equality dispatch form for equality dispatch with an explicit default. Use a switch statement for discriminated-union narrowing where TypeScript's exhaustive check catches misses. Use a ternary for simple two-branch expressions.

Beyond that:

- Prefer expressions over statements. Filter then map is clearer than a loop pushing to an array.
- Avoid mutable accumulators when a helper already captures the pattern.
- Prefer filter-map over separate map and filter calls when transforming and discarding nulls.
- Prefer the count helper over filtering and reading the length — it avoids allocating an intermediate array.
- Build lookup structures declaratively — for example, index orders by ship ID — rather than with manual loops.
- Do not force it. Imperative code is fine for inherently stateful logic such as tracking previous iteration, complex early-exit conditions, or canvas drawing.
- Prefer arrow functions over function declarations.

Linting

Required: treat linter and type-checker failures as blockers.

The Biome linter enforces a range of rules as errors rather than warnings, including: use-const, no-var, no-double-equals, use-arrow-function, no-for-each, use-flat-map, no-unused-imports, no-explicit-any, no-unused-variables, use-template, and no-non-null-assertion, among others. Exceptions for specific environments — for example, Cloudflare globals in the server module — are configured in the Biome config file.

Continuous integration and pre-commit hooks both run the linter and the type checker. The verify script runs the same pipeline locally.

Type checking is split intentionally. One TypeScript config checks application code. A second config checks tooling such as scripts, end-to-end tests, and root config files. Use the app-code typecheck command for application code and the all-inclusive typecheck command before pushing.

Formatting

Recommended: consistency over rigid rule-lawyering.

- Keep line width under eighty characters where practical. Break at natural points such as commas, operators, and arrow functions. Some lines will be longer — that is fine if breaking hurts readability.
- Use generous whitespace — blank lines between methods, top-level declarations, after import blocks, around loops, before return statements following logic, and between distinct logical steps. The linter collapses consecutive blank lines to one.
- For long function signatures, prefer a single typed options object at roughly five or more parameters. Otherwise put each parameter on its own line when the line exceeds roughly eighty characters.
- For long object or array literals, put one property or element per line.
- Break long conditional expressions and ternaries across lines.
- For long method chains, put one method call per line.

Practical Style

Recommended defaults for day-to-day authoring.

- Use descriptive names over abbreviations unless the abbreviation is standard.
- Use comments sparingly — only where they explain non-obvious intent.
- Prefer direct control flow over abstract indirection.
- Accompany behavior changes with tests, or provide a clear rationale where tests are not practical.
- Use for-of loops over forEach on arrays when you own the body. For index access, use the entries form of for-of.

- Prefer filter-map over separate map and filter calls when discarding nulls.
- Use the typed get-by-ID helper rather than the native get-element-by-ID method — it throws with a clear error and avoids non-null assertions.

Delta-V Pattern Catalogue

A guided tour of the patterns that hold Delta-V together. Read these when you want to understand why the code looks the way it does — not just the rules or the module inventory, but the recurring design choices and their tradeoffs.

Each chapter is self-contained. Start wherever the question is.

The catalogue is organized into six chapters. The first chapter, Engine and Architecture, answers questions like how event sourcing works, why the engine is side-effect-free, how layer boundaries are enforced, and where random-number generation comes from. The second chapter, Client, covers how input becomes game state, what state lives where, how the canvas renderer stays fast, and what replaces a user-interface framework. The third chapter, Protocol and Persistence, explains how events are stored and replayed, what shape each WebSocket message takes, how viewer-aware filtering works, and how Durable Object hibernation survives disconnects. The fourth chapter, Type System and Validation, explores what a branded type is, where input validation happens, how error codes travel, and how abuse is rate-limited. The fifth chapter, Scenarios and Config, describes how AI difficulty actually varies, how scenarios are defined declaratively, what the scenario-rules flags do, and how the solar-system map is built. The sixth and final chapter, Testing, covers where tests live, when property-based tests are used, how Durable Objects are mocked, and what coverage is enforced.

Each chapter follows the same structure. First, it describes what the pattern is — a short description of its shape. Next, it explains where it lives, pointing to the canonical implementation. Finally, it covers why the pattern is shaped that way, laying out the tradeoffs behind the choice. Actionable follow-ups live in the backlog document rather than inside the chapters themselves.

Engine & Architecture Patterns

How the authoritative server stays consistent and why the shared engine has the shape it does. The architecture documentation covers the high-level layer diagrams and module inventory; this chapter zooms into the recurring patterns behind those layers.

Each section has four parts: the pattern, a minimal example, where it lives, and why this shape.

Event-Sourced Match State

Pattern. Every mutation to an in-progress match is a versioned event appended to a per-match stream. Live state, replays, and reconnection are all projections of that stream plus periodic checkpoints — never a separate "current state" slot maintained by hand.

The flow works like this: a client-to-server action arrives at the engine, which is a pure function that returns events alongside updated state. That result passes through the publication pipeline, which appends events to a chunked stream, saves a checkpoint at turn boundaries, runs a parity check comparing the live result to the projected result, and then broadcasts filtered server-to-client messages.

Recovery follows the same shape in reverse: the system loads the latest checkpoint, loads the event tail since that checkpoint, and projects them together to arrive at current state.

Minimal example. The engine processes an astrogation action and returns a result containing state, movement data, and a list of engine events. The server then calls the publication pipeline with that result, attaching metadata like a game identifier, a sequence number, a timestamp, and the acting player to each event envelope.

Where it lives. The engine events module in the shared engine defines a 32-member union type covering every kind of engine event. The archive module on the server handles the append-only stream, grouping events into chunks of 64 per storage key. The event projector sub-package projects a stream back to state. The publication module is the single writer that coordinates all of this.

Why this shape.

- Recovery is free — any observer who can read the stream can reconstruct the game. No drift between live state and what was saved.
- Replays use the same code path as live play, filtered for the viewer. There is no second implementation.
- Chunked storage at 64 events per key keeps individual storage values well under the size limit while handling the hundred to three hundred events a typical match produces in a handful of writes.
- Checkpoints at turn boundaries reduce projection cost — reconnecting mid-match reads the latest checkpoint plus a short tail, not the whole history.

Parity Check Between Live and Projected State

Pattern. After every incremental publication, the server reconstructs state from checkpoint plus event tail — the same code path used for reconnection — and compares it to the live in-memory result. Any mismatch is logged to telemetry but does not halt the match.

Where it lives. The publication module contains the parity verification function. The telemetry module records a projection parity mismatch event when a discrepancy is found. A dedicated projection module handles the reconstruction logic.

Why this shape.

- Replay correctness is the core invariant of the whole event-sourcing design. If the live result and the projected result differ, the stream is misrepresenting the match.
- The check is observability-only, not fatal — a parity bug should not take matches offline. The log-and-move-on approach accepts some correctness risk in exchange for availability while alerts are firing.

Side-Effect-Free Shared Engine

Pattern. Everything in the shared layer has zero input or output: no Document Object Model access, no network calls, no storage, no random number generation, no console logging. Turn-resolution entry points — such as those handling astrogation, combat, and ordnance — deep-clone their input state on entry, then mutate the clone internally for speed. The caller's state is never touched.

Minimal example. Inside the combat processor, the first thing that happens is a deep clone of the incoming state. All mutations, such as applying damage to a ship, happen on the clone. The function then returns the cloned state along with engine events and results. The caller must use the returned state — their original reference is unchanged.

Where it lives. All files in the shared engine directory follow this rule. It is enforced by a clone-on-entry test file and by pre-commit checks that reject any use of the global random function, inner HTML assignment, or console logging.

Why this shape.

- Rollback safety — if the engine throws mid-turn, the server's real state is untouched.
- Speculative branching — artificial intelligence search and projection verification can invoke engine functions freely without defensive cloning.
- Testability — every engine call is a pure function given a random number generator. Property-based fuzzing is straightforward.

Deterministic RNG via Per-Match Seed

Pattern. The server generates a 32-bit seed per match using the cryptographic random values API, persists it, and emits it in the game-created event. Before each engine call, a derive-action-RNG function derives a deterministic pseudo-random number generator from the match seed and the current event sequence number. This means replaying events from position N to M reproduces the same randomness without having to replay events from the beginning up to N. Providing a random number generator is mandatory on turn-resolution entry points.

Minimal example. For each action, the server derives a random number generator from the match seed and the state's event sequence number. Passing that same state, sequence number, and match seed always produces an identical result — every time.

Where it lives. The shared pseudo-random number generator module provides two functions: a Mulberry 32 generator and the derive-action-RNG function. The server actions module provides a helper that retrieves the action RNG. All engine entry points accept a random number generator as a required parameter typed as a function returning a number.

Why this shape.

- Replay determinism — the event stream alone is enough to verify history offline.
- Jumpable RNG — a Knuth multiplicative hash step means the system does not need to replay events one through N-minus-one to derive the random state for the Nth action.
- Injectable in tests — passing a fixed value or a seeded sequence pins outcomes reliably.

Single Choke Points for Side Effects

Pattern. Where a side-effecting domain has an obvious owner, one function owns it. Instead of many call sites each doing a small piece of "persist plus broadcast plus schedule," a single applier function is the only path.

Where it lives. There are six of these choke points across the system. On the server, one function owns action execution and another owns state publication — handling event appending, checkpointing, parity checking, archiving, timer restarting, and broadcasting all in one place. On the client, one function owns command dispatch, another owns authoritative state application, another owns state-transition side effects, and a final one owns UI visibility changes.

Why this shape.

- Drift between similar flows is the main cost of duplication. If five call sites each "save state and broadcast," one of them will eventually forget to restart the turn timer.
- Tests get a narrow seam — asserting that the whole publication pipeline fired requires stubbing only one collaborator.

Composition Root for Client Construction

Pattern. One function wires every collaborator and returns an object with narrow exports: dispose, renderer, and show-toast. No module constructs its own dependencies; they come in through dependency objects typed as callable getters.

Minimal example. A combat actions factory receives a dependencies object containing a get-game-state getter, a get-player-ID getter, a stable user interface reference, and a show-toast function. The getter pattern — calling get-game-state as a function rather than holding a direct reference — ensures collaborators always read the current state. The factory for combat actions is then created from those dependencies.

Where it lives. The client kernel module contains the game client factory function, which is the composition root. Input handler, UI manager, renderer, camera, bot client, and the various action factories all accept their dependencies at construction time.

Why this shape.

- Factories over classes make testing trivial — pass mock dependencies and inspect the returned methods.
- Callable getters ensure collaborators always read the current state and let the system break circular initialization-order dependencies without requiring a Proxy.
- The kernel is the only place where which module talks to which is visible. Collaborators do not reach for globals.

Layer Boundaries (shared, server, client)

Pattern. Three top-level source directories operate under strict import rules. The shared layer imports only from shared. The server layer imports from shared and server, never from client. The client layer imports from shared and client, never from server.

Where it lives. Two import-boundary test files — one for the server and one for shared — enforce the directional rules at test time. Pre-commit checks enforce the sub-rule that shared code contains no input or output.

Why this shape.

- The shared engine is the contract between client and server. If one side pulls in the other's code, the boundary blurs and replay and parity break silently.
- Running the engine in Node for simulation or in the browser for local AI games uses the exact same code.

Cross-Cutting Theme: Initial Game Creation Is the Outlier

Five of the patterns on this page note that initial match setup goes through a separate code path from incremental actions. It does not use the publication pipeline. It does not thread the match seed into the game creation function. It emits the game-created and fugitive-designated events outside the normal return path. And its reproducibility is only partial.

Consolidating initial creation into the same pipeline as every other action would close all five gaps at once. This is the single highest-leverage architecture refactor available in the codebase today.

Protocol & Persistence Patterns

How WebSocket actions turn into persisted events and back into broadcast state. A separate architecture document shows the high-level action path; this chapter zooms into the patterns that make replay, reconnection, and spectating correct.

Each section covers the pattern, a minimal example, where it lives, and why this shape.

Chunked Event Storage

Pattern. Per-match events live in fixed-size chunks of 64 events per storage key, not one key per event and not one giant key per match. New events append to the current chunk; the chunk count and sequence counter update together in a single atomic storage write.

The storage layout for a given match uses separate keys for each chunk of events, a key tracking the total number of chunks, and a key tracking the current sequence number. For example, a match that has accumulated 128 events would have two chunks of 64, a chunk count of 2, and a sequence number of 128.

Where it lives. The archive storage module defines the chunk size and provides read and write helpers. A separate archive module builds the event envelopes. A compatibility module lazily migrates legacy single-key event streams on first access.

Why this shape.

- **Value-size ceiling.** Durable Object storage values max out at 128 kilobytes. A typical match produces 100 to 300 events, which works out to roughly 6 to 32 kilobytes per chunk — well inside the limit.
- **Atomic appends.** Writing the modified chunk, the chunk count, and the sequence number in a single storage call prevents split-brain state after a crash.
- **Fast tail reads.** A simple division tells the server which chunk to start from when replaying events for a reconnecting client — no need to scan the whole stream from the beginning.

Event Envelopes

Pattern. Raw engine events are wrapped in envelopes before persistence. Envelopes carry identity metadata — the game identifier, the sequence number, a timestamp, and the actor — separate from the event payload. The engine never knows about envelopes; the publication pipeline wraps them.

An envelope holds the stable per-match game identifier, a sequence number that counts monotonically within that match, a server timestamp, the actor (either a player identifier or the string "server"), and the underlying engine event, which can be one of 32 domain event shapes.

Where it lives. The envelope type is defined in the shared engine events module. Envelopes are constructed in the archive module. They are consumed by the event projector and by replay timeline code.

Why this shape.

- **The sequence number is authoritative ordering.** Timestamps can drift; sequence numbers do not.
- **The game identifier is stable even across room reuse.** A single room code can host multiple matches — for example, a rematch gets a new suffix — so the game identifier is what disambiguates which match an event belongs to.
- **The actor field enables per-player provenance** for replay viewers and anti-cheat audits without having to parse the event body.

Checkpoints at Turn Boundaries

Pattern. After every turn-advanced or game-over event, the server saves a full game state snapshot. Reconstruction loads the latest checkpoint plus the event tail after it — not the whole stream from the beginning.

Where it lives. The publication module contains the checkpoint logic, and snapshots are keyed by game identifier and turn number. Schema migration runs on both save and load to handle evolving state shapes.

Why this shape.

- **Bounded projection cost.** Without checkpoints, reconstructing a 50-turn match requires projecting all 50 turns' worth of events. With per-turn checkpoints, reconstruction only needs at most one turn's worth of tail events.
- **Turn boundaries, not every event.** Checkpointing on every event would multiply storage writes by roughly ten with no recovery benefit — a turn's events are a natural atomic unit.
- **Non-atomic with event append is benign.** A crash between the checkpoint save and the event append leaves a stale checkpoint, but the event tail covers the gap. Correctness never depends on both writes committing together.

Publication Pipeline (Single Writer)

Pattern. Every state-changing action runs through one function that appends events, checkpoints at turn boundaries, verifies parity, writes the match archive on game-over, restarts the turn timer, and broadcasts. Not six separate helpers — one pipeline.

Calling the publication pipeline with a result from an engine call is equivalent to running all of these steps in sequence: appending events, optionally checkpointing, verifying projection parity, optionally archiving, restarting the turn timer, and broadcasting the result.

Where it lives. The publication module is called by the game Durable Object's action handler for client-to-server actions, and by the turn timeout handler for alarm-driven timeouts.

Why this shape.

- Any new action automatically gets event persistence, checkpoint cadence, parity verification, archive-on-end, timer management, and broadcasting — for free. There is no way to forget a step.
- Tests stub one collaborator at a time — storage, broadcast, or archive — and assert on the whole pipeline's behavior.

Discriminated Union Messages (Client-to-Server and Server-to-Client)

Pattern. All WebSocket messages are TypeScript discriminated unions keyed by a type field. Runtime validation produces typed values; TypeScript's exhaustive checking catches missing handlers.

The client-to-server union has variants for actions like submitting astrogation orders, submitting combat attacks, and skipping ordnance. A dispatch map on the server side maps each action type to its handler, and TypeScript enforces that every variant has a corresponding handler at compile time. A separate auxiliary message type covers non-state-mutating traffic like chat, pings, and rematch requests.

Where it lives. The message types are defined in the shared protocol types module. Runtime validation lives in the shared protocol module. The dispatch map lives in the game Durable Object's actions module.

Why this shape.

- **Single source of truth.** Client and server share the union; neither can send a message shape the other does not understand.
- **Compile-time exhaustiveness.** Adding a new client-to-server variant fails to compile on the server until a handler is added.
- **Clean rate-limit boundary.** The auxiliary message type lets the Durable Object route chat and pings separately from game-state actions without re-parsing the message.

Single State-Bearing Message per Action

Pattern. Every state-mutating action produces exactly one server-to-client message that carries the full updated game state. Clients replace their state wholesale — no delta patching, no optimistic updates.

When the server processes a movement action, it broadcasts one message containing the movements, the events, and the complete new game state. The client receives that message and replaces its local state entirely. The game-over message is intentionally separate and sent after the final state-bearing message.

Where it lives. The stateful server message union in the shared protocol types covers game start, state update, movement result, combat result, and single combat result. Message builders live in the game Durable Object's message builders module.

Why this shape.

- **Reconnection is trivial.** A returning client gets one state update message — no need to replay a sequence of deltas.
- **No ordering bugs.** With delta patching, mis-ordered or dropped frames desynchronize the client from the server. With wholesale state replacement, the latest message is always the truth.
- **Bandwidth cost is acceptable.** A typical game state is a few kilobytes and messages are turn-paced.

Viewer-Aware Filtering

Pattern. Before any state goes to a socket, it passes through a filter function that strips hidden information per viewer. Players keep full visibility into their own state; unrevealed enemy ships have their identity stripped; spectators receive a spectator-safe projection.

When broadcasting, the server iterates over all open WebSockets. For each socket it retrieves the viewer identifier — which is zero, one, or the string "spectator" — and sends a version of the message with the state filtered for that specific viewer.

Where it lives. The filter function lives in the shared movement resolution module. The broadcast function that applies it lives in the game Durable Object's broadcast module. Sockets are tagged with their viewer identity at the time of the WebSocket upgrade handshake.

Why this shape.

- **Server-authoritative hidden state.** In the Escape scenario, the fugitive ship's identity field is stripped before the message reaches any opponent's socket — the filter runs on the server, not the client.
- **Early-return optimization.** When no ship has identity fields and the scenario does not use hidden-identity rules, the filter returns the original state by reference with no allocation.
- **Same filter for replays and live spectators.** A spectator-tagged socket and a replay timeline both pass through the same filter code. There is no parallel "filter for replay" implementation to keep in sync.

Hibernatable WebSocket

Pattern. The Durable Object uses Cloudflare's hibernation API instead of classic event listeners. The Durable Object can hibernate between messages — no in-memory state survives a wake.

When upgrading a connection, the server accepts the WebSocket with a tag identifying the player. Later, when the Durable Object wakes on an incoming message, it recovers the player's tags from the socket itself and reloads game state from storage rather than from memory.

Where it lives. The main game Durable Object module handles the accept call and the message and close callbacks. Supporting modules handle hibernation entry points, the welcome and reconnect flow, and the disconnect grace period.

Why this shape.

- **Cost.** Hibernation lets a Durable Object idle without tearing down its sockets. Alarm-driven wake-ups stay cheap.
- **Tags replace an in-memory socket map.** Looking up sockets by tag eliminates the need to maintain a separate map from player identifier to WebSocket that could fall out of sync.
- **Per-socket data survives wake cycles.** Cloudflare preserves WebSocket objects across hibernation, so per-socket rate limits and replaced-socket tracking remain valid within a wake.

Single-Alarm Multi-Deadline Scheduling

Pattern. One alarm per Durable Object, rescheduled after each state change. Three independent deadlines — a 30-second disconnect grace period, a 2-minute turn timeout, and a 5-minute inactivity timeout — are tracked in storage. A helper computes the nearest deadline; a discriminated action type tells the alarm handler which deadline fired.

Where it lives. The alarm module contains the logic that resolves which action fired, returning one of three variants: disconnect expired, turn timeout, or inactivity timeout. The disconnect grace constant is defined in the session module. The inactivity timestamp is cached in memory and flushed to storage at most once every 60 seconds to avoid write amplification from frequent pings.

Why this shape.

- **Single alarm slot.** Durable Objects have one alarm. Storing three deadlines and taking the minimum is the only clean way to multiplex them.
- **Discriminated action type.** The handler dispatches with an exhaustive switch statement; there is no guesswork about which deadline fired.
- **Flush throttling.** Inactivity pings happen often; each one does not need a storage write. An in-memory cache with a 60-second flush keeps writes bounded.

Cross-Pattern Flow

A single client-to-server action threads all of these patterns in order.

When a client sends a WebSocket message, the Durable Object wakes from hibernation and uses the socket's tags to identify the player. The message passes through rate limiting and runtime validation. The game state action handler calls the engine and gets back a list of engine events.

Next, the publication pipeline takes over. It appends the events to chunked storage. If the action crossed a turn boundary, it saves a checkpoint. It verifies projection parity. If the game ended, it writes the match archive to long-term storage. It restarts the turn timer. Finally, it broadcasts the result — applying viewer-aware filtering per socket before sending.

Every step has a single owner, a single reason to exist, and a single place to look when debugging.

Client Patterns

How the framework-free browser client stays coherent. A separate coding standards document covers naming conventions, function prefix tables, factory patterns, and the small DOM helper API. This chapter walks through the patterns that tie input, state, rendering, and the DOM together.

Each section follows the same structure: the pattern, where it lives, and why this shape.

Three-Layer Input Pipeline

Pattern. DOM events never reach game logic. They pass through three layers: raw capture, then game interpretation, then command dispatch.

Picture the flow as a left-to-right pipeline. Raw browser events — clicks, key presses, touch events — enter at the far left. Layer one, the input module, handles camera transforms and converts screen coordinates into meaningful game positions like a hex tile identifier, but it knows nothing about game rules. Layer two is a pure function called interpret-input that takes a snapshot of the current event, the current phase, and the current state, and returns a list of game commands. Layer three, the command router, receives those commands and dispatches them to the appropriate domain handler — astrogation, combat, ordnance, or UI.

Keyboard shortcuts and button clicks skip layers one and two entirely and enter directly at layer three, sharing the same dispatch point. This is a sibling path, not a violation of the design.

Why this shape.

- Layer two being a pure function means every combination of phase, state, and input can be tested without a browser.
- The handler map in layer three is checked at compile time: adding a new game command variant will fail to compile until a corresponding handler exists.
- Raw DOM and game rules never share a scope, so a hover event cannot accidentally alter a ship's velocity.

Client State Machine

Pattern. A flat set of named states describes where the UI is at any moment. Transitions are derived from the authoritative game state received from the server — they are never imperatively set. State-entry side effects all flow through a single applier function.

The full set of named states includes menu, connecting, waiting for opponent, several playing sub-states covering astrogation, ordnance, combat, logistics, movement animation, and opponent turn, plus game over.

The derivation works like this: given the current client state and the latest game state, a pure function produces the next client state. A single applier then owns updating the screen and triggering any side effects.

Why this shape.

- Derivation means the client state cannot diverge from the server state. The server sends game state; the client computes its view of it.
- Exhaustive records and switch statements catch missing cases at compile time.
- A single applier is the only place that mutates stored state, starts timers, resets cameras, or triggers tutorials — so state-entry effects cannot scatter across the codebase.

Reactive Signals (Zero-Dependency)

Pattern. A small, zero-dependency signals library — roughly two hundred lines — provides the primitives: signal, computed, effect, and batch, along with disposal scopes. This library owns durable UI state where it removes the need for imperative fan-out. Transient events like toasts and sounds remain imperative.

A signal holds a value. A computed derives a value from one or more signals and updates automatically. An effect runs a side-effecting function whenever its tracked signals change. Disposal scopes manage teardown.

The DOM helper functions for visibility, text content, and CSS classes accept signals directly, so they update automatically without manual wiring.

Why this shape.

- Signals eliminate the class of bug where the heads-up display forgot to re-render when the turn changed, without taking on a framework like React.
- Effects record which signals they read automatically — there is no manual subscription list to maintain.
- Teardown is last-in, first-out and automatic inside any scoped block.

Session Model as Aggregate Root

Pattern. One client session object owns every piece of durable client state. Reactive fields are defined so that a plain assignment transparently updates both the underlying field and its companion signal.

Every reactive field exists in two forms: a plain field you can assign to directly, and a read-only signal you can subscribe to reactively. Sub-stores for planning and logistics are owned by the session object, not shared across the codebase.

Narrow view types — each exposing only a specific slice of the session — limit what each collaborator can reach.

Why this shape.

- One root means one place to inspect in tests and one place to handle teardown.
- Reactive properties via property descriptor give imperative call-site syntax with reactive semantics, avoiding setter boilerplate.
- Narrowed context types act as compile-time access control: modules only see the fields they genuinely need.

Planning Store (Ephemeral Turn State)

Pattern. All uncommitted UI planning — burns, overloads, queued attacks, selections, hover state — lives in one store. A single revision signal bumps on every mutation. Phase transitions reset the relevant sub-state.

When a player sets a burn for a ship, the planning store records it and increments its revision signal. The heads-up display is built as a computed value that reads the revision signal, so it automatically re-derives whenever any planning change occurs.

The renderer holds the read-only planning state by reference, which means previews such as dashed courses, ghost ships, and fuel cost labels are always current without a setter API.

Why this shape.

- A coarse-grained revision signal is sufficient because planning changes cluster in time and re-derivation of the heads-up display is cheap.
- The split between a read-only planning type and a mutable planning store means the input pipeline cannot accidentally mutate planning state — TypeScript enforces this at compile time.

Canvas Renderer Factory

Pattern. A factory function called create-renderer composes roughly seventeen drawing modules and runs a request-animation-frame loop. A static scene layer — covering stars, the hex grid, gravity wells, asteroids, and celestial bodies — caches to an offscreen canvas keyed by camera position, zoom level, viewport dimensions, a coarse body-animation bucket, and the set of destroyed asteroids. This avoids redrawing thousands of hex tiles on every frame when the camera is idle.

The renderer is composed of distinct modules: one for the scene, one for ships, one for entities, one for vector overlays, one for effects, one for the overlay, one for the minimap, one for the static scene and its caching layer, one for animations, and one for the camera.

Why this shape.

- Using a factory rather than a class means the renderer owns mutable long-lived state without requiring inheritance or instance checks. The public type is inferred directly from the factory's return value, so it stays in sync with the implementation automatically.
- Separating view computation from Canvas drawing — building a body view first, then rendering it — makes view logic testable without a Canvas context.

Animation Manager with Completion Guarantees

Pattern. Ship and ordnance movement animations have explicit completion paths: a normal finish, a fallback timer set to the animation duration plus five hundred milliseconds, and an immediate skip when the browser tab is hidden. State is cleared before the completion callback fires, so the callback cannot observe mid-animation state.

The animation manager accepts injected dependencies for the clock and timer functions, which allows tests to advance time deterministically without real timers.

Why this shape.

- Game state is already in its post-movement form before the animation starts. The animation is purely cosmetic interpolation and cannot block the game engine.
- Three completion paths mean a network hiccup or a hidden tab will not leave the UI frozen in an animating state.

Trusted HTML Boundary

Pattern. All writes to inner HTML go through two dedicated helper functions in the DOM module. Nothing else in the codebase may write to inner HTML directly. This constraint is enforced by a pre-commit check.

All current callers pass internally generated markup — derived from game state, static strings, or computed display values — and nothing user-controlled.

Why this shape.

- If user-controlled content ever needs to render as HTML — for example, chat messages, player names, or modded scenario descriptions — a single boundary can add a sanitizer. If inner HTML writes were scattered throughout the codebase, that audit would be impossible.
- The pre-commit check keeps the boundary from quietly re-opening during refactors.

Disposal Scopes

Pattern. Anything with a lifetime — effects, computed values, DOM listeners, timers — registers with a disposal scope. Scopes dispose in last-in, first-out order. A second scope can be nested inside a first. A scope helper function implicitly registers anything created inside a given block.

When the scope is disposed, everything registered with it tears down in reverse order of registration. Every view and manager that creates computed or effect graphs owns a scope and exposes a dispose method.

Why this shape.

- Central scope registration means every module has one teardown path, eliminating the class of bug where a listener was never removed.
- On effect re-runs, previously registered nested disposers run first, so subscriptions do not stack.
- Last-in, first-out disposal handles child-before-parent teardown order correctly.

Transport Adapter (WebSocket vs Local AI)

Pattern. A game transport interface hides whether the current game is multiplayer over a WebSocket or a local game against an AI. Action handlers call transport methods like submit-astrogation — they never branch on whether this is a local game inside the submission logic itself.

Two factory functions implement the interface: one wraps a WebSocket connection, and the other runs the game engine directly in-process.

Why this shape.

- Action code is identical for multiplayer and single-player because the engine is the same either way.
- Adding a new mode — such as a Model Context Protocol-driven local session — only requires a new transport factory, leaving all existing action code untouched.

Cross-Cutting Theme: Pure Core, Imperative Shell

Several of the patterns in this chapter share a common shape: a pure function computes what should happen and returns it as data; a thin imperative layer performs the actual side effect. This is the functional-core, imperative-shell pattern, applied consistently across the client.

The main advantage is testability. A phase-transition derivation test does not need a DOM, and a screen-visibility test does not need a browser. The main discipline is keeping the pure core truly pure: the moment a planner reaches for the document object or a timer, the imperative shell has leaked inward.

Type System & Validation Patterns

How Delta-V keeps bad data out of the engine and bad requests out of the server. The coding standards document covers general TypeScript style; this chapter walks through the project-specific techniques.

Each section covers the pattern, a minimal example, where it lives, and why this shape.

Branded Types for Nominal Typing

Raw string values that mean different things — hex keys, room codes, player tokens — are branded at the type level so they cannot be substituted for each other. The brand is a phantom property, meaning it carries zero runtime cost.

In practice, there are two ways to build a branded value. The structured constructor takes a typed coordinate and produces a hex key from it. The unsafe cast converts any plain string into a branded type, and this is used only at serialization boundaries. TypeScript then prevents mix-ups: passing a plain string where a hex key is expected is a compile-time error, while passing a properly constructed hex key is fine.

The hex key type lives in the hex utility module in the shared folder. Room codes, player tokens, game IDs, agent tokens, and match tokens all live in the shared IDs module. That module also provides runtime guards for checking whether a value is a valid room code or player token, along with normalization functions.

The reasons for this shape are threefold. First, branded types vanish at runtime, so there is zero overhead. Second, using two distinct function names — one for the structured constructor, one for the unsafe cast — tells reviewers at a glance which invariants hold. Third, before branding, a function accepting any string could be called with anything; after branding, only the correct kind of string passes.

Multi-Stage Validation Pipeline

Incoming client messages pass through four distinct validation stages, each with its own error surface. Rate limits come first so flood attacks cannot exhaust the later, more expensive stages; the engine is last and trusts its input by that point.

The flow moves as follows. A WebSocket message arrives and immediately hits the rate-limit check. If the rate is exceeded, the socket is closed. Otherwise the message proceeds to JSON parsing. If parsing fails, a structured error is returned. If parsing succeeds, the now-typed but unvalidated message moves to shape and size validation. If the shape is wrong or the payload is too large, another structured error is returned and an error message is sent to the client. If validation passes, the message moves into the engine as a

typed client-to-server value. The engine can reject the action for rule violations — wrong phase, wrong turn, resource limits — returning an engine failure with an error code. On success, the engine produces a new state and a list of events.

Stage zero, the rate-limit check, lives in the socket module's rate-limit function. Stage one, JSON parsing, is in the same socket module. Stage two, shape and size validation, lives in the shared protocol module. Stage three, engine calls, lives in the engine entry points under the shared engine folder. A runner in the server actions module wraps engine calls in a try-catch so that unexpected throws become typed errors rather than state corruption.

Three design reasons explain this shape. First, failing early and cheaply means flood detection and JSON parsing happen at the byte level before spending time on shape checks. Second, each stage owns its error type: stage two returns a result with a clear invalid-shape message, while stage three returns an engine failure with a code. Third, size limits are enforced at stage two — maximum fleet purchases, maximum astrogation orders, maximum ordnance launches, maximum combat attacks — so no engine code has to defensively check array sizes.

Error Code Enum

A closed string enum carries the category of a runtime error across the protocol boundary. The client decides what to do with it; the server simply names the category.

The enum covers timing errors such as invalid phase and not-your-turn, reference errors such as invalid ship and invalid target, input errors, authorization errors, resource limit errors, and consistency errors. A server-to-client error message carries the error code alongside a human-readable message string.

The enum lives in the shared domain types module. It is attached to the server-to-client error message type in the shared protocol types module. The engine returns it inside an engine failure value.

Three reasons justify this shape. First, string values survive serialization cleanly and read well in telemetry dashboards. Second, the client can branch on specific codes: a state-conflict code means the client should re-read state and retry, while an invalid-input code means the interface sent something wrong and should surface a different message. Third, the code field is optional, so errors originating outside the engine — rate limits, server internals — do not need to invent a code.

Rate Limiting

Multiple rate-limit layers operate at different scopes. An edge or Worker layer caps requests per IP address; a Durable Object layer caps per socket and per player. Each layer has a tight, specific reason to exist.

At the socket level, there is a hard cap on messages per second. Exceeding it closes the socket with a standard protocol-violation close code. At the player level, there is a soft throttle on chat messages: a minimum gap must elapse between messages, and messages sent within that window are silently dropped.

Canonical values for all limits are documented in the security documentation. Constants for the Worker-level per-IP limits live in the server reporting module; constants for the Durable Object-level per-socket and per-player limits live in the socket module.

Three design reasons support this layered approach. First, each layer catches a different class of attack: per-IP Worker limits catch mass room-creation or probe scanning, per-socket Durable Object limits catch a single connection flooding, and the per-player chat throttle catches spam from a legitimately authenticated socket. Second, the rate-limit function takes the current timestamp as a parameter and stores its state in a weak map keyed by WebSocket, making it deterministic and straightforward to test. Third, the soft-versus-hard distinction is intentional: chat messages are silently dropped so the user simply retries, while a socket flood triggers a hard close.

Result and Engine-Style Returns

Two parallel conventions handle the success-or-error case depending on context. The generic result type is the workhorse for parsing, validation, and lookups. Engine-style union returns are specialized for engine calls where success can carry several heterogeneous fields alongside the new state.

With the generic result type, you check whether the result succeeded, and if not you send the error and return early. If it succeeded, you dispatch the validated value. With the engine-style union, you check whether the result contains an error field; if it does, you return the error; if it does not, you read the new state — and potentially also movements, engine events, or transfer log entries that vary by action type.

The generic result type lives in the shared domain types module. Engine-style return shapes are defined per entry point in the shared engine folder.

Three considerations explain this design. First, the two shapes exist because of two distinct patterns: validators return a single success value, while engine entry points return state plus per-action extras that differ across actions. Unifying them would force every engine call into one enormous success type. Second, neither convention throws for validation failures — callers receive structured errors and can decide how to respond. Only the outer try-catch in the action runner catches unexpected bugs. Third, there is an implicit invariant: the narrowing check for the engine-style union works because no success shape contains an error field. Adding one would silently break narrowing, and a more explicit discriminant would be safer if the types grow.

Cross-Pattern Flow

A single client-to-server message threads all these patterns in order.

First, the Worker-layer per-IP rate limits apply if the message arrives over an HTTP path. Second, the socket flood rate limit runs as stage zero. Third, JSON parsing produces an untyped result. Fourth, shape and size validation applies branded types and produces a typed client-to-server value. Fifth, the engine call runs against a cloned state with a random number generator, and either produces a list of engine events plus a new state, or returns an engine failure with an error code. Sixth, a server-to-client response is sent — either a state-bearing success or an error object carrying a code and message.

Each layer can reject independently. A developer tracing a bug from a wrong error surfacing to the user can walk the pipeline in one direction and find exactly which stage produced the unexpected result.

Scenarios & Config Patterns

How Delta-V varies behavior across scenarios and difficulty levels without branching engine code. The spec document describes the nine shipped scenarios; this chapter walks through the config patterns that drive their differences.

Each section covers the pattern, a minimal example of how it works, where in the codebase it lives, and why it takes this shape.

AI Config as Weights, Not Code

The AI does not use conditional branches to check whether the difficulty is hard or easy. Instead, a flat record of roughly sixty numeric weights and boolean flags drives pure scoring functions. Difficulty presets and per-scenario overrides adjust the weights; the logic itself stays identical across all difficulties.

The config type is pure data, not a class hierarchy. It holds fields like a global scaling multiplier, an escape-distance weight, a combat-closing weight, a combat-close bonus, and a flag for whether the AI may fire only a single weapon per turn, along with sixty or so additional fields.

The scoring side is a pure function that takes a ship, a candidate course, the map, and a config object and returns a numeric score. That total score is composed of sub-scores for navigation, escape, race danger, gravity look-ahead, and combat positioning — each sub-function also receiving the config so weights feed in uniformly.

The orchestration layer enumerates all seven possible burn options for a ship, scores each with the same function, and selects the one with the highest score.

The config types and presets live in the AI config module. The scoring functions live in the AI scoring module. The orchestration entry point lives in the AI index module. Per-phase decisions for astrogation, combat, ordnance, and logistics each live in their own AI sub-modules.

Why this shape.

- Adding a new scoring dimension means adding a new function and a new config key. No existing code changes. This is the Strategy pattern expressed as data rather than subclasses.
- Difficulty tuning is pure data. Adjusting the hard-difficulty weights does not touch any function — you change the record only.
- Each scoring function is pure, which makes it straightforward to write property tests asserting things like "a higher hard-difficulty weight causes the AI to close faster than normal."

Scenario-Scoped AI Overrides

The scenario rules structure includes an optional AI config overrides field, which is a partial version of the full AI difficulty config. At every AI call site, a resolve-AI-config helper merges the scenario override on top of the difficulty preset. Fields not listed in the override fall through unchanged.

For example, the Duel scenario sets its combat-closing weight to one (the default is three) and its combat-close bonus to ten (the default is forty). Every AI call site calls the resolve-AI-config helper with the current difficulty and the scenario's override object, then passes the resulting merged config into whichever AI phase function it needs — astrogation, ordnance, combat, or the passenger-escort look-ahead.

The resolve-AI-config helper lives in the AI config module. The override field is defined on the scenario rules type in the domain types module. Scenarios opt in to overrides via the scenario definitions module. All four AI call sites thread the resolved config through.

Why this shape.

- There are no special cases. A check like "if the scenario is Duel" never appears in AI code. The data decides.
- Scenarios that do not set overrides behave exactly as before — the feature is opt-in at the scenario level.
- The mechanism was designed to support empirical sweep testing. The duel pacing fix went through such a harness, iterating over candidate weight values to find the best feel.

Preset Registries with Closed-Union Keys

When a config set has a fixed, known number of entries, Delta-V indexes it by a closed-union string type so TypeScript can enforce exhaustiveness. A lookup into such a registry always returns a value — there is no fallback needed.

The AI difficulty type, for instance, is defined as the union of the three literal strings "easy", "normal", and "hard". The AI config registry is then declared as a record keyed by that union type, with an entry for each difficulty level providing its full config object. Looking up a difficulty in that record is always safe — the return type is the full config, never undefined.

This AI config registry lives in the AI config module. The same pattern appears anywhere a fixed enumeration maps to a value, such as the client state entry rules registry.

Why this shape.

- Adding a new difficulty value fails to compile until the registry has a corresponding entry. The compiler enforces exhaustiveness automatically.
- Runtime fallbacks like "use the normal config if the key is missing" are a code smell. A proper closed-union key guarantees the entry exists, so no fallback is needed.

Scenario Rules as Feature Flags

Scenario-specific behavior is expressed as a flat bag of optional flags on the scenario rules type. Defaults are permissive — omitting a field means the feature is available. Engine code checks these flags at decision points, and client UI derives button visibility from the same flags.

The scenario rules type includes fields such as: an optional list of allowed ordnance types (default is all types); an optional list of available fleet purchases (default is all); an optional boolean for whether planetary defense is enabled (default true); an optional boolean for whether combat is disabled (default false); an optional boolean for whether logistics are enabled (default false); flags for hidden-identity inspection, escape-edge direction, checkpoint bodies, shared bases, passenger rescue, target-win-requires-passengers, reinforcement definitions, fleet conversion definitions, and AI config overrides.

At a decision point in the engine — for instance, when determining which phase to enter — the engine checks whether combat is disabled. If it is, the engine sets the phase to logistics and returns immediately.

The scenario rules type is defined in the domain types module. Scenario-level settings live in the scenario definitions module. A derived capability layer in the scenario capabilities module provides a helper called derive-capabilities that computes what the current scenario permits.

Why this shape.

- New scenarios do not need engine changes. The Grand Tour scenario disabled combat; the Convoy scenario enables logistics. Each just sets a flag.
- Permissive defaults keep simple scenarios minimal. The Bi-Planetary scenario's rules object is short because it does not opt out of anything.
- Client derivation stays consistent. The ordnance heads-up display and ordnance-phase auto-selection read from the same helpers the engine uses, so restricted scenarios do not drift between UI and server behavior.

Declarative Scenario Definitions

Each scenario is a declarative object describing ships, positions, rules, and budget — with zero procedural logic. Positions use body-relative helpers so that if a body's coordinates change, the scenario definition does not need to be updated.

The scenarios registry maps scenario keys to scenario definition objects. Each definition includes a name, a description, and an array of player entries. Each player entry lists that player's ships with their type, position, and initial velocity; their target body; their home body; and whether escaping wins the game. It also includes a rules object that can hold any subset of the feature flags described above — for example, the Duel scenario sets its AI config overrides here.

A create-game function snapshots the definition into a game state object when a game starts.

The scenario definitions live in the scenario definitions module. Body-relative position helpers — including a body-offset helper that places a position a given number of hexes away from a named body, and a controlled-base-hexes helper — live in the map layout module.

Why this shape.

- There are no conditionals at scenario start. The create-game function simply reads the specification and builds the initial state.
- Because positions are body-relative, a scenario that places a frigate two hexes east of Mercury still works correctly if Mercury's center hex moves.
- Once create-game runs, the game state owns its ships. Editing a scenario definition does not affect games already in progress.

Data-Driven Solar System Map

The map is generated from eleven body definitions — Sol, Mercury, Venus, Terra, Luna, Mars, Ceres, Jupiter, Io, Callisto, and Ganymede — plus asteroid-belt coordinate arrays. There are no hand-drawn hex tables.

Each body is a declarative specification. A body definition includes the body's name, its center hex coordinates, its surface radius, the number of gravity rings it exerts, its gravity strength, whether it is destructive to ships, which of its six sides have orbital bases, its display color, and its render radius in pixels.

The map builder generates from these definitions: surface hexes typed as planet-surface or sun-surface, gravity rings with direction vectors pointing toward the body center, orbital bases placed according to the base-directions field, and asteroid belt hexes drawn from explicit coordinate arrays (irregular shapes do not lend themselves to a formula).

The body definitions, asteroid belt data, and map builder all live in the map data module. Body offset helpers and controlled-base helpers live in the map layout module.

Why this shape.

- Adjusting Mars's gravity strength is a one-field edit; the engine never needs to change.
- Body-relative helpers work off the same definitions, so if a body moves, scenarios and the renderer follow automatically.
- A lookup table indexed by body name is built at module load time from the body definitions array. Renderers, the AI, and any other reader all query the same object.

Cross-Pattern Theme: Untyped String Keys

The type-safety gap that cuts across these patterns is untyped string keys. The AI config registry shows the target state: a closed-union key paired with a record type gives compiler-enforced exhaustiveness with no runtime fallback needed.

A parallel rollout would introduce a scenario key union type covering all scenario names, a body name union type covering all eleven bodies, and branded identifier types for ship IDs, ordnance IDs, and game IDs — a topic covered in the Type System chapter.

Each step would replace a defensive runtime fallback with a compile error, catching renames and typos automatically.

Testing Patterns

How the Delta-V test suites are structured and what each layer covers. The coding standards document gives the test conventions around co-location, property tests, and coverage floors. The simulation testing document describes the simulation and load harnesses. This chapter walks through the patterns inside the Vitest suite.

Each section covers the pattern, a minimal example, where it lives, and why this shape.

Co-Located Tests

Every test file sits next to the module it tests, using the same name with a dot-test suffix. There are no separate test folders or test barrels. Property-based tests use an additional dot-property segment in the filename. Contract fixtures live in a fixtures subdirectory near the tests that consume them.

As a concrete example: the shared hex module, its standard tests, and its property-based tests all sit in the same directory, alongside the combat module and its tests, a fixtures folder for contract data, and the protocol module with its tests.

The Vitest configuration globs all test files under the source directory. The end-to-end directory is excluded from the main test runner and handled by Playwright separately.

The rationale for this shape is threefold. First, readers and reviewers see tests together with code — opening the combat module in an editor exposes its test file in the same folder, with no navigation tax. Second, coverage gaps are visible: scanning a directory for source files without a corresponding test file is a mechanical audit. Third, there is no test infrastructure to learn — a new contributor adding a file writes its test next to it, with no discovery step.

Property-Based Tests

Core engine functions have suites built with the fast-check library that verify invariants across generated inputs. Custom domain arbitraries — for coordinates, ship types, and odds ratios — keep inputs realistic and bounded.

As examples: one property asserts that hex distance is symmetric, meaning the distance from point A to point B equals the distance from point B to point A, for all possible coordinate pairs. Another property asserts that higher combat odds never produce worse results than lower odds, comparing outcomes across generated combat scenarios with a bounded number of runs to keep engine-heavy properties fast.

These tests live alongside the modules they exercise — in the shared hex property test file, the combat property test file, the movement property test file, and the client reactive test file. Arbitraries live alongside the modules they exercise.

Next, the rationale. Invariants over examples means a statement like "fuel never goes negative" is true for all inputs, not just the ones the author thought of. When a property fails, fast-check automatically reduces the counterexample to a minimal reproduction. And a roughly twenty-line property test often covers what would take a dozen example tests.

Data-Driven Tests with It-Each

Tables of input-output pairs use Vitest's parameterised test feature instead of N separate test blocks. Combat tables, damage lookups, and hex-math boundaries are natural fits. Using the as-const assertion gives literal type narrowing.

As an example, a single parameterised hex-distance test lists three coordinate pairs alongside their expected distances — including an axial cube diagonal — and runs one assertion against each row.

This pattern lives in the shared protocol test file and the client transport test file.

The benefits: boilerplate shrinks because twenty test bodies collapse into twenty input rows and one assertion. The tests read like a spec. And failure messages are descriptive because the parameter placeholders show the failing row.

Contract Fixtures for Protocol Shapes

Canonical wire-format payloads live as JSON fixtures paired with their expected validator output. Tests iterate fixtures and assert round-trip correctness. Large game-state values use a double-underscore STATE double-underscore sentinel so fixtures stay stable when engine types evolve.

As an example, a contracts fixture file contains two entries: one for a valid astrogation message, with a raw payload and its expected validated output; and one for an empty movement-result message, where the game-state field is replaced with the sentinel string.

These fixtures live in two locations: one for client-to-server messages in the shared module, and one for server-to-client messages and HTTP responses in the server game-do module. They are loaded via synchronous file reads and JSON parsing in the protocol and game-do tests.

With that established, the rationale: the wire format is an API, and representative payloads pinned as JSON catch accidental protocol changes better than purely behavioral tests. The sentinel string insulates fixtures from game-state churn, though envelope changes still require manual fixture updates. Tests also convert undefined values to null to mirror JSON fidelity.

Mock Durable Object Storage

Tests that touch the Durable Object plumbing use an in-memory map that implements enough of the Durable Object storage interface to exercise the code under test.

The mock provides get, put, and delete operations backed by an in-memory map. Get resolves with the stored value. Put handles both single-key and multi-key record forms. Delete returns whether the key was present. List and transaction are omitted unless a specific test needs them.

This pattern appears in several server-side test files covering archiving, game-do logic, alarms, match archiving, and fetch handling. The game-do test file also mocks the Cloudflare workers module to stub the Durable Object base class.

The benefits: tests skip the Wrangler runtime, making them faster, deterministic, and easier to debug. Each mock implements only the surface area its test needs — list and transaction are not implemented because Delta-V uses simple key-value I/O.

Deterministic RNG in Tests

Tests pin random number generation via seeded pseudo-random number generators. The mulberry32 seeded generator is used for full runs. Short fixed functions — always returning the same value — are used for single-call assertions. Fast-check arbitraries supply random values for property-based runs. No test calls the built-in Math random function.

Three examples illustrate the pattern: a fixed seed produces a fixed sequence, making specific damage values reproducible across runs. A property test lets fast-check supply random doubles in the zero-to-one range. And an algorithm snapshot test captures a sequence of five generated values with an inline snapshot, catching any future changes to the generator.

This lives in the shared pseudo-random number generator module, which provides the mulberry32 function and a derived-action RNG helper. The generator test file covers determinism, value range, uniformity, collision rate, and cross-seed divergence.

Turning to the rationale: making RNG an explicit required parameter on turn-resolution entry points means tests cannot forget to pass one — it is the only viable approach. The derived-action RNG helper means replaying an event range does not require replaying all the history before it.

Coverage Thresholds

V8 coverage thresholds are enforced on all TypeScript files under the shared module. Both the pre-commit hook and continuous integration run the coverage target. Thresholds are treated as a ratchet, not a target.

The Vitest configuration specifies thresholds for the shared module: eighty-four percent of statements, seventy-five percent of branches, eighty-eight percent of functions, and eighty-five percent of lines. Reports are generated in text, HTML, and JSON summary formats, and the coverage output folder is gitignored.

The configuration lives in the Vitest config file at the root of the project.

The rationale: thresholds prevent backsliding — a refactor that adds untested code fails CI. Coverage is enforced only on the shared module because the engine is the replay contract, making its coverage the most critical. Server and client coverage is useful but not yet enforced. The branch threshold is intentionally lower than the line threshold because defensive branches in complex game rules are hard to exercise, and forcing a higher branch figure would encourage tests that do not add real confidence.

Cross-Pattern Reinforcement

The testing patterns reinforce each other.

Property-based tests covering invariants, combined with deterministic RNG for reproducibility, drive high coverage numbers. Coverage thresholds catch any backsliding from changes to property tests. Contract fixtures and data-driven tests exercise the protocol boundary comprehensively. Mock storage enables unit testing of the entire event-sourced persistence layer without the Cloudflare runtime.

To recap, the main consolidation opportunity is a shared mock-storage module plus broader negative contract coverage — both would tighten what is already a mostly solid foundation.

Delta-V Protocol & Data Contracts

This chapter covers the wire format, state shapes, and coordinate math that an agent writer or server-side developer needs to understand. Gameplay rules and scenarios are covered separately in the spec document; module layout is covered in the architecture document. The TypeScript source files are the authoritative reference when the prose and the code disagree — the relevant files define client-to-server and server-to-client messages, the game state and ship domain types, and the scenario definition types.

Cloudflare Topology

Delta-V runs on Cloudflare Workers with three Durable Object classes. The first is the game room object, which manages one room per active match — it handles the event stream, checkpoints, WebSocket connections, and alarms. The second is the matchmaker object, a singleton that maintains the quick-match ticket queue and pairs players into seats. The third is the live registry object, another singleton that tracks matches currently in progress for the public match-listing API.

Beyond the Durable Objects, the system uses a D1 database for telemetry and match archive metadata, and an R2 bucket for storing completed match JSON. The architecture document explains how these pieces fit together.

HTTP Endpoints

The server exposes a set of HTTP routes. A GET to the root serves the single-page application. A POST to the create route generates a five-character room code and a creator reconnect token, and locks in the scenario. A POST to the quick-match route enqueues a matchmaking ticket; a GET to that same route with the ticket identifier polls the ticket state, which can be waiting, matched, or expired.

A GET to the join route with a room code performs a preflight join or reconnect validation. A GET to the replay route fetches an authenticated replay for a specific match. A GET to the WebSocket route — which the client upgrades — connects the browser to the room's game object.

On the API side, a POST to the agent-token route issues a twenty-four-hour HMAC-signed agent token; an optional claim body binds that agent to a leaderboard username at the same time. A POST to the claim-name route binds a human player key to a leaderboard username on a first-call-wins basis. A GET to the main leaderboard API returns the public ranking ordered by Glicko-2 rating, with provisional rows hidden by default. A GET to the per-player leaderboard lookup returns a single player's current rating and rank. A POST to the Model Context Protocol endpoint handles stateless remote MCP requests. A GET to the matches API returns a match listing, filtered by status — live for in-progress from the live-registry, or default for completed from the database. There are also HTML pages for match history, the public leaderboard, and the agent landing page, a machine-readable agent manifest, and a phase-and-action map

for agents. A version manifest endpoint exposes the package version and the assets hash for support and cache-bust debugging. Finally, POST routes for telemetry and error reporting accept small JSON payloads from the client. Rate limits for each route are detailed in the security document.

Room Lifecycle

The lifecycle of a match follows seven steps.

1. The client posts to the create route. The Worker allocates a five-character room code and a creator token, initialises the game Durable Object, and returns the code and token to the client.
2. Optionally, the client performs a preflight GET to the join route, passing the player token as a query parameter.
3. For replays, an authenticated GET to the replay route reconstructs the match state from a checkpoint plus the stored event stream.
4. The client opens a WebSocket connection to the room. The Durable Object accepts the connection and tags the socket with a player identity — Player Zero, Player One, or spectator.
5. Once both unique seats are connected, the server calls its internal game-creation function and broadcasts a game-start message to all connected clients.
6. The game loop runs: the client sends an action, the engine processes it, events are saved, timers are restarted, and the server broadcasts the result back to all clients.
7. On disconnect, a thirty-second grace period begins. The player may reconnect using their token within that window; otherwise the game is forfeited.

Players are seat-based — Player Zero and Player One. The creator's seat is protected by a token; the guest seat is claimed by room code alone until the server issues a player token in the welcome message.

WebSocket Protocol

All messages are JSON, sent over WebSocket. Because the game is turn-based, message frequency is low.

Client to Server Messages

Clients send a discriminated-union message for each action. The available message types are: fleet ready (carrying a list of fleet purchases), astrogation orders (a list of movement orders), surrender (a list of ship identifiers), ordnance launches, orbital base emplacements, skip ordnance, begin combat, combat (a list of attacks), combat single (a single attack), end combat, skip combat, logistics transfers, skip logistics, rematch, chat (carrying a text string), and ping (carrying a timestamp).

A fleet purchase is either a ship of a given type or an orbital base cargo unit.

All messages are validated at the protocol boundary using discriminated unions. Chat payloads are trimmed of whitespace before validation, and blank messages after trimming are rejected.

Server to Client Messages

The server sends its own discriminated-union messages back to the client. The available types are: welcome (carrying the player's identifier, the room code, and the player token), spectator welcome (carrying the room code), match found, game start (carrying the full game state), movement result (carrying ship movements, ordnance movements, movement events, and the updated game state), combat result (carrying a list of combat results and the updated state), combat single result (carrying a single combat result and the updated state), state update (carrying the updated state and optional logistics transfer events), game over (carrying the winner identifier and reason), rematch pending, chat (carrying the sender's player identifier and text), error (carrying a message and an optional error code), action rejected (carrying a rejection reason and the updated state), pong (carrying a timestamp), and opponent status (carrying a connected boolean).

Every action that mutates state produces exactly one state-bearing message — one of game start, state update, movement result, combat result, or combat single result. Clients replace their local state wholesale on receipt; there is no incremental patching. The game-over message is sent after that terminal state-bearing message.

For scenarios that involve hidden information — such as the Escape scenario — the server filters the game state for each player before sending, so each client only sees what their player is permitted to see.

Game State

The game state is a single object that the server broadcasts after every state-mutating action. It carries a schema version number, a stable per-match game identifier, the scenario name, the full set of scenario rules, a flag for whether an escape moral victory has been achieved, the current turn number, the current phase, and the active player (zero or one).

It also carries arrays of ships, ordnance, pending astrogation orders, pending asteroid hazards, destroyed asteroid hex keys, and destroyed base hex keys. Finally it holds a two-element array of player states, and an outcome field that is null while the game is in progress and carries the winner and reason when the game ends.

Each ship in the state has an identifier, a type key, an owner, an original owner (which may differ from owner if the ship was captured), a position, an optional last movement path, a velocity vector, a fuel value, a cargo-used mass value, a count of nukes launched since last resupply, a flag for whether the ship was resupplied this turn, a lifecycle status of active, landed, or destroyed, a control status of own, captured, or surrendered, a detected flag, a heroism-available flag, an overload-used flag, an optional base status of carrying base or emplaced, an optional identity block for hidden-identity scenarios (carrying whether the ship has fugitives and whether the identity has been revealed — the fugitive flag is stripped for unrevealed opponents), optional pending gravity effects, and a damage block carrying the number of disabled turns remaining (zero means operational; six or more means eliminated).

Each player state carries a connected flag, a ready flag, the player's current target body, their home body, an array of controlled base hex keys, and an escape-wins flag.

The phase can be one of: waiting, fleet building, astrogation, ordnance, logistics, combat, or game over.

Hex Math

The board uses axial coordinates — a horizontal axis called q and a diagonal axis called r — with a standard flat-top hex orientation. There are six possible movement directions: east, north-east, north-west, west, south-west, and south-east, each expressed as a delta-q and delta-r pair.

A coordinate is a pair of q and r integers. A vector is a pair of delta-q and delta-r integers representing movement.

The hex utility module provides a set of core functions. The hex-distance function returns the number of hexes between two points, calculated as the maximum of the absolute delta-q, the absolute delta-r, and the absolute sum of delta-q and delta-r. The hex-line-draw function returns the sequence of hexes along a straight line, used for plotting courses and computing line-of-sight. The hex-neighbors function returns the six adjacent hexes. The hex-to-pixel function converts a hex coordinate to a screen position. The pixel-to-hex function converts a screen tap or click to the nearest hex coordinate. There are also add and subtract functions for combining coordinates and vectors, and a hex-key function that produces a branded string in the form "q comma r" for use as a map or set key.

For a deeper introduction to the underlying mathematics, Red Blob Games publishes a widely referenced guide on hexagonal grids.

Vector Movement Algorithm

Movement is resolved by a course-computation function that takes a ship, a burn order, and the map. It works as follows.

1. The predicted destination is initialised to the ship's current position plus its current velocity — that is, where the ship would coast if it made no burn.
2. If a burn direction is provided, one fuel unit is consumed and the predicted destination is shifted one hex in that direction. Warships can make a second burn in an overload manoeuvre, consuming two fuel units total.
3. The path is traced as a straight line from the ship's current position to the predicted destination.
4. For each hex along that path, the function checks whether the hex exerts gravity. If it does, and if the player has not chosen to ignore a weak-gravity hex they are entering for the first time, the gravity deflection is queued as a deferred effect — it will be applied on the next turn, not the current one.
5. The path is recomputed as a straight line incorporating the gravity deflection.
6. The new velocity is calculated as the difference between the final predicted destination and the ship's starting position.

The function returns the destination, the final path, and the new velocity. The pending gravity effects field on each ship tracks which deflections are queued for the following turn.

Scenario Definition

A scenario definition carries a name, a description, a list of player configurations, an optional rules block, an optional starting player (zero or one), optional starting credits (either a single value for both players or a pair of values), and an optional list of available fleet purchases.

Each player configuration carries a list of starting ships, a target body (empty string if none), a home body, optional starting base positions, an escape-wins flag, and an optional hidden-identity flag.

The rules block is where scenario-specific behaviour is configured. Its fields include: the allowed ordnance types, the available fleet purchases, whether planetary defense is enabled, whether hidden-identity inspection is enabled, the escape edge (any edge or north only), whether combat is disabled, the checkpoint bodies, the shared bases, whether logistics is enabled, whether passenger rescue is enabled, whether a target win requires passengers, any reinforcement definitions, fleet conversion rules, and AI difficulty config overrides.

Defaults are permissive — omitting a field means that feature is enabled. The patterns documentation walks through the full scenario configuration in detail.

Map Data

The solar-system map is generated programmatically from TypeScript body definitions rather than loaded from a static JSON asset. Each position on the board becomes a map hex object carrying its q and r coordinates, a terrain type (one of space, asteroid, planet surface, or sun surface), an optional gravity block (with a direction, a strength of full or weak, and the name of the owning body), an optional base block (with an owner — either neutral or a player faction — and a type of planetary, asteroid, or orbital), and an optional body name if the hex is covered by a celestial body.

A build function assembles the full array of map hexes at engine entry.

Error Model

Server-to-client error messages carry an optional error code. The defined codes fall into several categories. Timing errors cover invalid phase and not-your-turn. Reference errors cover invalid ship, invalid target, and invalid selection. Input shape errors cover invalid input. Authorization errors cover not allowed and invalid player. Resource errors cover resource limit. Consistency errors cover state conflict.

Rate-limit closures use WebSocket close code 1008 and are not sent as error messages. Multi-stage validation is walked through in the type system and validation patterns document.

Further Reading

The spec document covers gameplay rules and scenarios. The architecture document covers module inventory and data flow. The security document covers rate limits, authentication, and abuse controls. The agent spec document covers the agent-specific observation and action model built on top of this protocol.

The protocol and persistence patterns document explains the reasoning behind the wire format design.

PART 3

Game Design and World

This section covers the canonical rules, scenarios, and the visual direction of the setting and ship roster.

Delta-V Game Rules

An online multiplayer space combat game — vector movement and gravity in the inner Solar System.

Overview

Delta-V is a turn-based strategy game where ships move using realistic vector physics on a hex grid. Ships maintain velocity between turns, can burn fuel to accelerate, and are affected by planetary gravity. Combat uses dice-based resolution with modifiers for range and relative velocity.

The client renders the board as a continuous-space experience — no visible hex grid — while the engine uses hex coordinates internally for all game logic. Ships animate along their velocity vectors with thrust and gravity effects.

How To Read This Document

This document covers rules and scenarios only. The protocol definitions and type shapes live in a separate protocol document. Module layout and data flow are covered in the architecture document. Open work is tracked in the backlog. Implementation gotchas are documented per pattern in a patterns directory.

When the prose here disagrees with the code, the code wins — but the rules below are the canonical design reference.

Table of Contents

- Hex Grid
- Solar System Map
- Vector Movement
- Turn Structure
- Ships and Cargo
- Combat
- Ordnance
- Gravity, Orbit, Landing, Takeoff
- Bases, Detection, and Support
- Other Rules
- Scenarios
- Implementation Status
- Design Decisions

Hex Grid

The game uses axial hex coordinates internally, expressed as a column value and a row value. The grid is roughly forty columns by fifty-five rows. The client renders hex centers as pixel positions but does not draw hex borders — subtle dots or a faint radial guide may indicate valid destinations during planning.

The hex orientation is flat-top. The coordinate math primitives are defined in the protocol document.

Solar System Map

The map represents the inner Solar System along the ecliptic plane.

Turning to the celestial bodies and their gravity hexes: Sol, the Sun, sits at the center of the map. It is a radius-two body with two full-gravity rings, and any contact with it destroys a ship outright. Mercury is a single-hex body with one full-gravity ring and two base hexes. Venus is a radius-one body with one full-gravity ring and bases on all six sides. Terra — Earth — is also a radius-one body with one full-gravity ring and bases on all six sides. Luna, Terra's moon, is a single-hex body with one weak-gravity ring and bases on all six sides. Mars is a single-hex body with one full-gravity ring and bases on all six sides. Jupiter is a large body positioned in the north of the map with two full-gravity rings. Jupiter has three moons: Io and Callisto each have one weak-gravity ring and one base, while Ganymede has one weak-gravity ring but no base. Finally, Ceres is a single-hex asteroid body with one base and no gravity at all. Scattered asteroid hexes fill the belt between the inner planets and Jupiter.

Next, the two gravity types. Full gravity imposes a mandatory one-hex deflection toward the parent body for any object passing through that hex. Weak gravity — found around Luna, Io, Callisto, and Ganymede — gives the player a choice: when passing through a single weak-gravity hex, the player may use or ignore the deflection. However, two consecutive weak-gravity hexes of the same body make the deflection mandatory on the second hex.

The map is generated programmatically from static definitions for each body, ring, and asteroid field.

Vector Movement

Vector movement is the core mechanic. Each ship has a velocity vector — a displacement from its current hex to its destination, repeated each turn until thrust or gravity changes it.

The canonical movement procedure has six steps. First, predict the course: the current position plus the current velocity gives the projected destination. Second, optionally burn fuel: spend one fuel unit to shift the projected destination by one hex in any of the six directions. Third, overload — warships only: once between maintenance stopovers, a warship may spend two fuel total for a two-hex shift. Fourth, apply deferred gravity: gravity hexes entered on the previous turn deflect the current move by one hex per hex entered. Fifth, move: the phasing player's ships travel simultaneously along their final plotted paths. Sixth, queue new gravity: gravity hexes entered during this move will apply on the following turn.

With that established, the detailed movement rules. Gravity takes effect on the turn after entry, not the turn of entry. A single weak-gravity hex may be ignored, but two consecutive weak-gravity hexes of the same body make the second deflection mandatory. A course that travels exactly along the edge of a gravity hex does not count as entering it. Ships keep their velocity vectors between turns, and a stationary ship has a zero vector. Any ship whose final course ends off the map is eliminated — intermediate hexes may leave the map as long as the final position lands back on it.

Turn Structure

Each game turn consists of one player-turn per player. A player-turn has six stages.

One: Astrogation — plot fuel burns, overloads, and weak-gravity choices. Two: Ordnance — launch mines, torpedoes, or nukes, limited to one item per ship. Three: Movement — the phasing player's ships and ordnance move simultaneously. Four: Combat — resolve asteroid hazards, gunfire, counterattacks, planetary defense, and anti-nuke fire. Five: Logistics, which is conditional on whether the logistics rules are enabled — transfer fuel and cargo, and loot. Six: Advance — base resupply, damage recovery, and rotation of the phasing player.

After both players complete their turns, a new game turn begins. The engine's internal phase tracking collapses movement resolution into the astrogation phase for implementation purposes, as described in the protocol document.

Ships and Cargo

Nine ship types plus orbital bases are available. Here is each type with its characteristics.

The Transport has a combat factor of one — the D suffix meaning it is a commercial vessel that may not attack or counterattack — with ten fuel capacity, fifty cargo capacity, and the ability to carry orbital bases. The Packet has a combat factor of two, ten fuel, fifty cargo, and can also carry orbital bases. The Tanker has a combat factor of one with the defensive suffix, fifty fuel, and no cargo capacity. The Liner has a combat factor of two with the defensive suffix, ten fuel, and no cargo. The Corvette has a combat factor of two, twenty fuel, and five cargo — the smallest warship. The Corsair has a combat factor of four, twenty fuel, and ten cargo — a mid-size warship. The Frigate has a combat factor of eight, twenty fuel, and forty cargo — a large warship. The Dreadnaught has a combat factor of fifteen, fifteen fuel, and fifty cargo — the heavy warship. The Torch has a combat factor of eight, unlimited fuel, and ten cargo, but may not transfer fuel to other ships. Finally, the Orbital Base has a combat factor of sixteen, unlimited fuel, and unlimited cargo, and is a stationary emplacement.

Turning to the special capacity rules: a combat factor with the defensive suffix marks a commercial ship that may not attack or counterattack. Only warships may overload. Only warships may launch torpedoes. Any ship may carry and launch nukes if it has the cargo capacity, but non-warships are limited to one nuke launch between resupplies. Only transports and packets may carry orbital bases. Fuel is not cargo.

Each ship tracks its position, velocity, fuel, cargo and ordnance load, damage state, detection state, and any scenario-specific flags such as capture status or heroism.

Combat

Standard gun combat is the default for the online implementation.

The sequence is as follows. First, the phasing player declares attacks. Second, combine the chosen attackers' combat factors and compare them to the defender's factor to determine the odds — which fall into bands of one-to-four, one-to-two, one-to-one, two-to-one, three-to-one, or four-to-one. Third, apply the range modifier by subtracting one for each hex of range, measured as the attacker's closest approach to the target's final position. Fourth, apply the relative velocity modifier by subtracting one for each hex of relative velocity above two. Fifth, roll one six-sided die on the gun combat table. Sixth, counterattack is resolved before attack damage is implemented.

Next, the detailed mechanics. Line of sight is blocked by planets, moons, and Sol; ships, ordnance, and asteroids do not block line of sight. A defender may counterattack if still eligible, and ships sharing the defender's hex and course may join the counterattack. Attacks may be declared at less than full strength. When multiple ships attack together, use the greatest applicable range and velocity penalties, not the average. A ship may not attack more than once per combat phase. A group attacking from the same hex may split its combat strength across multiple targets if all targets share one hex. Planetary-defense shots follow normal gunfire rules except where a specific exception is noted.

With that established, the damage rules. Damage is expressed in disabled turns — values from one through five disable a ship for that many turns. Disabled ships drift on their current vector and may not maneuver, attack, counterattack, or launch ordnance. Damage is cumulative, and six or more accumulated disabled turns eliminates the ship. Every damaged ship repairs one disabled turn at the end of each of its own player-turns. Maintenance at a friendly base repairs all damage and restores one overload allowance.

Other damage sources work as follows. Torpedoes roll on the Other Damage Table and can hit only one ship. Mines roll on the Other Damage Table against every affected ship. Asteroids roll on the Other Damage Table for each asteroid hex entered at speed greater than one. Ramming rolls on the Other Damage Table for both ships. Nukes automatically destroy everything in the detonated hex.

Ordnance

General ordnance rules apply to all types. All ordnance is affected by gravity. Ordnance moves only during its owner's movement phase. Each ship may release only one item per turn. A ship may not launch ordnance while at a base, while taking off or landing, while refueling or transferring fuel, or during any player-turn in which it resupplies. Mines, torpedoes, and nukes detonate when they enter a hex containing a ship, an astral body, another mine, a torpedo, or a nuke — and conversely, when any of those things enter their hex.

Turning to mines, which have a mass of ten. Mines inherit the launching ship's velocity vector. The launching ship must immediately change course so it does not remain in the mine's hex. Mines remain active for five turns, then self-destruct. They detonate on hex intersection. Guns and planetary defenses have no effect on mines.

Torpedoes have a mass of twenty. They inherit the launching ship's velocity vector, then may accelerate one to two hexes in any direction on the launch turn. Only warships may launch torpedoes. A torpedo hits only a single target; if multiple ships share the affected hex, they are resolved in random order until one is damaged or destroyed, or all roll no effect. Torpedoes continue moving if they miss.

Nukes also have a mass of twenty. They inherit the launching ship's velocity vector and remain active for five turns before self-destructing. A nuke explodes when it enters a hex containing a ship, base, asteroid, mine, or torpedo. It destroys everything in the detonated hex automatically. It converts an asteroid hex to clear space. If a nuke reaches a moon or planet without earlier detonation, it devastates one entire hex side — any base or ship on that side is destroyed. Guns and planetary defenses may attack nukes at two-to-one odds with normal range and velocity modifiers, and any disabling result destroys the nuke. Whether nukes are available at all is determined by scenario rules.

Gravity, Orbit, Landing, Takeoff

Each gravity hex has an arrow pointing toward its parent body. Deflections are cumulative. Orbit is not a special game state — it emerges naturally from speed-one movement through the gravity ring.

Turning to landing and takeoff. To land on a planet or satellite, a ship must first be in orbit and then spend one fuel to land on a base hex side. Intersecting a planet or satellite any other way is a crash. A ship may land on Ceres, the clandestine asteroid, or an unnamed asteroid by stopping in that asteroid hex. Takeoff from a planetary base is free — boosters push the ship outward, surface gravity cancels that boost, and the ship begins stationary in the gravity hex directly above the base. After takeoff, fuel must still be spent normally to enter or leave orbit. Landed ships at planetary bases are immune to gunfire, mines, torpedoes, and ramming — but not nukes. Landed ships may not fire guns or launch ordnance.

Bases, Detection, and Support

Planetary bases provide fuel, maintenance, cargo handling, ordnance reloads, detection coverage, and planetary defense. Each base may fire at every enemy ship in the gravity hex directly above it during its owner's combat phase, at two-to-one odds with no range or velocity modifiers.

Asteroid bases provide all normal base functions but have no planetary defense capability. They may launch one torpedo per turn. They are harmed only by nukes unless a specific scenario overrides this rule.

Orbital bases may be carried only by transports or packets. They may be emplaced in a gravity hex while the carrying ship is in orbit, or placed on an unoccupied world hex side. They do not literally orbit once emplaced. They may fire one torpedo per turn if not currently resupplying another ship. They cannot be moved once placed.

The clandestine base is a secret asteroid base that uses orbital-base statistics. Special dense-asteroid and scanner rules apply around it, as defined by the specific scenario.

Next, the resupply and maintenance rules. All bases provide unlimited fuel, mines, and torpedoes. Planetary bases resupply landed ships on their base hex side. Asteroid bases resupply ships stopped in the base hex. Orbital bases resupply ships that match the base's position and course. Refueling always includes maintenance — all damage is repaired and one overload allowance is restored. A ship may take any mix of mines and torpedoes that fits its cargo capacity. No ship may fire guns or launch ordnance during a player-turn in which it resupplies. An orbital base that resupplies any ship may not fire guns or launch ordnance that same player-turn.

Turning to detection. Ships and orbital bases detect at range three. Planetary bases detect at range five. Once detected, a ship remains detected until it reaches a friendly base. In hidden-identity scenarios such as Escape, an enforcer may reveal a hidden ship through inspection — ending a turn in the same hex with the identical velocity vector. Detection matters most in hidden-information scenarios; in fully open scenarios it has less player-facing impact.

Other Rules

Asteroids: roll once on the Other Damage Table for each asteroid hex entered at speed greater than one. Moving along a hex edge between two asteroid hexes counts as entering one asteroid hex. Mines and torpedoes detonate on entering asteroid hexes.

Capture: a disabled ship can be captured by an enemy that matches its course and position. A captured ship may not fire and must be brought to a friendly base before reuse.

Surrender: ships may surrender by agreement, which is distinct from capture.

Looting and rescue: transfers happen only when positions and courses match. Only disabled or surrendered ships may be looted.

Heroism: longer scenarios can award a one-time plus-one attack bonus after a qualifying underdog success.

Advanced combat, which is the optional alternate combat model with separate weapon, drive, and structure damage tracks, is out of scope.

Scenarios

Nine scenarios ship with the game, listed here in rough difficulty order.

Bi-Planetary (learning scenario)

This two-player learning scenario has Player One starting with a corvette on Mars and Player Two starting with a corvette on Venus. The goal is for each player to navigate to the other player's starting world and land there. This scenario teaches vector movement, fuel management, gravity assists, and orbital mechanics.

Escape (asymmetric)

This scenario pits Pilgrims against Enforcers. The Pilgrims control three transports departing from Terra; the Enforcers control one corvette near Terra and one corsair near Venus. The Pilgrims must escape the solar system while the Enforcers must stop them.

The scenario uses hidden identity: one of the three transports carries the fugitives, but the opponent does not know which one. The server strips the identity information from unrevealed opponent ships to enforce this. A moral victory is tracked if the Pilgrims disable an Enforcer ship before being lost.

Lunar Evacuation (escort)

This escort scenario has Evacuees — a transport and corvette departing from Luna — facing an Interceptor controlling a corsair from Terra. Logistics and passenger-rescue rules are enabled. Victory requires landing survivors at Terra.

Convoy (escort mission)

The Escort side controls a liner carrying passengers, a tanker, and a frigate, all departing from Mars. The Pirates control two corsairs and one corvette. Passenger-rescue logistics are enabled, and the target-body victory condition requires survivors to be aboard upon arrival.

Duel (combat training)

Two frigates start near Mercury in a last-ship-standing battle. This scenario teaches combat mechanics, ordnance use, and gravity-assisted combat maneuvers.

Blockade Runner

One packet ship faces one corvette in an asymmetric scenario where the packet must reach Mars. The dynamic is speed and agility against raw firepower.

Fleet Action

A fleet-building battle with a tuned first-player ordering for a shorter, balanced clash. This is a full combined-arms engagement.

Interplanetary War

A tuned fleet-building war using Terran and Rebel roles drawn from the original rulebook. It uses a smaller budget than the full paper campaign. The scenario emphasizes strategic home-base positioning and mixed-fleet combat.

Grand Tour (race)

Each player starts with a corvette at a different habitable world. To win, a player must pass through at least one gravity hex of each major body — Sol, Mercury, Venus, Terra, Mars, Jupiter, Io, and Callisto — and then return to land at their starting world. There is no combat; this is a pure navigation and gravity-

management race. Bases at Terra, Venus, Mars, and Callisto are shared.

Unimplemented rulebook scenarios

Several scenarios from the original rulebook are out of scope due to dependencies on logistics and extended-economy mechanics that have not yet been built.

Lateral 7 is a short two-player scenario that requires dummy counters, a clandestine base, scanners, and dense asteroids. Piracy is a long three-player scenario requiring the clandestine base, scanners, trade cycles, cargo delivery, and the Merchants, Patrol, and Pirates faction roles. Nova is a short three-player scenario requiring an alien fleet AI, nova bombs, and multi-faction support. Retribution is a medium two-player scenario requiring sequential corvette reinforcements for the Sons of Liberty and a Freedom Fleet conversion mechanic. Fleet Mutiny is a long two-player scenario requiring hexside suppression, base capture, and planetary-defense suppression. Prospecting is a long multi-player scenario requiring automated mines, robot guards, ore, crystalline trading shards, and grapple mechanics. Campaign is a full multi-player scenario requiring a complete economy, a referee, and all of the above.

Implementation Status

The backlog document is the authoritative list of open work. This section is a snapshot of what exists versus what diverges from the rulebook.

Implemented faithfully

Vector movement with deferred gravity, weak-gravity player choice, and overload burns are all implemented. Overload allowance tracking is in place — warships may overload once between maintenance stopovers, and base resupply restores the allowance. The gun combat table matches the 2018 rulebook, including the minimum damage threshold and correct per-odds values. Per-source Other Damage Tables are implemented, with torpedo, mine, asteroid, and ramming each using their own column. Limited-strength attacks, multi-target attack queuing, and landed-ship immunity are implemented. Resupply-turn restrictions preventing firing or launching when resupplied are in place. Ordnance is implemented: mines with the five-turn self-destruct and course-change requirement, torpedoes with the one-to-two hex launch boost, and nukes with hex devastation, base destruction, and asteroid clearing. Anti-nuke fire at two-to-one odds for guns and planetary defense is implemented. Per-base ownership driving planetary defense, detection, and resupply is implemented. Hidden identity — including the Escape scenario's fugitive concealment and server-side state filtering — is implemented. Inspection mechanics that reveal hidden ships by matching position and velocity are in place. Split-fire, which allows allocating an attacking group's strength across multiple targets in one hex, is implemented. Detection at range three for ships and range five for bases, persistent once triggered, is implemented. Damage tracking with cumulative disabled turns, recovery, and elimination at six or more is implemented. The dreadnaught exception — allowing dreadnaughts to fire guns even when disabled — is in place. Landing validation requiring orbit first, takeoff mechanics, and landed-ship immunity are implemented.

Ramming, asteroid hazards, and crash detection are implemented. The Escape inspection, concealment, and moral-victory flow are implemented. Counterattack targeting defaults to the strongest attacker. Event-sourced server recovery from a persisted match stream plus checkpoints is implemented.

Accepted divergences

Contact geometry is approximated rather than exact. Mine and torpedo contact is determined by hex occupancy and path intersection rather than the stricter board-game rule requiring literal geometric line intersection with the printed hex area. This is a standard digital approximation; correcting it would require sub-hex geometry that is incompatible with axial coordinate math.

Logistics are partially implemented. Surrender, fuel and cargo transfer, looting, passenger and colonist transfer, and the Torch fuel-transfer restriction are all implemented. Dummy counters for concealment scenarios and any rescue-scenario tuning beyond the Convoy scenario remain open.

Extended economy is deferred. Shipping lanes — including the Piracy trade cycles and cargo delivery — and asteroid prospecting — including automated mines, robot guards, ore, and crystalline trading shards — are scenario-specific economies from the Piracy and Interplanetary War scenarios. These are deferred until those scenarios are on the roadmap.

Design Decisions

One: the game uses alternating turns rather than simultaneous movement, matching the original board game. Simultaneous movement would change game dynamics significantly.

Two: the standard combat system is used, with the disabled-turn damage model including implemented exceptions such as the dreadnaught disabled-fire rule and the orbital-base reduced-operation rule.

Three: contact geometry uses digital hex-path intersection rather than literal geometric line intersection on the printed map.

Four: the game is two-player only. The original board game supports two or more players with a referee. Multi-player support would require lobby changes, turn ordering, and faction assignment in the user interface.

Delta-V Lore & Visual Direction

Visual identity guide for the Delta-V fleet. Not the gameplay source of truth.

For gameplay stats and legality, refer to the shared constants module where ship stats and ship classes are defined, the spec document covering rules and scenario behavior, and the architecture document covering system design.

Design Philosophy: "Symmetrical NASA-Punk"

In vacuum, form follows delta-v and orbital mechanics. Ship designs should feel engineered, practical, and zero-g-native.

- **Zero-g symmetry first:** no aerodynamic fuselages, no implied "up/down", no airplane cockpits.
- **Visible propulsion:** large engine bells, obvious thrust vectors, believable maneuvering clusters.
- **Thermal realism:** integrated radiators and cooling features rather than decorative fins.
- **Functional materials:** bare trusses, tank geometry, thermal foil, exposed service panels.

Technology Pillars

Each ship incorporates some or all of these visual elements. These are speculative sci-fi extrapolations for art direction, not engineering claims.

Turning to the individual pillars: Fusion drives are represented by massive engine bells or magnetic nozzle rings, blinding cyan or white exhaust, and confinement coil rings. Thermal management appears as flush-mounted hull panels glowing cherry-red under load, liquid droplet streams between catch-basins, and no protruding radiator fins. Directed-energy weapons take the form of long accelerator spines along the keel, superconducting magnet rings, and laser variants rendered as clustered emitter lenses behind armored shutters.

Ablative armor and Whipple shields show up as thick overlapping hexagonal outer plating, beam scoring and ablation marks, and modular replacement segments. Cryogenic fuel tanks appear as spherical tanks in clusters, wrapped in reflective multi-layer insulation and Kapton gold foil, supported by skeletal truss frameworks. Reaction control system thrusters are small quad-block nozzles at hull corners with scorch marks around the bells and short pale exhaust bursts.

Solar arrays are enormous dark-blue rectangular grids on unfolding booms — flat and fragile-looking, used on bases and auxiliary systems. Communications arrays are large parabolic dishes on articulating gimbals, clustered near the command deck. Light detection and ranging sensors appear as long booms protruding from the prow with geometric sensor heads, domed cameras, and phased-array panels. Point

defense cannons are small multi-barreled rotary cannons in armored turrets on fast-turning gimbals giving three-hundred-and-sixty-degree coverage. Ordnance tubes are flush-mounted hexagonal launch tubes in the armor plating with amber or red pre-heat glow and heavy armored hatch covers.

Fleet Visual Taxonomy

Reference boards for each vessel class are stored in the documentation assets folder.

Warships

- **Corvette** — Compact interceptor. Proportionally large fusion bell. Gunmetal with warning amber accents. Point-defense blisters and forward particle-beam spine.
- **Corsair** — Improvised raider. Mismatched ablative armor over exposed truss core. Matte navy and grey with heavy scoring. Visible torpedo tubes.
- **Frigate** — Long-range gun platform. Linear hull with heavy forward armor. Symmetrical sensor booms. Recessed torpedo bays.
- **Dreadnaught** — Heavy capital ship. Brutalist hexagonal armor. Central spine housing heavy particle accelerators. Multiple ablative shield layers.
- **Torch** — Experimental high-energy craft. Colossal magnetic confinement drive occupying roughly seventy percent of the vessel length. Blinding cyan exhaust. Obsidian heat-resistant hull.

Civilian and Utility Vessels

- **Transport** — Modular cargo hauler. Skeletal truss with detachable cargo pods. White tile and Kapton gold. Can carry and emplace orbital bases.
- **Tanker** — Fuel-centric design. Enormous cluster of spherical cryogenic tanks around a central thrust spine. Minimal crew section.
- **Liner** — Civilian long-haul. Stark white tiled hull with modular passenger habitats. Reflective observation blisters.
- **Packet** — Armed courier. Heavily armored front plate shielding cargo sections. Integrated defensive laser turrets. Metallic silver. Can emplace orbital bases.

Fixed Strategic Structure

- **Orbital Base** — Immobile fortress. Central docking and refueling hub. Vast solar array grids. Thick armor over command sectors and fixed weapon batteries.

Concept Art Assets

The documentation assets folder contains reference images for each technology pillar: fusion drive, thermal management, particle beam, ablative armor, cryogenic fuel, reaction control system thruster, solar arrays, communications dish, sensor spine, point defense cannon, and missile tubes.

Color Direction

Warships use gunmetal, navy, and warning amber as their primary palette. Industrial and civilian vessels favor white tile, metallic silver, and Kapton gold. Experimental craft in the Torch family use high-energy cyan and obsidian black.

Maintenance

Concept art assets are reference-only; gameplay code must not depend on them. This guide should be kept synchronized with the playable ship roster defined in the shared constants module. If a ship is added or removed in gameplay code, this file should be updated in the same pull request.

PART 4

Quality, Operations, and Release

Operational guidance for testing, accessibility, observability, security, privacy, reviews, and coordinated releases.

Simulation and Load Testing

Delta-V has three complementary simulation layers. Because the shared engine is side-effect-free and deterministic given an injected random-number generator, high-speed simulation is cheap and reproducible.

The three layers are: a headless engine that runs AI versus AI games, a network load and chaos harness that validates the server lifecycle and WebSocket handling, and a large-language-model agent bridge that lets external model-driven agents play real online matches.

---

1. Headless engine simulation

Goal. Run large batches of AI-versus-AI games to test balance and find crash edges deep in the game tree. This layer runs in Node with no browser or web-worker runtime.

Loop shape. The simulation creates a game, then loops until the phase reaches game-over, calling the appropriate AI helper for the active phase — covering astrogation, ordnance, logistics, and combat — and passing each result to the matching engine entry point. All engine calls take a mandatory random seed, so a given seed reproduces the game exactly.

Commands. The main simulate command runs one hundred games of the default scenario by default. Passing "all" and a count with the continuous-integration flag runs all nine scenarios at that count, failing on engine crashes while treating balance warnings as non-fatal. A separate randomize-start flag forces per-game seat randomization; the duel, interplanetary-war, and fleet-action scenarios auto-randomize seats anyway so seat-order bias does not dominate short batches. A duel sweep command runs the duel harness across many base seeds in one table, showing pacing and seat-balance variance — useful before changing duel geometry or rules.

Continuous-integration and pre-commit hooks both run the full scenario suite at sixty games each. The manual verify command uses forty games to stay responsive for local invocation. If the iteration count changes, all three should be updated together.

The duel sweep accepts options for the number of iterations, a seed range, a comma-separated list of specific seeds, a scenario key, and a JSON output flag.

CI balance warnings use per-scenario win-rate bands for decided games. Cooperative and race scenarios such as Grand Tour are excluded from balance checks.

---

2. Network load and chaos

Goal. Validate the Durable Object lifecycle, WebSocket handling, reconnection, and concurrency. This layer drives real rooms over HTTP with both seats operated by bots.

Loop shape. The harness posts to the create endpoint, opens WebSocket connections for both seats using a bot-client helper, and on each state-bearing server-to-client message, the active player thinks briefly and submits a valid client-to-server message drawn from the existing AI helpers. Chaos mode forces drops and reconnects using the stored player token.

Commands. The load-test command accepts a games count and a concurrency level. A disconnect rate option, expressed as a fraction between zero and one, enables chaos mode. For example, running twelve games at four concurrent connections with a disconnect rate of twenty-five percent exercises reconnection paths.

Local setup. If a local Wrangler D1 database predates the latest schema, migrations should be applied before long runs.

Reporting. The harness prints a per-match summary covering the result code, winner, turn count, duration, actions sent, and reconnect count. It also prints an aggregate summary covering completed and failed matches, reconnect success rate, server and socket errors, totals, and win reasons.

3. LLM / agent bridge

Goal. Let external model-driven agents — large language models, custom planners, tool-using bots — play real online matches using the same room protocol as browser clients.

The bridge can create a new match and wait for an opponent, or join an existing room by code. Running two bridge processes covers large-language-model versus large-language-model end-to-end play.

Full agent onboarding — including the choice between the Model Context Protocol (MCP) and the bridge, the contract, a reliability checklist, and a tuning workflow — lives in the agents documentation. The per-turn payload shape is defined in the agent specification.

Quick examples. One typical use is hosting a match with a standard-input/standard-output agent and sharing the printed room code with a browser opponent. Another is joining an existing room code with an HTTP agent that accepts turn requests on a local port. A third is running a baseline match with the built-in policy and no external agent, which is useful for comparison.

If the agent's output is invalid, times out, or mismatches the current phase, the bridge falls back to the built-in policy, with a configurable difficulty level.

Manual Test Plan

Hands-on verification for release candidates and significant changes. Each section is self-contained, so run only the ones relevant to what changed.

Release gate

Any of these is a blocker: a new player cannot start Bi-Planetary and finish turn one using only in-game guidance; the current player, selected ship, objective, or next required action is unclear at any point; multiplayer create, join, reconnect, chat, rematch, or disconnect resolution is flaky; mobile and touch has blocked actions, overlapping heads-up display, or unreadable text; Progressive Web App offline single-player is broken; or a twenty-to-thirty minute session shows serious stutter, dropped input, stale interface, or unclear win or loss messaging.

When a test fails, record the browser, device, scenario, seat, steps taken, and whether the failure is about correctness, clarity, performance, or recovery.

Contrast and readability spot-check

Use the browser's accessibility inspector, a color picker, or an external contrast checker on the translucent panels — not just on the solid menu backgrounds. The help overlay's stacked panels place body copy on semi-transparent layers with a backdrop blur, so aim for a ratio of four-and-a-half to one for normal-sized text against the effective background. The game-over screen's stat pills and any queue or status strips with muted text on glassy fills are the other likely candidates.

Agent and MCP smoke (pre-release, optional)

Run this when changes touch the agent protocol, the Model Context Protocol adapter, matchmaking, or the benchmark script. First, run the in-process benchmark against each built-in AI difficulty for a small number of games; from its JSON summary, confirm the action-validity rate is at least ninety-five percent, the parse-error rate is zero, and the timeout rate is below five percent. Second, run the six-agent harness and confirm it finishes three concurrent matches without Durable Object errors or stuck tickets. Third, run the quick-match agent script against each difficulty for a small number of games, confirming stable acceptance and no systematic JSON parse failures. Skip this section when the release does not touch the agent or MCP surfaces.

Recommended matrix

Test across desktop Chromium with mouse and keyboard, desktop Safari or Firefox with a trackpad, a phone-sized viewport and one real device if available, an installed Progressive Web App shell if the browser supports it, and a fresh browser profile at least once per release to cover tutorial, reconnect, audio, and help-state scenarios.

Cover single-player online, single-player offline, two-tab multiplayer, and refresh, reconnect, and rematch flows.

---

1. Smoke test (2 min, Bi-Planetary vs AI)

1. Open the app. The menu shows the title, difficulty, and play options.
2. Pick Easy, then Bi-Planetary. The status indicates first-burn fuel cost and free takeoff.
3. Click a direction arrow or press one of the number keys one through six, or zero to clear. A course preview and fuel-cost label appear.
4. Click Confirm or press Enter. The ship animates and the heads-up display updates.
5. The AI takes its turn, then yours resumes.

Pass: takeoff, movement, and AI response all work.

1a. First-time UX (fresh profile, 5 min)

Start from the menu without opening any external documentation. Within the first ten seconds it should be obvious which ship is yours, what the goal is, and what the next action is. Finish turn one without guessing what Confirm will do. Make and recover from a deliberate mistake — such as a wrong burn or wrong selection — using only in-game affordances like deselect, undo, or help. After the AI turn, within three seconds you should know the objective, the selected ship, the next action, and why the last event happened. Reach turn three without any external explanation.

---

2. Vector movement (Bi-Planetary)

Verify that velocity persists: confirm a burn, then confirm with no burn and the ship should coast at the same velocity. Confirm that a burn shifts the endpoint by exactly one hex in the chosen direction and that the fuel-cost label appears. Gravity should deflect one turn later, shown by yellow arrows in passing gravity hexes, with cyan dashed arrows appearing next turn to show deferred deflection; edge-of-hex passes do not deflect. For the Grand Tour scenario and areas past Luna or Io, weak gravity applies: a single hex is optional, but two consecutive make the second mandatory. For Duel warships, the overload option is shown with a double-circle icon, costs two fuel, and is unavailable after use until resupply. Non-warships such as the Convoy tanker and Liner cannot overload, so that option should be absent.

---

3. Landing & takeoff (Bi-Planetary)

Starting landed, take off, enter orbit, and land on the opponent's base. Booster takeoff is free but the initial burn costs one fuel. Landing requires first being in orbit. Intersecting a planet off-base results in destruction.

4. Combat (Duel vs AI Hard)

For attacks, the preview should show attacker and defender factors, odds, range modifier, velocity modifier, and whether a counterattack is available. Attack queues the action; Fire All resolves it; a result notification shows the outcome. A counterattack fires at its own odds if the defender is still eligible. Disabled ships drift for the stated number of turns, decreasing by one per turn; six or more cumulative disabled turns means elimination; landing at a friendly base repairs all damage. Planetary defense in Bi-Planetary fires at two-to-one odds against any enemy entering the gravity hex above your base, with no range or velocity modifiers. The dice penalty for range and velocity is one per hex of range and one per hex of velocity difference above two. Line of sight is blocked by planets, moons, and Sol; ships and asteroids do not block it.

5. Ordnance (Duel or Convoy vs AI)

Mines inherit ship velocity, the launching ship must change course that turn, they self-destruct after five turns, and they detonate on hex intersection. Torpedoes are warships-only, receive a one-to-two hex launch boost, target a single ship, and misses continue moving. Nukes are available only in the Escape scenario, inherit velocity, destroy everything in their hex, can be shot down by guns or point defense at two-to-one odds, and clear asteroid hexes. No ordnance may be launched while resupplied, while landed, or more than once per ship per turn.

6. Scenarios (verify each starts correctly and applies its rules)

Each scenario has its rules fully specified in the game specification document. Quick verification points follow. Bi-Planetary: land on the opposite planet. Escape: hidden fugitive ship, only nukes available with no mines or torpedoes, planetary defense disabled, and a moral victory is awarded on Enforcer disable. Convoy: liner, tanker, and frigate escort with logistics enabled; land the liner on Venus with colonists. Lunar Evacuation: passenger rescue enabled and the win requires passengers aboard. Duel: last ship standing. Blockade Runner: packet with a head-start velocity, land on Mars. Fleet Action: four-hundred credit fleet build, covered in section eight. Interplanetary War: eight-hundred-fifty credit fleet build with logistics and longer play. Grand Tour: combat disabled, shared bases, visit eight checkpoints and return home.

7. Multi-ship management (Escape vs AI)

No ship is auto-selected; the heads-up display prompts for selection. Clicking a ship shows a selection notification. Clicking stacked ships cycles through them. The Tab key cycles through your own ships. After setting burns for all ships, the status shows that all burns are set, and confirming animates them simultaneously.

8. Fleet building (Fleet Action / Interplanetary War)

The budget is shown, ship cards display stats and cost, and over-budget ships are greyed out. Removing a ship clears it from the selection; the clear button resets the entire fleet; Launch Fleet starts the match. The AI also builds its own fleet.

9. Logistics (Convoy vs AI)

Match two friendly ships to the same hex and velocity and a logistics phase appears after movement, where the transfer panel can be used. Torch ships cannot transfer fuel to other ships in the Interplanetary War scenario. The logistics phase is skipped in scenarios that do not have logistics enabled, such as Bi-Planetary and Duel.

10. Combat edge cases

When the last two ships kill each other, the non-attacker wins, indicated by a mutual destruction message. Dreadnaughts may fire while disabled; damaged orbital bases may fire at the first damage level. In Fleet Action, multiple ships can combine strength for an attack, using the highest applicable range and velocity penalties. Ships with only defensive ratings — such as the Transport, Tanker, and Liner — do not show an attack button. Ramming occurs when a course passes through an occupied hex, causing both ships to take damage from the ramming table; mines and torpedoes in that hex also detonate. For asteroid hazards, one roll is made per asteroid hex entered at speed greater than one; crossing a hexside between two asteroid hexes counts as one roll.

11. HUD & information

The top bar shows the turn number and phase name, fuel and speed, objective, and fleet count. A brief centered phase banner appears on phase change and auto-dismisses. Hovering over or tapping a ship shows a tooltip with its type, fuel, cargo, velocity, and damage. The log panel is toggled with the L key on desktop and displays color-coded, chronological entries. The game-over screen shows victory or defeat with the reason, turn count, fleet counts, and working rematch and exit buttons. At any moment you should be able to answer in three seconds whose turn it is, which ship is selected, whether the game is waiting for input or animating, and what Confirm or Fire All will do. Unavailable actions explain why they are unavailable. For accessibility basics, one-hundred-fifty percent zoom stays readable, and low-fuel, urgent-timer, and victory states are not conveyed by color alone.

12. Camera & navigation

Panning is available via drag, the W A S D keys, or the arrow keys. Zooming uses scroll, pinch, or the plus and minus keys, with a zoom range from very small to four times normal. The view auto-frames during movement. The minimap supports click-to-jump. The H key centers on your fleet; the E key focuses the nearest enemy.

13. Keyboard & focus safety

The number keys one through six set burn direction; zero clears it. Enter confirms an action or fires. Escape deselects. Tab cycles through ships. The question mark key toggles help. L toggles the log panel on desktop. The N, T, and K keys trigger mine, torpedo, and nuke respectively during the ordnance phase. E focuses the nearest enemy and H centers on your fleet. M toggles sound. W A S D and the arrow keys pan the camera. Plus and minus zoom in and out.

In the menu, Tab and Shift-Tab work correctly and a visible focus indicator is always present. Typing in the chat input does not trigger game hotkeys, and clicking back into the game re-enables them.

14. Mobile / touch (375 × 812)

In the menu, all buttons should be tappable, the scenario list scrolls if needed, and touch targets should be at least forty-eight pixels. During gameplay, the log starts collapsed as a single-line bar and expands to an overlay when tapped; all action buttons have adequate touch targets; the top bar never overflows; and the ship list scrolls without overlapping other elements. The status text should say Tap rather than Click, with no Enter or keyboard hints shown; burn circles have no number labels; and the help overlay shows touch-specific instructions only. In landscape orientation, the heads-up display compacts, the canvas remains usable, and nothing overlaps. Comfort checks: panning should not issue commands or select text; pinch-to-zoom should not zoom the browser page; opening the log, help, or chat should not hide controls behind the keyboard or safe area; and backgrounding then restoring the app should preserve layout and selection state.

15. Resupply & bases

Land a damaged or low-fuel ship at a friendly base. On the next turn, fuel should be restored, damage repaired, overload allowance restored, and ordnance reloaded. Same-turn gun and ordnance use is blocked while landed. In Grand Tour, shared bases serve both players.

16. Multiplayer (two tabs or devices)

For creating and joining: room codes, the copy-link button, and manual entry all work; an invalid code produces a clear error; a full room fails clearly. For presence and chat: the transition from waiting to playing is visible on both sides; each chat message appears once with correct attribution; objective, turn ownership, and ship state match on both sides; and the latency indicator does not clash with the heads-up display. The turn timer appears after a grace period, its styling becomes urgent near expiry, the warning sound and visual fire once, and any action resets the timer. For reconnect: refreshing one tab reconnects to the same seat; the stale tab stops receiving updates; closing and reopening within thirty seconds with a stored token continues the match; and the post-reconnect interface matches the other player's view. If one player is disconnected for more than thirty seconds, the other wins by forfeit with a clear reason. Rematch

starts a fresh match with reset state and Exit returns to the menu cleanly. After finishing two matches in the same room, the replay selector shows both matches labeled so you can navigate between them using start, previous, next, and end controls, and exiting the replay restores the latest match outcome.

17. AI opponent

On Easy the AI makes basic moves and is beatable by a beginner. On Normal it uses gravity assists and tactical choices for a fair challenge. On Hard it plays aggressively with optimal movement and uses ordnance.

Next, run the simulation suite using the command that runs forty iterations across all scenarios. The continuous integration suite uses sixty iterations. Expect zero engine crashes across all scenarios. The harness randomises the starting seat during bulk runs.

18. Sound

There should be no audio before user interaction. The M key toggles sound. Thrust, gun, explosion, phase-change, and timer-warning cues play in a timely manner and never blast on load, reconnect, or rematch.

19. Help & tutorial

The question mark key opens an overlay with sections matching the current controls. On a fresh profile, tutorial tips appear during each relevant phase; copy matches the device type using "Click" or "Tap" as appropriate; tips do not cover primary buttons and do not linger; skipping the tutorial returns control cleanly. Returning players are not forced through the tutorial again.

20. PWA / offline single-player

Install the app if the browser supports it; the launched shell should look correct. Online local AI play should work in the shell. To test offline, use browser developer tools to simulate an offline or airplane-mode state: the app shell should still load; start a single-player AI match; play at least three turns; multiplayer should fail clearly without hanging. Re-enabling the network should allow online play to recover after a retry or reload.

21. Leaderboard

Run when changes touch the leaderboard server code, the shared rating module, or any of the leaderboard database migrations.

On the public leaderboard page, the table should load ordered by rating descending, and agent rows should show an "Agent" badge. By default, provisional players are hidden; flipping the include-provisional query flag should surface them at a lower confidence. Newly created players start in the hidden bucket until their rating deviation shrinks and they meet the distinct-opponents threshold.

For human claims, a fresh browser profile should be able to claim a unique username through the home-screen callsign field, which is backed by the claim-name endpoint. Re-claiming the same username from a different player key should return a 409 conflict response.

For agent claims, minting an agent token with an associated username claim should return a token valid for twenty-four hours and mark the row with the "is agent" flag. Playing one rated match should then update the player's row.

The per-player rank lookup endpoint should return the username, rank, rating, and related fields when the player key is known, or a 404 when the key is unclaimed.

22. Edge cases / regression grab-bag

1. Zero fuel: the ship drifts at current velocity with no burn options; gravity, resupply, and map-exit rules still apply.
2. Map exit: a final course that goes off the edge of the map results in elimination.
3. Nuke clears asteroid: an asteroid hex becomes clear space after a nuke detonates.
4. Destroyed-ship cleanup: no ghost highlight or selection; clicking a wreck hex does not produce an error.
5. Empty combat phase: a skip-combat option is available, or the phase auto-skips.
6. Stacked ships: clicking cycles through all ships in the hex.
7. Turn timer: after two minutes of idle time the turn times out; a warning fires at thirty seconds.
8. Rematch: the same scenario and opponent restart with a fully cleared state.

Automated checks

These checks run in continuous integration and do not replace manual experience checks. The full local release gate runs lint, type-checking, coverage, build, end-to-end tests, accessibility checks, and simulation together. A separate command runs unit, property, and regression tests. A thin browser smoke test uses Playwright. A Playwright plus axe accessibility baseline test checks the accessibility baseline. The simulation command sweeps all scenarios for engine stability and balance, using forty iterations locally and sixty in pre-commit and continuous integration. Code-style and type-safety checks run lint and type-checking across all source files.

Playwright is intentionally small. Scenario walk-throughs and deep rule validation belong in the unit and simulation test layers, not in Playwright.

Exploratory Testing

A toolkit and technique catalogue for discovery-oriented testing — open-ended probes designed to surface bugs, documentation inconsistencies, edge cases, and ergonomics issues that pre-defined checklists miss.

This is not a release-gate checklist. For release verification use the manual test plan. For automated AI-versus-AI sweeps and load tests use the simulation testing guide. For recurring architecture and security review cycles use the review plan. For Cloudflare Worker and database internals when a probe surfaces something to triage, see the observability guide.

When to run an exploratory pass

Run one after a milestone or significant refactor that changes user-visible surface — user interface, Model Context Protocol, or public application programming interface. Run one before a release when a feature crosses subsystem boundaries, for example fleet building combined with matchmaking and a leaderboard write. Run one on a quarterly cadence even when nothing has changed, to catch production drift, third-party breakage, and documentation rot. And run one whenever a real user report hints at a class of issues worth probing more broadly.

A pass typically takes sixty to a hundred and twenty minutes of agent or human time and should produce five to fifteen backlog entries — or none, if the surface is genuinely tight.

Toolkit

Each vantage is a separate way to look at the running system. Use several per pass — single-vantage sessions miss too much.

The Browser MCP vantage, through Claude in Chrome or the Playwright preview, drives the single-page application as a real user would. It lets you inspect the DOM, the console, and the network. Connect the browser extension and open the production site.

The Local Agent MCP vantage uses standard input and output. It drives a seat programmatically and lets you inspect the raw observation, candidates, and tactical hints. You start it with the local MCP run script, as described in the MCP reference.

The Hosted Agent MCP vantage calls the same tools via streamable HTTP. It is useful for probing the deployed adapter and the agent-token flow. You mint an agent token through the token-issue endpoint, then send it as a bearer header on every Model Context Protocol call. The endpoint requires clients to accept both application JSON and text event-stream content types.

The Play skill is a higher-level autonomous play loop that sits on top of the agent Model Context Protocol. It is useful for smoke runs — but not for probing, as noted in the anti-patterns section.

The curl vantage hits public endpoints directly. Use it to probe the HTTP application programming interface surface, validation behaviour, and error shapes. The set of documented endpoints is in the well-known agent manifest.

The D1 query vantage uses Wrangler's database-execute command in remote mode. It lets you inspect or mutate the four main tables: events, match archive, player, and match rating. It authenticates through Wrangler's interactive login. For headless runs, set a Cloudflare application programming interface token with database edit scope.

The Worker tail vantage streams live Cloudflare Worker and Durable Object logs in JSON format, including Cloudflare metadata. It also authenticates through Wrangler login. Important: the tail output captures real client IP addresses, geolocation data, and TLS fingerprints, so never paste the raw output anywhere shared.

The Cloudflare dashboard logs view shows historical persisted logs — observability is enabled in the Wrangler configuration. You reach it through dashboard access. Wrangler version four has no command-line interface for historical queries, so the dashboard is the canonical path.

The R2 object inspection vantage uses Wrangler's R2-object-get command on the match archive bucket to read replay JSON and archive payloads. Bulk listing is not a command-line primitive in Wrangler version four, so use the dashboard or a throwaway Worker route instead.

Tooling is layered: confirm something looks wrong from one vantage, then triangulate with another. A bug confirmed by browser observation and agent Model Context Protocol observation and a Worker tail log line is dramatically less likely to be a misread than any single source.

Lenses (what to look for)

A lens is a question to keep mentally active while exploring. Strong lenses force you to notice things you would otherwise filter out. The ten lenses that have produced the most findings:

Validation gaps. Does every input boundary actually validate? Probe with empty, oversize, wrong-type, unknown-enum, and Unicode payloads.

Documentation versus behaviour drift. Do the well-known agent manifest, the agent playbook, the agents page, the play skill, and the manual test plan all say the same thing the engine actually does?

State invariants under failure. What happens if the WebSocket drops mid-phase? If the wait-for-turn call times out? If the user navigates away mid-game? If both seats disconnect?

Ergonomics for agents. Can a large language model agent parse the response without re-deriving geometry? Are error shapes self-describing? Are there enough labelled candidates to choose from?

Personally identifiable information and privacy surface. What user-typed strings end up in public application programming interfaces, logs, replays, and archives? Are IP addresses hashed where they should be? Is anything inadvertently long-lived?

Cost and abuse surface. Are rate limits enforced where the documentation says they are? Can a single client churn Durable Objects, fill R2, or spam telemetry?

Cross-vantage consistency. Does the browser heads-up display show the same turn, phase, and active player as the Model Context Protocol observation, the database row, and the tail log line at the same instant?

Mobile and accessibility. Does the single-page application reflow at a three-hundred-and-seventy-five-pixel viewport? Does prefers-reduced-motion or prefers-contrast actually take effect? Can keyboard-only users complete a turn?

Recovery surface. Reload mid-game — does state restore correctly? Two tabs on the same player — what wins? A stale share-code query parameter — graceful?

Public-discovery surprises. Is anything indexable, cacheable, or Wayback-archivable that shouldn't be?

Probe recipes

Concrete techniques. Combine freely — most yield more findings when chained.

R1. Public API surface scan

Sweep documented and undocumented paths in one shot. Look for unexpected two-hundred responses, which indicate leakage. Look for unexpected four-oh-four responses, which indicate broken documentation. Look for verbose error bodies, which indicate information disclosure. A small shell loop that issues head requests against roughly a dozen candidate paths — the agent manifest, the playbook, the agents page, the how-to-play page, the leaderboard, the sitemap, the robots file, the manifest, the favicon, the metrics endpoint, the health-check endpoints, and so on — prints a status code next to each path for quick eyeballing.

R2. Endpoint filter and validation probing

For every documented query parameter, send a deliberately wrong value and check the response. Silent acceptance is the bug. Probe the match list endpoint with a nonsense scenario or an absurd limit. Probe the token-issue endpoint with a mis-prefixed player key.

For state-changing POST endpoints, test rate limits and payload validation as a pair — they are often filed together because both sides of the same boundary are involved. A burst test issues thirty concurrent requests and counts successes versus four-twenty-nines, comparing against the documented limit. A payload-validation test sends empty bodies, fake values, and five-kilobyte strings.

Cloudflare's rate-limits binding is per-edge-colo and best-effort — observed limits will exceed the documented per-IP cap by five to ten times, sometimes more. That is a documentation bug worth filing whenever the gap is large. If the application programming interface silently coerces, ignores, or accepts garbage, log a finding.

R3. Doc-consistency sweep

Compare the same fact across all sources of truth. Check the agent manifest's scenarios list against the lobby's scenario select and against the simulation harness's all-scenarios run. Check the playbook's phase-action map — specifically whether each phase is simultaneous — against the play skill's "I go, you go" claims and against observed activePlayer cycling. Check the manual test plan's release-gate phrasing against current actual behaviour. And fetch every badge, screenshot, and link in the main readme to confirm they still return a two-hundred.

R4. Database and object-storage cross-check during a session

Open one terminal with the Worker tail in JSON format, and in another re-run a short SQL snapshot query a few times during and after a match. The snapshot query counts event types in the last ten minutes and orders by frequency.

Then play one match end-to-end via the browser or through Model Context Protocol. Watch three things. Telemetry counts should climb in the order the engine emits them — create, join, game-started, action-submitted, and so on — as described in the observability event catalogue. The Worker tail should emit a request line per HTTP call and a Durable Object line per console-log output; quiet phases that should be busy, or vice versa, are findings. After game-over, exactly one match-archive row should appear, at most one match-rating row, and an R2 object at the matches prefix keyed by the game identifier.

Discrepancies — missing rows, double writes, mismatched counts, log lines that don't correspond to any event — are findings.

R5. MCP edge cases

Run against a connected session, or with a fabricated session identifier to exercise rejection paths. Send actions with: unknown ship identifiers, wrong phase, malformed orders array, oversize cargo, empty attacker identifiers. Send a chat with a text field of five hundred characters — it should be rejected at the two-hundred-character limit. Send a get-observation call with a nonsense session identifier — check the error shape. Send a quick-match-connect call with a nonsense scenario — see if validation happens at queue time. After a wait-for-turn timeout, call get-state with the same session identifier — verify the session is still queryable, or, if the current behaviour drops it, verify that is intentional versus a footgun worth a backlog entry.

R6. MCP-versus-browser pairing without disturbing real users

The public quick-match queue contains real humans. Two intended pair-mates can be split across other players. Two safer options: pair the Model Context Protocol and browser seats on a less popular scenario like grand tour or interplanetary war, where the queue is usually empty — still not guaranteed. Or use a private match by code — better, but not currently exposed by the local Model Context Protocol, as tracked in the backlog under match-isolation flag. Until that lands, prefer Play-versus-AI for any test that doesn't strictly require a second human-driven seat.

When you must use the public queue, time-box it to one match, surrender immediately if you accidentally pair with a real user, and never run automated turn loops against unknown opponents.

R7. Browser scenario sweep via Play-versus-AI

For each scenario in the agent manifest, open the lobby, pick Play-versus-AI, pick Easy, and click the scenario card. Confirm the game launches without console errors. Step through one full turn — astrogation digit-1 plus Enter, then phase confirms — and watch for missing buttons, locked heads-up display, soft-locks, or layout overflow. Compare the objective copy, for example "Land on Mars" or "Destroy all enemies", against the scenario description in the manifest. Differences in objective phrasing or missing controls per scenario type are findings.

A tooling caveat for the Chrome Model Context Protocol console: the read-console-messages tool only captures messages emitted after the tool is first called in the session; messages from page load are missed. Workaround: call read-console-messages once with a throwaway pattern to start the listener, reload the page, then call again with the real pattern.

Another tooling caveat for long synchronous JavaScript loops: driving five or more scenario launches in one JavaScript-tool call frequently times out the Chrome DevTools Protocol with a forty-five-second timeout. Drive each scenario in a separate tool call instead.

R8. Live observation during a paired match

In one window, run the Worker tail in JSON format filtered to Durable Object entries. In another, drive a match. Look at three things. Durable Object console-log lines — names, frequency, payload size; anything noisy is a performance or cost finding, and anything containing user-typed strings is a privacy finding. Exception traces — even ones that don't break gameplay. Durable Object request paths — do internal routes leak in any way?

R9. Reconnect and disconnect surface

Mid-match in a browser tab: hard refresh — does the single-page application restore the same turn, phase, and selected ship? Open the same match universal resource locator in a second tab — what wins? Does the server enforce one socket per player token? Toggle airplane mode for thirty seconds, then reconnect — does the WebSocket auto-recover, or does the user have to reload? For the Model Context Protocol: kill the standard-input-output server, restart it, try get-state with the old session identifier.

R10. Mobile and accessibility triangulation

Resize the browser to three-hundred-and-seventy-five by eight-hundred-and-twelve pixels — an iPhone 13 viewport — and re-run the scenario sweep on at least one scenario. In DevTools Rendering, emulate prefers-contrast more and prefers-reduced-motion. Tab-only navigate through the lobby and one full astrogation phase. Cross-reference with the accessibility document and any open contrast-audit items in the backlog.

R11. Fresh-start: wipe persisted data between runs

For a clean baseline — for example before measuring whether a regression introduces ghost rows, or after a destructive test — confirm no live matches first, then truncate the D1 tables and the R2 archive bucket. Production-only. Confirm scope with the operator first; everything below is destructive and irreversible.

First, check that the live-matches endpoint returns an empty array. Then execute a single SQL block that deletes every row from match-rating, match-archive, player, and events.

R2 has no object-list subcommand in Wrangler version four — purge via the Cloudflare dashboard's delete-all-objects action on the bucket, via a temporary Worker route that uses the binding, or via rclone configured with R2 S3 credentials.

After the wipe, verify with the same snapshot query from the database-cross-check recipe — all four tables should report zero rows — and confirm the match-list, leaderboard, and live-match endpoints all return empty.

R12. Doc-link fetch sweep

Walk every universal resource locator referenced from the main readme, the docs directory, the agent manifest, the playbook, and the agents page. Fetch each and confirm a two-hundred. Broken outbound links and four-oh-fours on documented internal paths — the original symptom that surfaced the missing health-check and sitemap endpoints in the first pass — are documentation-rot findings even when the underlying behaviour is unchanged.

R13. Worker-tail exception triage

This was the single highest-yield probe in the April pass. Run the Worker tail in JSON format for the duration of a paired-match session, then post-filter for any chunk with a non-empty exceptions array. A short Python snippet splits the concatenated JSON stream by newlines between close-braces and open-braces, parses each chunk, and for each chunk prints every exception's name, message, and stack.

Even outcomes that look successful from the client side — game completed, archive landed — can mask thrown exceptions in Durable Object close handlers, alarm handlers, or async logging paths. The April pass surfaced a type error — "The Durable Object's code has been updated, this version can no longer access storage" — only because of this filter. Continuous delivery means every exploratory pass should include this recipe: a deploy may have just landed.

Also worth grepping for: any exception name other than expected error classes; any stack with async frames into archive, match-archive, or live-registry modules; any chunk with an exception outcome.

R14. Client-side state audit

Open the single-page application in a fresh browser profile, complete one matchmaking-paired game and one Play-versus-AI game, then enumerate everything the client persisted: local-storage keys, session-storage keys, cookies, IndexedDB databases, Cache Storage entries, the registered service worker script, and the web manifest.

For each persisted blob, ask: who owns it? When does it get pruned? What happens if the device is shared? Does it contain anything user-typed — callsign, real-name pattern? Is any authentication credential stored in plaintext local-storage? The April pass surfaced unbounded growth of the match-tokens cache and a player-profile entry storing the raw callsign indefinitely.

R16. Simulation-harness balance sweep

The simulation AI-versus-AI engine harness, as described in the simulation testing document, is the cheapest way to surface scenario-balance regressions without playing a hundred games by hand. Three representative runs: all scenarios at thirty games each under continuous-integration gates; a specific scenario at a hundred games for tighter signal; the same specific scenario with randomised starting seats to check seat-balance independence.

Read the per-scenario block: player-zero win percentage, player-one win percentage, draws or timeouts, and average turns. Useful triage rules. A player-zero or player-one win-rate outside the forty-to-sixty-percent band at a hundred games is a balance issue — the continuous-integration gate fires at forty-five to eighty-five percent for player zero, deliberately wide, but tighter thresholds catch real drift earlier. A timeout rate above five percent means the AI is stalemating — either the scenario lacks pressure or the turn cap is too short. An average-turn count below five means the scenario is being decided too quickly to be interesting; first-player edge dominates. Any engine-crash count above zero is a fail-closed result to file under architecture correctness.

The April sweep surfaced evacuation at ninety-six to three and duel at sixty to forty from this single command. Run before any AI heuristic change, and before any release.

R15. Post-game pipeline cross-check

After each completed paired match, verify the data landed in all four persistence stores within about thirty seconds. First, capture the game identifier — from the live-match response or the close-loop send-action result. Then run four database and object-storage checks: match metadata in the match-archive table, Glicko-2 rating delta in the match-rating table, updated rows in the player table for anyone who played in the last two minutes, and the full event-stream archive in R2 at the matches prefix.

Plus the public surfaces: the match-list endpoint for recent matches, and the leaderboard endpoint including provisional players.

Any of these returning empty for a game that completed in the user interface is a finding — most likely a thrown exception from the Worker-tail triage recipe interrupted the archive cascade. The April pass found the match-archive row appeared eventually — after about ninety seconds, once the alarm path reconciled. Eventual archival is acceptable but worth measuring; missing archival is a bug.

Workflow: probe, finding, backlog

The standard loop has seven steps. First, probe: run a recipe, or invent one. Capture the exact reproduction — universal resource locator, payload, response, console line, screenshot, game identifier. Second, triangulate: confirm from at least one other vantage. A finding seen from only one tool is one self-

edit away from being a tool bug, not an app bug. Third, classify: correctness, ergonomics, privacy, performance, documentation rot, or interesting-but-not-actionable. Drop the last category. Fourth, decide on actionability: if a thirty-second inline fix exists and is clearly safe, fix it instead of filing. Otherwise, file. Fifth, file an entry in the backlog — most exploratory findings land in the agent and Model Context Protocol ergonomics section or the cost and abuse hardening section; pick by primary impact. Each entry needs a short noun-phrase heading, one to three paragraphs on what was observed and why it matters and suggested fixes, and a files line listing the likely files to edit. Sixth, cross-reference: if the finding came from a recipe, name the recipe in the entry — for example, "found via the endpoint validation probing recipe". This builds confidence that the recipe is worth keeping. Seventh, close the loop: when a backlog entry ships, the commit message should reference the recipe or finding, so future passes can reuse the technique.

For high-stakes findings — security, data loss, public personally identifiable information leak — surface them to the operator immediately rather than only filing.

Anti-patterns

Things that have wasted exploratory time in past passes — don't repeat them.

Running on the public quick-match queue without time-boxing. You will pair with real users and disturb their match.

Using Worker-tail output verbatim in commits or chat. It contains real client IP addresses, geolocation data, and TLS fingerprints. Sanitise or summarise.

Trusting a single tool's observation. When the browser shows one thing and the agent Model Context Protocol shows another, confirm via Worker tail or the database before deciding which is wrong.

Letting the play skill drive a session you're observing. The skill is the system under test. If it makes a decision that masks a bug, you'll miss it. For probing, use raw send-action calls.

Long blocking wait-for-turn calls when the other seat may be slow. Local Model Context Protocol sessions can be sensitive to timeouts; prefer short timeouts — thirty seconds or less — and explicit retries, and check current behaviour in the backlog before committing to a long block.

Speculative git commits after exploratory edits. Exploratory passes usually shouldn't change source — only documentation, plus the occasional recipe-driven inline fix. If you find yourself editing engine code mid-pass, you switched modes; commit those changes separately, ideally on a dedicated branch.

Mass-purging production data without a paired live-matches check. The fresh-start recipe destroys real matches if any are in flight. Always verify zero live matches first, and confirm scope with the operator.

Adding interesting-but-not-actionable entries to the backlog. They drown out real items. If you can't write a fix, you don't have a finding yet — keep probing.

Forgetting that exploratory identities still persist in the database. Use the reserved non-public prefixes — QA, Bot, or Probe — for test callsigns. The leaderboard now filters them, but they remain queryable in operator tables and logs.

Pass log

Append a one-line entry per pass recording the date, whether the operator was an agent or human, the scope of the pass, and the count of new backlog entries filed.

On the eighteenth of April 2026, an agent pass on Model Context Protocol and browser pairing, public application programming interface surface, scenario sweep, and documentation consistency filed nine new backlog entries.

On the nineteenth of April 2026, a series of agent passes produced further entries. A rate-limit verification, payload validation, and health-check audit pass filed five. A listing-endpoint silent-caps, leaderboard validation, join-metadata, auth-failure log silence, and validation shape consistency pass filed five. An end-to-end paired-match-to-leaderboard verification pass filed four, surfacing the Durable Object deploy-eviction crash, a misleading surrender error, a matchmaker double-pair, and leaderboard test pollution. A follow-up pass tracing the Durable Object eviction and auditing the progressive web app, tutorial, and local-storage filed three more, including favicon gaps and on-device personally identifiable information. A spectator and join-flow security pass filed four — unauthenticated seat-hijack, missing spectator mode, local-game reload loss, and partial filter validation. A combat and ordnance rules-conformance pass combined with a simulation balance sweep filed four — the evacuation imbalance, the duel and biplanetary first-player edge, and the grand-tour and fleet-action timeout rates. An AI-difficulty stratification and matchmaker-seat-assignment pass filed three. A post-fix sweep confirming evacuation rebalance and the seat-shuffle ship, plus a security-header and cross-origin audit, filed two. A match-history replay-link probe filed one. A final lobby-flows pass filed one.

Accessibility (DOM) - audit guide

Delta-V uses a Canvas game board. Full non-visual play would require a parallel interface and is not currently in scope. This guide focuses on the accessible parts of the product we do control: the menu, lobby, overlays, heads-up display chrome, fleet builder, chat, and buttons.

Scope

The surfaces covered by this audit are the client user-interface modules and the DOM helper module, along with any overlay and lobby controls rendered outside the Canvas.

What is out of scope for now is the pointer-first ship and hex interaction on the Canvas board, and full keyboard-only tactical play parity.

Audit cadence

Run this checklist before release candidates, after major user-interface refactors, and after introducing new modal, form, or heads-up display interaction patterns.

Automated checks

Run both automated paths when possible. First, run a Lighthouse Accessibility audit on the main page. Second, run the end-to-end accessibility test suite, which covers the menu, waiting lobby, and heads-up display and help overlay DOM. That suite fails on serious or critical violations and excludes the Canvas and color-contrast checks, which are tracked manually in the checklist.

Pass criteria are that there are no critical or serious accessibility violations on the audited views, and that any new violations are either fixed or explicitly documented in the project backlog.

Manual checklist (pass/fail)

- **Keyboard order:** Tab order reaches the Create and Join actions and primary controls in the expected sequence.
- **Activation semantics:** Enter and Space activate controls; native button elements are preferred over clickable divs.
- **Form naming:** Room code and chat inputs have visible labels or a correct ARIA label attribute.
- **Focus visibility:** The keyboard focus ring or outline is visible on all actionable elements.
- **Modal behavior:** Focus is trapped while a modal is open and restored to the trigger element on close.
- **Contrast:** Heads-up display and overlay text is readable on translucent backgrounds, targeting WCAG AA compliance where feasible.

- **Status messaging:** Important toasts and errors use the appropriate ARIA role — either "status" or "alert" — where a signal is needed without excessive noise.

Canvas limitation note

Document Canvas constraints explicitly in pull requests and issues when relevant. Do not create placeholder keyboard-support tasks without a concrete product decision and implementation plan.

Baseline record template

When running an audit, add a short note to the pull request or backlog using the following structure: the date, the surfaces checked, the automated results, a manual checklist summary, any new issues found, and links to any follow-up tickets or backlog entries.

Latest audit

The most recent audit was conducted on April 3, 2026. The surfaces checked were the main menu, waiting lobby, heads-up display chrome, help overlay, chat and log DOM, and the toast, reconnect, and game-over overlays.

The automated end-to-end accessibility suite passed all five tests. The manual checklist results were as follows. Keyboard order passed for menu primary actions and the help overlay open and close flow. Activation semantics passed on all audited controls, with native buttons remaining the default. Form naming was fixed for the room-code and chat inputs by adding explicit ARIA labels. Focus visibility had no code regression found, though it remains worth checking on real devices and browsers during release passes. Modal behavior improved significantly: the help overlay now traps focus and restores it on close, and the reconnect, game-over, and help overlays now expose dialog semantics. Contrast remains a human visual check, especially on translucent heads-up display surfaces. Status messaging was addressed by giving toasts, the latest-log bar, the reconnect banner, and the phase alert live-region semantics.

No serious or critical automated violations were found after these fixes. Future visual and manual accessibility issues should be added to the project backlog if later passes find concrete regressions.

Automated code scan (2026-04-03)

This scan was also conducted on April 3, 2026. It covered all client user-interface view files, the main HTML entry point, and the DOM helper module. The method was a grep-based scan for clickable non-interactive elements missing a role attribute, a tab-index, an ARIA label, or keyboard handlers.

Three issues were found and fixed. First, fleet cart chips in the fleet-building view were clickable div elements that lacked a button role, a tab-index, an ARIA label, and keyboard activation — all of which were added. Second, ship list entries in the ship-list view already had a button role and an ARIA label but were missing a tab-index and an Enter and Space keyboard handler — both were added. Third, the log latest bar in the game-log view was a clickable div used to expand the game log, and it lacked a button role, a tab-index, and a keyboard handler — all three were added.

No missing alternative text on images, no missing ARIA labels on icon buttons, and no form inputs without labels were found. The TypeScript compiler check and the full Vitest suite both pass.

Follow-ups

Track concrete fixes in the project backlog. Prefer specific, testable items over broad "improve accessibility" entries.

Observability

What Delta-V emits today and how to use it for incidents and tuning. This chapter complements the security documentation on telemetry abuse and the architecture documentation on server layout.

Sources

Delta-V collects observability data from six main sources. First, Worker and Durable Object logs capture console output and errors from the main server entry point, the game Durable Object, and request handlers. These logs are readable in the Cloudflare Workers Logs interface, which requires observability to be enabled in the Wrangler configuration file.

Second, the events table in D1 — Cloudflare's SQL database — stores client telemetry, client errors, server-side event types such as engine errors, projection parity mismatches, and game abandoned rows, plus lifecycle and side-channel rows. This table operates on a thirty-day rolling retention window. Third, the match archive table in D1 holds one row per completed match as a metadata index. Fourth, the leaderboard's player and match-rating tables in D1 hold the claimed-username identity records and per-rated-match Glicko-2 snapshots. Fifth, the match archive bucket in R2 object storage holds the full JSON record for every match, one file per game. Sixth, client-side code sends telemetry to a telemetry endpoint and error reports to an error endpoint via HTTP POST.

D1 events schema and event catalog

The events table schema defines the following columns: a timestamp, an anonymous identifier, the event name, a properties field stored as a JSON string, an IP hash field that contains a hashed version of the connecting IP address for client posts or the literal word "server" for Durable Object inserts, a user-agent string, and a creation timestamp.

Current event values

Events arriving through the telemetry endpoint cover the full player journey. Game creation events include "create game attempted" with the scenario name, "create game failed" with the scenario, reason, and optional status, and "game created" with the scenario, mode, and optional difficulty.

Joining events include "join game attempted" with a flag indicating whether the player had a token, "join game succeeded" with no additional data, "join game retried without token" with the reason, and "join game failed" with the reason, optional status, and token flag.

Quick match events include "quick match attempted," "quick match queued," "quick match found," "quick match failed," "quick match expired," and "quick match cancelled," each carrying properties that vary depending on which path through the matchmaker was taken.

Spectating and reconnection events include "spectate join succeeded," "reconnect attempt scheduled" with the attempt number and delay in milliseconds, "reconnect succeeded" with the total number of attempts, and "reconnect failed" with the total attempts.

Replay events include "replay fetch failed" with the reason, game identifier, and optional status; "replay fetch succeeded" with the game identifier; and equivalent archived replay variants for both failure and success.

Game outcome events include "game over" with a won flag, reason, scenario, mode, and optional turn number.

Error and connection events include "server error received" with a message and optional code; "action rejected received" with the reason, expected turn and phase, actual turn and phase, and active player — this fires in the browser when action guards reject a move; "WebSocket parse error" with no additional data; "WebSocket invalid message" with the error text; "WebSocket connect error" with no additional data, which fires on error before the connection closes; and "WebSocket connect closed" with the close code, a clean-close flag, and the reason length.

Turn timing events include "turn completed" and "first turn completed," both carrying the turn number, total milliseconds, phase breakdown, scenario, and mode.

Onboarding events include "scenario browsed," "tutorial started," "tutorial completed," and "tutorial skipped," all with no additional properties.

Submission events include "fleet ready submitted" and "surrender submitted."

Finally, "AI game started" carries the scenario and difficulty level and is emitted for local AI play.

Events arriving through the error endpoint take the form of "client error" with an error description, the page URL, the user-agent string, and arbitrary context. Global error handlers attach either the source file location and line and column numbers, or a flag indicating an unhandled promise rejection.

Next, events emitted by the server-side telemetry module cover three cases. "Engine error" carries an error code, the current phase and turn, the message, and optionally a stack trace. "Projection parity mismatch" carries the game identifier along with the live turn and phase and the projected turn and phase, allowing comparison of the two. "Game abandoned" carries the game identifier, the current turn and phase, the reason, and the scenario name. All three of these server-side events have a null anonymous identifier and the literal string "server" as the IP hash field.

Server-side lifecycle and side-channel events

Server-side events are emitted from the game Durable Object's telemetry module and from the matchmaker Durable Object. All of them share a null anonymous identifier and the "server" IP hash value. Lifecycle events write to the standard console log stream; side-channel failures write to the error stream, making it straightforward to separate the two in Workers Logs.

Lifecycle events represent normal operating signals. "Game started" carries the game identifier, room code, and scenario. "Game ended" carries the game identifier, room code, turn number, winner, and reason, where winner is null on draws. "Disconnect grace started" carries the room code, the disconnected player, and the millisecond timestamp at which the grace period expires. "Disconnect grace resolved" carries the room code and player, and is emitted when the player reconnected before the grace period elapsed. "Disconnect grace expired" carries the room code and player, and is emitted when the grace period ran out and the engine will forfeit the disconnected player. "Turn timeout fired" carries the room code, game identifier, turn, phase, and active player. "Matchmaker paired" carries the room code, scenario, the keys for the left and right queued players, and how long each had been waiting.

Side-channel failures are worth investigating any time their counts spike. "Live registry register failed" carries the room code, scenario, and optional status and error text; it means the newly started match may be absent from the live matches listing. "Live registry deregister failed" carries the room code and optional status and error text; it means a stale "live now" entry may persist. "Model Context Protocol observation timeout" carries the handler name and the timeout duration in milliseconds, representing a ten-second hard ceiling; any occurrence suggests that future asynchronous paths could hang requests. "Matchmaker pairing split" carries an optional room code, the reason, and optional conflict data, status, and attempt count; the reason is one of room code collision, allocation failed, or max retries exceeded.

With that established, there is also a static version endpoint built into the distribution at bundle time that exposes the package version and an assets hash for support purposes. It is not written to D1.

Incident triage quickstart

Use this process when someone reports that the game is broken or when metrics look wrong.

1. Confirm scope quickly in Workers Logs: determine whether the problem affects one room code or many, and whether it is isolated to one browser or device type or is broadly observed.
2. Check for an error spike in the events table, first looking at the last twenty-four hours and then splitting by hour over the last six hours, grouping by event name to see counts for client errors, engine errors, and projection parity mismatches. The hour-by-hour split helps identify whether a deployment or an external event triggered the issue.
3. If projection parity mismatches or engine errors are rising, treat the situation as a server-authority or replay integrity risk first. Review the recent deployment and commit history and disable any risky rollout if needed.
4. If client errors are rising without any corresponding server errors, suspect a client runtime or browser-specific regression. Correlate by user-agent string and recent UI changes.
5. If match completion looks wrong, inspect the match archive table for missing or abnormal completion rows, and retrieve the full JSON record for a specific broken match from R2 object storage to examine a concrete sample.

Starter alert thresholds (tune to baseline)

These are practical defaults until formal dashboards and alerts are added.

Any engine error in a fifteen-minute window should page the maintainer. Any projection parity mismatch in a fifteen-minute window should also page the maintainer and is considered high severity.

For client errors, warn when the count in the current fifteen-minute window is more than three times the baseline for the same weekday and hour. A spike in four-twenty-nine rate-limit responses on the telemetry or error endpoints should prompt investigation of abuse or noisy clients and a review of WAF caps.

Any Model Context Protocol observation timeout in a rolling fifteen-minute window is unexpected and should be investigated; the expected rate is zero. More than two live registry register or deregister failures in fifteen minutes suggests the live registry Durable Object may be down, which means the live matches listing will lag behind reality.

A ratio of matchmaker pairing splits to total pairings above five percent sustained over one hour means the queue is hot enough to consider coalesced allocation. A ratio of disconnect grace expirations to total disconnect grace outcomes above fifty percent over one hour means the reconnect flow is failing for most players.

Operational D1 queries

These queries can be run in the Cloudflare D1 console or via the Wrangler command-line tool. Each one targets a specific operational question.

The telemetry volume query groups all events by name over the last twenty-four hours and orders them by count descending, capped at thirty rows, giving a quick overview of which event types are most active.

The match completions query looks at the match archive table over the last seven days, grouping by scenario to show the number of matches, average turns, and average duration in seconds.

The active unique clients query counts distinct anonymous identifiers in the events table over the last twenty-four hours, excluding rows where the identifier is null.

The scenario popularity query extracts the scenario name from the event properties for all game-created events over the last seven days, grouping and ordering by count descending.

The disconnects and reconnects query counts reconnect-related events — reconnect attempt scheduled, reconnect failed, WebSocket parse error, and WebSocket invalid message — over the last twenty-four hours.

The top client errors query extracts the error message from the properties of client error events over the last twenty-four hours, grouping by message and showing the top ten.

The lifecycle cadence query counts game started, game ended, disconnect grace started, disconnect grace resolved, disconnect grace expired, turn timeout fired, and matchmaker paired events over the last twenty-four hours.

The disconnect grace outcomes query computes two totals over the last seven days: how many grace periods were resolved because the player reconnected, and how many expired and triggered a forfeit. A high expired-to-total ratio means reconnects are failing.

The matchmaker split rate query computes two totals over the last seven days: successful pairings and failed splits. If splits exceed roughly one percent of pairings during a quiet period, consider coalesced enqueuing or a longer retry budget in the matchmaker Durable Object.

The matchmaker split reasons query breaks down the split counter by reason over the last seven days.

The Model Context Protocol observation timeout query counts occurrences over the last twenty-four hours. The expected value is zero; any spike suggests a hung dependency in the observation builder or Durable Object contention.

The live registry failures query counts register and deregister failures separately over the last twenty-four hours. Non-zero values mean some matches never appeared in the live listing or remained visible after ending.

The turn timeout rate query groups turn timeout events by game phase over the last seven days. Correlating this with game abandoned counts helps distinguish an away-from-keyboard player from an engine wedge.

Workers log filters

In the Cloudflare Workers Logs interface, the following text filters are useful. Filtering for "Engine error" catches console error output from game action failures. Filtering for "Inactivity timeout" surfaces room cleanup events. Filtering for "Rate limit exceeded" surfaces WebSocket abuse. Filtering for "projection parity mismatch" surfaces replay integrity issues and is considered critical. Filtering for the game started or game ended lifecycle markers lets you trace the full lifecycle of a single match. Filtering for the matchmaker pairing split marker is a useful smoke test when the matchmaker looks unhealthy. Filtering for the Model Context Protocol observation timeout marker is useful because any hit is unexpected; the handler name in the event properties identifies which path timed out.

PII / privacy stance (technical)

On the client side, the anonymous identifier is a randomly generated UUID stored in the browser's local storage. The error reporter may include the page URL, the user-agent string, and arbitrary context supplied by the caller. Call sites should keep that context non-sensitive.

On the server side, the events table stores a hashed version of the connecting IP address for client-originated rows, not the raw IP. Chat text is not written to the events table by default; it remains in-game only.

User-facing privacy policy copy is out of scope here. Any public privacy text should be aligned with the technical behavior described above. The technical privacy document covers this in more detail.

Gaps and follow-ups

There are no built-in dashboards or alerts at this time. Use Cloudflare's built-in tools, D1 data exports, or third-party monitoring tools. The operational queries documented above are the current primary means of investigation.

Rate limiting details, including the canonical rate limit table, live in the security documentation under the rate limiting architecture section. An optional cross-edge Web Application Firewall can be added if distributed abuse is observed.

Retention: events table rows older than thirty days are deleted daily by the scheduled Worker cron, which runs at four in the morning and calls the purge-old-events helper. The other tables — match archive, player, match rating, and the R2 archive bucket — have no automatic time-to-live today. Operational levers and user-deletion guidance live in the security document's data retention section.

Sampling or event caps can be added in the server's main request handler before the insert-event call if volume grows to a point where unsampled storage becomes impractical.

Security & Competitive Integrity Review

This document describes the current security posture of Delta-V with emphasis on competitive multiplayer. It distinguishes protections that are already enforced from the risks that still remain if the game were exposed to untrusted public players.

Use it as an implementation-level security baseline for engineering decisions. It is not a legal policy document.

Current Protections

Delta-V now has a materially stronger authoritative-server boundary than the original prototype.

WebSocket actions are still resolved server-side against the authoritative game engine. Hidden-identity state is filtered per player before broadcast, so the fugitive flag itself is not sent to the opponent. Room creation is now authoritative: the create endpoint initializes the room, locks the scenario up front, and rejects room-code collisions. The room creator receives a reserved player token for the first seat.

The guest seat is shared by room code or copied room link — anyone with the five-character code can claim the open seat. Reconnects require the stored player token, and seat reclamation is keyed to player identity even if the previous WebSocket connection has not finished closing yet.

Client-to-server WebSocket messages are validated at runtime before any engine handler executes, and malformed payloads are rejected instead of being trusted structurally. After a WebSocket is accepted, per-socket message rate limiting caps garbage traffic: ten messages per second are allowed, then the connection is closed. Chat is also throttled in memory, with a minimum gap of five hundred milliseconds between accepted chat messages per player.

Room codes are generated from a cryptographically strong random number generator rather than the standard non-cryptographic math random function. The join, quick-match, and match-listing endpoints share a hashed-IP probe throttle — one hundred requests per sixty seconds per isolate — so join traffic cannot exhaust other budgets. The replay endpoint uses a separate, more generous bucket of two hundred fifty requests per sixty seconds so replay traffic cannot crowd out join requests. WebSocket upgrade requests are similarly capped at twenty per sixty seconds per hashed IP. The telemetry and error reporting endpoints accept only JSON bodies, cap body size at four kilobytes, and apply their own hashed-IP window limits.

These changes make private multiplayer substantially safer than before, especially for host-seat integrity, reconnect safety, and server authority.

Remote MCP token model

Agents that connect via the hosted Model Context Protocol (MCP) endpoint use a layered two-token scheme so raw match credentials never reach the agent's large language model context.

The first token is the agent token. It is a long-lived credential that embeds the player key, lasts twenty-four hours and is renewable, travels in the Authorization header as a bearer token, and is issued by the agent-token creation endpoint.

The second token is the match token. It is a per-match credential that encrypts the room code and player token together, lasts four hours, is passed as a tool-argument field, and is minted by the quick-match tool when the caller is authenticated with an agent token.

Both tokens are signed with HMAC-SHA-256 using a server secret. The server fails closed when that secret is unset: the MCP endpoint and the agent-token endpoint return a server-misconfigured error instead of signing with a placeholder. Local development and tests opt into a placeholder by setting a development-mode flag in the deployment configuration — production deployments do not set that flag, so the placeholder path never engages there. The deployment script also checks that the required secret is present on the target environment and refuses to proceed when it is missing.

The match token embeds a hash of the issuing agent token. A leaked match token — for example, one that appears in tool-call logs — cannot be replayed by a different agent, because the server requires the matching agent token in the Authorization header.

The legacy path of passing room code and player token directly as tool arguments is preserved for room-creator users and bridge agents that do not go through the agent-token flow.

Token revocation is currently coarse: rotating the server secret invalidates every issued token. Per-token revocation lists are out of scope for the initial release. Agents that suspect a leak should re-issue credentials and rotate the secret.

Remaining Competitive Risks

1. Guest-seat claiming is still code/link based in the default flow

Current status: acceptable for friendly matches, weak for public matchmaking.

The creator seat is token-protected. The guest seat is still usually claimed through possession of the five-character room code or a copied room-link. Room-code guest joins are the deliberate product model for now — designed for friends sharing codes in conversation.

This means anyone who gets the room code before the intended guest can still occupy the second seat. This is weaker than authenticated accounts, signed invites, or tournament admin tooling, but it is appropriate for the current friendly-match scope.

The recommended next step for public matchmaking or tournament play is to add longer opaque room identifiers or authenticated invite links.

2. Room secrecy is limited by short codes

Current status: acceptable for friendly matches, weak for public matchmaking.

Room codes are collision-checked and generated from a cryptographically strong source using thirty-two possible characters — the alphabet excluding the letters I and O, plus digits two through nine. That yields roughly thirty-three-point-six million possible combinations. Short codes are a deliberate product choice designed for voice or chat sharing between friends. For public matchmaking or tournament play, longer opaque identifiers would be needed to prevent code guessing.

3. Rate limiting architecture

This section is the canonical rate-limit reference for the project.

Starting with room creation in production: five requests per sixty seconds per hashed IP, enforced through Cloudflare's global edge rate limiter, returning a four-twenty-nine status with a retry-after header on excess. The same five-per-sixty limit applies to the agent-token creation endpoint and the quick-match creation endpoint, both sharing the same global limiter.

The same five-per-sixty limit also applies to the claim-name endpoint, which shares the global create limiter.

WebSocket upgrade requests are capped at twenty per sixty seconds per hashed IP, enforced in memory per isolate. The join, quick-match ticket, match-listing, main leaderboard, and per-player leaderboard-lookup endpoints share a budget of one hundred requests per sixty seconds per hashed IP, also in memory per isolate. The replay endpoint has its own separate bucket of two hundred fifty requests per sixty seconds per hashed IP, ensuring replay traffic does not exhaust join capacity.

The telemetry endpoint allows one hundred twenty requests per sixty seconds per hashed IP with a four-kilobyte body cap, and the error endpoint allows forty per sixty seconds per hashed IP with the same body cap. Both run behind their own Cloudflare rate-limiter bindings, falling back to per-isolate in-memory limits only when the binding is absent, as in a local Wrangler dev environment.

The hosted Model Context Protocol endpoint allows twenty requests per sixty seconds, keyed on the agent-token bearer hash when one is present or on the hashed IP otherwise, with a sixteen-kilobyte body cap checked before JSON-RPC dispatch. This is enforced by Cloudflare's edge rate-limiter binding.

After a WebSocket is connected, the per-socket message limit is ten per second, enforced in the Durable Object layer and resulting in a connection close on excess. Chat messages are limited to one per five hundred milliseconds per player, enforced in memory and silently dropped when exceeded.

Room-code space is protected by the roughly thirty-three-point-six million possible combinations from cryptographic random generation, with collision checking at room creation.

Next, the architectural tiers behind this table. Four Cloudflare rate-limit bindings apply globally across the edge: the create limiter, the telemetry limiter, the error limiter, and the Model Context Protocol limiter. In production all four are live; lower environments — such as running a local Wrangler dev — fall back to per-isolate in-memory limits when a binding is absent. In-memory per-isolate limits cover WebSocket upgrades, join, replay, match-list, and leaderboard probes, plus the fallback path for the reporting endpoints. A distributed attacker spraying many edge locations could bypass per-isolate limits — optional WAF rules or additional rate-limit namespaces close that gap if needed. The per-socket Durable Object layer — WebSocket messages and chat — is enforced after upgrade inside the game Durable Object and does not scale with IP counts.

Turning to deployment recommendations: treat the checked-in deployment configuration as the production baseline. If the match-archive storage bucket does not exist, deployment should fail rather than silently shipping without replay storage. Cloudflare WAF or extra rate-limit namespaces can cap telemetry, error, join, replay, and WebSocket upgrade traffic across all edge isolates if per-isolate limits prove insufficient. A Turnstile bot challenge can be added to room creation if automated room creation becomes a problem. WebSocket upgrades already have a per-isolate hashed-IP window in application code, but they are not globally throttled at the edge; abuse is further mitigated by there being only two seats per room and by per-socket message limits once connected.

4. Cost-abuse surface (current gaps)

Distinct from competitive integrity, these are paths where a motivated attacker can drive Cloudflare billing faster than current controls throttle them. These are tracked in the backlog under cost and abuse hardening.

The telemetry and error rate limits run on Cloudflare's edge bindings in production, falling back to per-isolate maps in environments without those bindings. Accepted requests write an at-most four-kilobyte row to the events table. Retention is now enforced: a daily scheduled Worker cron, configured to run at four in the morning in the Wrangler configuration, calls the purge-old-events helper to delete rows older than the retention constant, which is thirty days.

The hosted Model Context Protocol endpoint has a rate limit and a body cap as described above, but a distributed attacker cycling points of presence still sees the per-isolate fallback when the rate-limiter binding is absent. Separately, poll storms from the quick-match tool and long-polls from the wait path can hold game Durable Objects warm within the cap.

The matchmaker Durable Object serializes the full quick-match queue under a single key with a one-hundred-twenty-eight-kilobyte ceiling. Sustained distributed heartbeats can push the queue past that limit, causing quick-match to stop working globally. The room-archiving function leaves room configuration, event chunks, and checkpoints in Durable Object storage on inactivity timeout, so abandoned rooms accumulate permanently. The replay endpoint re-projects the full event stream on every uncached request. Terminal-state responses — matches that have reached game-over — are cached for one hour at the content-delivery-network edge and one minute in-browser, so repeated scrapes of finished matches don't hit the Durable Object. Mid-match timelines remain uncached and still pay the projection cost per request.

Stores without automatic application-level retention today include the match-archive index table, the player table, the match-rating table, the full match JSON archives in object storage, and per-room Durable Object storage. The events table is the only table with a scheduled purge.

5. Bot challenge protection (optional)

Cloudflare Turnstile can be added to the room creation flow with a narrow integration surface. The three steps are: add a Turnstile widget to the create-game lobby UI, include the resulting token in the room-creation request body, and validate that token server-side via the Turnstile verification API before proceeding.

This is not currently implemented, but the integration surface is narrow — only the room-creation handler and the lobby UI need changes.

Lower-Risk Notes

Hidden information

The current hidden-identity filtering is sound for the implemented escape-style fugitive mechanic, because the hidden fugitive flag is stripped before broadcast. If the game expands toward dummy counters, concealed movement, or more scenario-specific secrets, that filtering layer will need another audit.

That becomes more important as the project moves toward event-sourced match history. Raw internal event streams should be treated as authoritative private data, not as safe spectator or replay payloads. Replay and spectator APIs should serve filtered projections or explicitly redacted event views rather than exposing the internal event log directly. Live spectator WebSockets follow the same rule in practice: clients receive a filtered game-state view using the same spectator-filtering path as regular broadcasts, not the raw append-only event stream.

Randomness

Random outcomes are server-controlled, which is the key anti-cheat requirement here. The code no longer relies on client-provided randomness, but it should still be described as server-controlled rather than as a cryptographically audited randomness system.

Frontend XSS posture

Cross-site scripting, or XSS, is the risk of untrusted content being injected into the page as executable markup. All direct HTML-injection writes are now confined behind two helper functions in the client DOM module. No production code outside that module touches raw HTML injection directly, making the boundary easy to search and audit.

Today all callers pass internally generated markup — game state, static constants, computed display strings — so no user input or external content flows through this path.

If chat, player names, modded scenarios, or other freeform text are added later, a sanitizer should be added inside the trusted-HTML helper rather than scattering raw HTML-injection calls elsewhere. For plain text, the preferred approach is to use text-content assignment or the element-builder helper's text property.

Competitive Readiness Summary

Current assessment across key areas:

Rules authority is good. Reconnect and seat-hijack resistance is good. Host-seat integrity is good. Guest-seat integrity is acceptable for friendly matches, as the room-code model is deliberate. Match availability under hostile payloads is good.

Rate limiting is good. The create, agent-token, quick-match, claim-name, telemetry, error, and Model Context Protocol endpoints run behind Cloudflare's global edge rate-limiter bindings, with per-isolate in-memory fallbacks when a binding is absent — as in local Wrangler dev. WebSocket upgrades and join, replay, match-list, and leaderboard probes have per-isolate hashed-IP windows. WebSocket message flooding is capped per socket inside the game Durable Object. Optional WAF rules can still provide cross-edge caps on the in-memory paths. XSS posture is good, with a trusted HTML boundary and no user-generated content flowing through injection paths. Room secrecy and public matchmaking readiness is weak due to short codes, and the default join and replay throttles are per-isolate rather than global.

With that established: Delta-V is well-hardened for private matches between friends. For public matchmaking, tournament play, or open lobbies, the remaining gaps are longer room identifiers, join throttling if guessing becomes real, and optional bot challenge protection.

Future Security Work

If the product scope expands beyond friendly matches, the following work is on the horizon.

Longer opaque room identifiers would support public matchmaking. Turnstile integration on room creation would provide bot protection. Account binding would enable organized competitive play. Stronger join and replay HTTP throttling — using global controls rather than just per-isolate windows — would address room-code guessing or Durable Object wake abuse if those become measurable problems. Optional global cross-edge rate limits via WAF or rate-limit namespaces would cover reporting, join and replay, or WebSocket upgrades if per-isolate limits stop being sufficient.

Concrete abuse-hardening follow-ups belong in the backlog when the current per-isolate limits stop being sufficient for the product shape.

Data retention (D1, R2, DO)

Turning to what persists today. The events table holds telemetry, errors, and server-side lifecycle rows; it runs on a thirty-day rolling retention window maintained by the scheduled Worker cron that invokes the purge-old-events helper daily. The match-archive metadata index in the D1 database holds one row per

completed match, with no automatic time-to-live. The leaderboard's player and match-rating tables hold Glicko-2 identity and per-rated-match snapshots, with no automatic time-to-live. Full match JSON archives live in R2 object storage when that binding is configured, with no automatic time-to-live. Durable Object storage holds live match chunks, checkpoints, and room configuration; this data is evicted when the Durable Object is archived on inactivity, and an optional archive copy is written to R2 at match end.

Operational controls available today include Cloudflare database export and backup, R2 lifecycle rules for tiering or deletion after a set number of days, and manual SQL deletion batches when a shorter retention window is mandated for the tables without an automatic purge.

For user deletion requests: if a jurisdiction requires erasure, the anonymous identifier field and time windows in the events table can be used to scope deletes. Match archives may require correlation by game identifier or room code. A runbook should be documented when needed. Automated purge or stricter programs are engineering and operations work for when the product requires it.

Operational References

Related documentation covers the observability setup including database schema, sample queries, and what is logged. A separate technical privacy document describes what the stack stores. The Open Web Application Security Project publishes reference guides on cross-site scripting prevention, both for general XSS and for DOM-based XSS specifically. Cloudflare's documentation covers WAF rate-limiting rules and the Turnstile bot-challenge integration.

Privacy – technical summary

Describes what the Delta-V stack stores. Not a privacy policy, legal review, or jurisdiction-specific compliance advice.

Scope

This document covers telemetry and error collection, server-side diagnostics, and match-archive storage. It does not define legal basis, consent language, or user-facing policy wording.

Client data flow

Each client is assigned a random anonymous identifier — stored in the browser's local storage — that is attached to all client reporting payloads. When an error occurs, the reporting helper sends the error itself, any context the caller supplies, the page URL, and the browser user-agent string. When an event is tracked, it sends an arbitrary event name along with any associated properties. A firm requirement at every call site is that secrets, credentials, and direct personal data must never appear in telemetry or error properties.

Server storage

The server exposes two endpoints for receiving telemetry and error reports, each accepting JSON payloads up to four kilobytes. Before any data is written to the database, the client's IP address is transformed into a hashed value using SHA-256 and then truncated; the raw IP address is never written.

Events are stored in a database table that holds a timestamp, the anonymous client identifier, the event name, the event properties, the hashed IP, and the user-agent string.

Turning to server-side diagnostics: events generated by the game engine — such as engine errors, state-projection mismatches, abandoned games, and other lifecycle events — are written with a fixed server sentinel in place of the hashed IP field. These diagnostic payloads may include stack traces.

Match and gameplay data

Match metadata is stored in the main database. Completed match archives are stored in object storage, keyed by game identifier. Chat messages are transmitted over WebSocket only and are not written to the database through the normal telemetry or error path.

Leaderboard data

Humans claim a leaderboard username through the claim-name endpoint, and agents claim one as part of the agent-token mint flow. Either path binds a client-held player key to a user-chosen username in the player table. The "is agent" flag is set only from the verified agent path, never from the player-key prefix alone.

A second table, called match rating, stores one row per rated match. Each row records the game identifier, both participating player keys, the winner, and Glicko-2 rating snapshots from before and after the match. Rows are keyed on game identifier, so a replay or retry of the same match is an idempotent no-op.

The public leaderboard API — both the main listing and the per-player rank lookup — exposes only the username, the Glicko-2 rating triple, games played, distinct opponents, the timestamp of the last rated match, and the "is agent" flag. Player keys are never returned in public responses.

Operational note

Access to logs, database query surfaces, and match archives is restricted to trusted maintainers. The product ships a short in-app operational disclosure describing anonymous diagnostics and completed-match archives. That disclosure is not a substitute for legal review.

Further reading

The security document covers abuse controls, endpoint hardening, and data retention. The observability document covers the telemetry event catalog and incident queries.

Cross-Cutting Review Plan

A recurring checklist for reviewing aspects of Delta-V not covered by day-to-day feature work. Concrete follow-up work belongs in the backlog document.

How to use: pick a section, run the steps, and update the review log with the date and status — either pass, fail with a backlog item, or partial.

When to run: after major architecture, protocol, or deployment changes; before release candidates; or on a periodic cadence such as monthly or quarterly.

All sections are independent and self-contained, so they can be run concurrently by separate agents with no ordering dependencies between them.

Items that require a human reviewer are marked with the Human tag. Everything else is agent-executable.

Related documents include the architecture guide, the security guide, the backlog, and the manual test plan.

---

1. CI and local development friction

Goal: pre-commit and continuous integration run cleanly; no hooks need skipping.

Steps

1. Run the full verification command end-to-end. Pass: exits cleanly with no errors. Fail: file a backlog item with the failing step and error message.
2. Run the test coverage command three times in a row. Pass: no file-not-found or stale-merge failures across any of the three runs. Fail: check the Vitest and coverage provider versions for known issues, and consider enabling coverage cleanup or using a stable reports directory.
3. Run the pre-commit hook while a dev server is already running on the default local port. Pass: the end-to-end test suite uses a dynamic port and does not conflict. Fail: check the dynamic port logic in the pre-commit hook configuration.
4. Check that the contributing guide documents what the pre-commit hook runs, what to do when it fails, and how to invoke the full verification command. Pass: all three are covered. Fail: update the contributing guide.

---

2. Observability, data lifecycle, and privacy

Goal: know what is stored, how long it lives, and whether the implementation matches the documentation.

Scope: This review covers the D1 database tables for events and match archives, the R2 object store for match files, Durable Object ephemeral storage, client telemetry, anonymous identifiers, IP hashes, user-agent strings, and chat text.

Steps

1. Read the insert-event function in the server reporting module and the database migrations; list every event type written to D1. Cross-check against the observability documentation. Pass: the doc lists all event types. Fail: update the doc.
2. Read the client telemetry module; list every telemetry payload shape sent from the client. Cross-check against the observability documentation. Pass: the doc lists all payloads. Fail: update the doc.
3. Read the match archive module; confirm the R2 key pattern and what data is stored. Cross-check against the data retention section of the security documentation. Pass: the retention policy matches the implementation. Fail: update the doc or file a backlog item.
4. Search for anonymous ID, IP hash, user-agent, and chat references across the source directory; list every location where personally identifiable information or user-generated content is persisted. Cross-check against the privacy technical documentation. Pass: no undocumented PII storage. Fail: update the doc.

---

3. Security posture

Goal: rate limiting, input validation, and trust boundaries match what the security documentation claims.

Steps

1. Read the rate-limit constants in the WebSocket socket module — specifically the WebSocket message rate limit and the chat rate limit — and also read the rate-limit constants in the server reporting module, which cover game creation, join probes, replay probes, WebSocket connections, telemetry, and error reports. Cross-check all values against the security documentation. Pass: all values match. Fail: update the doc or the code.
2. Read the client-to-server message validation function in the shared protocol module. Confirm every client-to-server message type has validation. Pass: no unvalidated message types. Fail: add validation or file a backlog item.
3. Search for inner-HTML assignments outside the client DOM module. Pass: zero hits — the pre-commit hook also enforces this boundary. Fail: move any hits to use the trusted-HTML setter.
4. Search for uses of the Math dot random function in the shared engine directory, excluding tests and injected default random-number-generator fallbacks. Pass: no remaining hits. Fail: replace with the injected random-number generator or narrow the exception intentionally.
5. Read the input-limit constants in the shared protocol module — covering maximum fleet purchases, maximum astrogation orders, maximum ordnance launches, and maximum combat attacks. Confirm they are enforced in runtime message validation before any engine dispatch. Pass: all limits are checked before any engine handler runs. Fail: add the missing validation or file a backlog item.

6. Check room-code generation in the server directory — confirm it uses a cryptographic random-number generator, not Math dot random. Pass: uses the Web Crypto API's get-random-values function or an equivalent. Fail: fix it.

---

4. Game engine correctness

Goal: engine rules match the specification; simulation does not surface logic errors.

Steps

1. Run the simulation command for all scenarios with one hundred iterations in continuous-integration mode. Pass: exits cleanly with no engine errors in output. Fail: investigate the error details and file a backlog item.
2. Read the current rule-owning engine modules — covering astrogation, combat, logistics, ordnance, movement resolution, post-movement processing, turn advance, and victory — and include fleet building and game creation modules when scenario setup rules have changed. Cross-check phase transitions, post-movement resolution, and victory logic against the game specification. Pass: no contradictions. Fail: file a backlog item noting the specification versus implementation discrepancy.
3. Run the test coverage command. Check coverage for executable engine modules, ignoring type-only or re-export shims. Pass: no executable engine module is below eighty percent line coverage. Fail: identify untested branches and file a backlog item.

---

5. Error handling and resilience

Goal: disconnects, Durable Object alarm failures, and database or object-store errors do not crash games or lose state.

Steps

1. Read the alarm and turn-timeout modules. Confirm every alarm handler has a try-catch block and reschedules on error rather than crashing. Pass: all paths are wrapped. Fail: add error handling.
2. Read the match archive module. Confirm that object-store put and get operations and database insert failures are caught and do not block gameplay. Pass: failures are fire-and-forget with logging. Fail: add error handling.
3. Read the WebSocket and socket modules. Confirm that invalid JSON is caught, rate-limited sockets are closed cleanly with the appropriate close code, and unhandled message errors return typed errors to the client. Pass: all three hold. Fail: add handling.
4. Read the disconnect and reconnect logic, including the grace period handler. Confirm a disconnected player can rejoin within the grace window and resume. Pass: state is preserved and resent on reconnect. Fail: file a backlog item.
5. Run the server Durable Object tests with verbose output. Pass: all pass. Fail: investigate.

---

6. Bundle weight and client runtime

Goal: know the cost of loading the client; avoid surprise regressions.

Steps

1. Run the build command. Record the raw size and gzip-compressed size of the client bundle output. Pass: the gzip size is within twenty percent of the baseline recorded in the architecture documentation. Fail: investigate new dependencies or dead code and file a backlog item.
2. Check for obvious runtime package imports by scanning client source files — excluding tests — for bare package import statements. Compare any hits against the package manifest and lock file changes since the last review. Pass: no unexpected new runtime dependencies in the client bundle path. Fail: evaluate whether the dependency is justified.

Human: Take a Chrome DevTools heap snapshot after twenty or more turns in a live game and check for unbounded growth. This requires a running game in a browser and cannot be automated.

7. Supply chain and release hygiene

Goal: predictable upgrades and a clear vulnerability response process.

Steps

1. Run the npm security audit command. Pass: no high or critical vulnerabilities. Fail: fix the issue or document the accepted risk in the backlog.
2. Compare the Node version across the Node version manager configuration file, the package manifest engines field if set, and the continuous integration workflow. Pass: all match. Fail: align them.
3. Run the outdated packages command. Flag any dependencies more than two major versions behind their current release. Pass: nothing critically outdated. Fail: file a backlog item for the upgrade.

Review log

The review log records each completed pass over a section. As of the most recent review in early April 2026, all seven sections received a passing status with one exception: the bundle and runtime section was marked partial because the client bundle gzip size of approximately one hundred thirty-two kilobytes was within the baseline, but the human-only heap profiling step remains outstanding.

Specific notes from that review:

- Section one, continuous integration friction: the full verification command exits cleanly, the pre-commit dynamic port logic is confirmed working, and the contributing guide was updated to document the grep-based boundary checks.

- Section two, observability and privacy: documentation was synced to current database event names, telemetry payload shapes, retention notes, and sample queries.
- Section three, security posture: all six checks passed — rate limits match the documentation, message validation is exhaustive, no inner-HTML or Math-random leaks were found, and room codes use the cryptographic random-number generator.
- Section four, game engine correctness: the simulation command ran one hundred iterations with zero crashes; the specification cross-check was clean; coverage for the combat and conflict modules was above eighty percent.
- Section five, error handling: one hundred thirty-four Durable Object tests pass; the game alarm runner now has a top-level try-catch with reschedule; five new error-handling tests were added.
- Section six, bundle and runtime: the client bundle was six hundred fifty-five thousand bytes raw and one hundred thirty-five thousand bytes gzip, within the baseline; runtime heap profiling remains a human-only item.
- Section seven, supply chain: the security audit found zero vulnerabilities; no packages were two or more major versions behind; Node version twenty-five is consistent across the version manager file and continuous integration.

Next, decisions already recorded elsewhere that do not require recurring review:

- Internationalization: English-only, recorded in the architecture documentation.
- Protocol compatibility: same-version deploy model, recorded in the architecture documentation.
- Replay and simulation parity: covered by the coding standard.
- Accessibility: a human-only manual keyboard and screen-reader audit per the accessibility documentation; automated checks are available via the end-to-end accessibility test command.

Caveats

- Numbers in the documentation — such as bundle size and Node version — are baselines that go stale. Update them alongside meaningful changes.
- Technical documentation is not a legal or compliance sign-off; counsel and public notices remain outside this repository.

Coordinated release checklist

Delta-V ships the Worker and static assets as a single version line, as described in the architecture documentation. Use this checklist whenever you increment the game-state schema version, change the shape of server-to-client or client-to-server protocol messages, alter the semantics of replay projection, or run a database migration that touches live tables.

Engine and protocol

Update the schema version constant in the shared domain types module and adjust any dependent validators in the shared protocol module. Then run the full type-check pass and the test coverage suite to confirm nothing is broken.

Replay and recovery

Extend or adjust the projector and archive tests inside the server game-do folder if the meaning of the event stream has changed. If checkpoints or envelope layout changed, manually spot-check one archived match using either the replay route or an export from the object-storage bucket.

Agents and MCP

MCP here stands for Model Context Protocol. Refresh the agent playbook configuration file and any agent-facing documentation if the set of legal actions or phase rules has changed. Then run the MCP and bridge smoke test, following the quick-start steps in the agents documentation, against either a local or staging Worker.

Client bundle

Run the build command so that the version manifest picks up a new assets hash. Confirm that the main HTML entry point's query-string cache-busting parameters reference the new hash.

Deploy

Deploy the Worker and static assets together using the deploy command or the continuous-integration deploy job. After the deploy completes, request the version manifest from the deployed host and confirm that the package version and assets hash both match what you expect for this release.

If old HTML is cached at the edge, a mismatch between the assets hash and server behavior is a strong hint that something went wrong. Correlate that with spikes in the database client-error log or telemetry to pinpoint the issue.

PART 5

Agents and Integrations

Everything needed to understand the agent-facing APIs, MCP integration, and the deeper agent protocol model.

Delta-V Agents: Practical Guide

The fastest path to a working Delta-V agent. Start here, and read the deeper documentation only when you need it. Three companion documents are available: a deep protocol and design reference, a catalog of Model Context Protocol tools and host configuration, and a guide to large-scale simulation and load and chaos testing.

Choose an integration path

There are three ways to connect an agent to Delta-V. The first, and recommended, option is the Model Context Protocol path — referred to as MCP, which stands for Model Context Protocol — because it provides the easiest robust loop and includes legal action candidates. The second option is the Bridge path, which is well suited to custom command or HTTP-based agents. The third option is a raw WebSocket connection, which gives maximum control at the cost of maximum implementation work.

Quick start (MCP path)

Starting with the MCP path involves two stages. First, you launch the MCP server using the appropriate run command. Second, your agent runs a loop: it connects to a quick match, waits for its turn, picks a candidate action or supplies a custom action, sends that action, and repeats until the game ends. Once the game is over, it closes the session.

Quick start (bridge path)

With the bridge path, one process acts as the host and another joins. The host creates a game in a chosen scenario — for example, a duel — and wires in an agent command to drive decisions. The joining process connects using the game's short code and similarly wires in an agent command.

Several optional flags are worth knowing. You can set a decision timeout in milliseconds to cap how long the agent is allowed to think. A think delay adds a brief pause before each decision. Disabling automatic chat replies is recommended for autonomous test runs. A verbose flag produces detailed output for debugging.

Agent contract (what your model/process receives)

Bridge agents receive a structured turn input that includes a version number, the game code, the player's identifier, the current game state, a list of candidate actions, the index of the recommended candidate, and optionally a summary and legal metadata. The agent must return one of two things: either an object specifying a candidate index, or an object carrying a raw client-to-server action.

The key source of truth lives in three places: the observation builder module in the shared agent directory, the protocol types module in the shared types directory, and the bridge loop script.

Reliability checklist (high value)

Several practices are high value for reliable agent behavior. Prefer candidate actions unless you need custom tactical logic. Always guard against stale turn or phase information — do not assume state is unchanged after a thinking delay. Treat action rejection as normal runtime behavior and re-decide based on fresh state. Keep chat output low-noise during autonomous scrimmage runs. Record per-game metrics such as the action rejected count, ordnance mix, and number of turns, for later tuning.

Recommended tuning workflow

1. Run a small live batch of two to five games and export the results as JSON.
2. Identify one problem class — for example, a stale opening action, over-aggressive ordnance use, or chat noise.
3. Apply one targeted change.
4. Re-run and compare four metrics: rejection rate, win split by seat, average number of turns, and ordnance composition.

Common pitfalls

Turning to the things most likely to go wrong: matchmaking can split in dual-queue scripts, so use retry pairing logic. Stale first-turn sends are a common source of errors — re-check the current phase before sending. Chat echo storms can occur if automatic replies are not disabled or heavily gated. Hidden-state leaks are a risk if you rely on anything other than the server-provided seat-scoped observations.

Where to make changes

The codebase is organized so that related concerns live together. Runner behavior and retry logic live in the bridge runner and scrimmage scripts. Agent policy logic lives in the agent scripts. Shared tactical features and candidate generation live in the shared agent directory. MCP server behavior lives in the MCP server script.

Delta-V MCP Reference

This document is the Model Context Protocol server and tool reference only.

For onboarding and workflow, refer to the agents quick-start and tuning workflow guide, and to the deep protocol and architecture specification at the repo root.

Transports

Delta-V exposes three transports for MCP. The local stdio server runs as a subprocess per agent and speaks JSON-RPC over standard input and standard output; it is stateful and manages per-session WebSockets plus a buffered event stream through its list-sessions, get-events, and close-session helpers. The hosted HTTP endpoint receives POST requests at the MCP path on the production domain, serves streamable-HTTP JSON-RPC responses without server-sent events, and is stateless per request, using a layered scheme of long-lived agent tokens as bearer credentials and short-lived match tokens passed as tool arguments. Finally, a local HTTP variant reproduces the hosted flow through Wrangler dev, which is useful when you want to validate the hosted path without deploying. The security document covers the full token model, including HMAC-SHA-256 signing with the agent-token secret.

Discovery endpoints

The Delta-V server exposes three discovery endpoints. Agents can find the server at the agents listing path, the well-known agent descriptor path, and the agent playbook path, all hosted under the main Delta-V domain.

Running the local MCP server

To start the local MCP (Model Context Protocol) server, run the package's MCP start script from the project root. By default the server connects to the hosted Delta-V service. To point it at a local instance instead, set the server URL environment variable to your local address before running the same script.

MCP host config

The repository includes a Cursor editor configuration file that sets up an stdio MCP server with the working directory pointed at the project folder. Open the project folder in Cursor and enable that MCP server if it is not picked up automatically.

The preferred host configuration names the server "delta-v-mcp", runs the package manager's MCP script, and sets the working directory to the project root. A fallback configuration is available for hosts that ignore the working directory setting — it passes the project path as a prefix argument to the package manager instead.

Tool catalog

All tools accept a session identifier unless otherwise noted. The following tools are available.

The quick-match connect tool queues and connects a seat. It accepts a scenario name, a username, and an optional player key, and returns a session identifier, a room code, a player identifier, a player token, and a status value.

The list sessions tool returns a list of active local sessions and takes no arguments.

The get state tool returns the raw authoritative game state along with the latest event identifier, given a session identifier.

The get observation tool returns an agent observation payload. In addition to the session identifier, it accepts optional flags to include a summary, legal action information, tactical data, a spatial grid, and candidate labels. It returns an object compatible with the agent turn input format.

The wait for turn tool blocks until an actionable turn window is available. It accepts the same session identifier and optional include flags as the observation tool, and returns an observation payload. It throws on timeout and may also return or reject when the game reaches a game-over state.

The get events tool reads from the buffered event stream. It accepts a session identifier and optional arguments to start after a given event identifier, limit the number of results, and clear the buffer. It returns a list of events and a count of remaining buffered events.

The send action tool submits a client-to-server action. It accepts a session identifier and an action object, and returns the action type along with a richer action result when that feature is enabled.

The send chat tool sends a chat message. It accepts a session identifier and a text value (also aliased as "message") and returns the sent text.

The close session tool closes a local MCP session. It accepts a session identifier and returns a confirmation.

Next, a few notes on the catalog. The wait-for-turn tool throws on timeout and may return or reject when the game reaches game-over. The get-events, list-sessions, and close-session tools are local-session helpers that follow the stdio MCP ownership model. The get-observation tool is the preferred read surface for most agents; the get-state tool is the lower-level alternative.

Rate limits and body caps

Both the hosted MCP endpoint and the HTTP APIs your session uses are throttled at Cloudflare's edge and inside the Workers. The canonical numbers — per route, per window, per scope, and what the server does on exceed — live in the security document. A few highlights agents should internalize. The hosted MCP path allows twenty requests per minute, keyed on the bearer token hash when present or the hashed client IP otherwise, with a sixteen-kilobyte JSON body cap checked before any JSON-RPC dispatch. The quick-

match and agent-token endpoints share a per-IP bucket of five requests per minute in production. After a WebSocket connects, the Durable Object enforces ten messages per second per socket; exceeding that closes the socket with protocol code one-thousand-eight. Local stdio MCP inherits the same limits once it opens a WebSocket to the server. In practice, agents should space out their tool calls rather than learn the limits from 429 responses or size rejections.

The send action tool – payload examples

The send action tool accepts different action types depending on the current game phase.

For astrogation, you supply an action of type "astrogation" along with a list of orders. Each order names a ship by identifier and specifies a burn value and an overload value.

To skip the ordnance phase, you supply an action of type "skipOrdnance" with no additional fields.

For combat, you supply an action of type "combat" along with a list of attacks. Each attack names one or more attacker ship identifiers and a single target ship identifier.

Delta-V Agent Spec

> Machine-native multiplayer — how AI agents join, observe, and play Delta-V.

This spec defines how autonomous agents — scripts, large language models, reinforcement-learning models, and humans-in-the-loop — interact with Delta-V as first-class players through a typed, discoverable protocol.

Contents

1. Design Principles
2. Architecture
3. Agent Contract
4. Observation Model
5. Action Model
6. Spatial Representation
7. Match Lifecycle
8. Human-Agent Coaching
9. Evaluation and Benchmarking
10. Discovery and Metadata
11. Security and Authentication
12. Integration Paths
13. Roadmap

---

1. Design Principles

Delta-V is **agent-native**. Every decision process — script, large language model, reinforcement-learning model, or human — joins as a legal player through a typed protocol, not through the rendered user interface. Six rules govern the agent interface.

First, stable identifiers over positions: every entity — ship, ordnance, celestial body — has a persistent identifier, so agents never infer identity from screen coordinates.

Second, machine state plus natural language: observations include structured data and a human-readable summary field, and agents pick whichever format suits their architecture.

Third, legal actions, not trial and error: the server always returns a list of candidates or an action mask, so agents never have to guess what is permitted.

Fourth, coarse and meaningful actions: the astrogation phase uses ship-level orders rather than freeform keypress sequences.

Fifth, causal metadata in results: every action response carries an accepted flag, a reason, the turn it was applied, and a list of effects, so agents can close the feedback loop without re-parsing the world state.

Sixth, rules, logs, and replays as addressable resources: game knowledge is structured data, not prose buried in a prompt.

The game server is authoritative. Agents submit inputs; the server validates, resolves, and broadcasts. Agents cannot bypass the rules a human seat follows. There is no privileged admin channel, no hidden-state leak, and no debug overlay on agent sessions.

---

2. Architecture

The system has three layers: an authoritative game core, an agent adapter, and a human user interface. The core runs as a Cloudflare Durable Object and owns all state, rules, clocks, and random-number generation. Both the agent adapter and the human interface consume the same authoritative state projections from that core — neither has privileged access.

The agent adapter communicates over the Model Context Protocol (MCP) using standard input and output or HTTP, or over a raw WebSocket. The human interface runs in the browser using a canvas renderer and DOM overlays. The adapter adds agent ergonomics — pre-computed candidates, legal action masks, summaries, and phase-guarded submissions — but never bypasses the same validation the browser client uses. The engine is side-effect-free, and any new agent surface delegates validation to that shared engine.

---

3. Agent Contract

3.1 Tool catalog

The canonical tool catalog for both local MCP (using standard input and output) and remote MCP (over HTTP) is described in a dedicated document. The canonical turn loop is: connect to a quick match, wait for your turn, pick a candidate, send the action, then repeat.

The wait-for-turn tool blocks server-side until it is that agent's turn, so agents do not need to poll. The send-action tool automatically stamps submission guards by default, so stale submissions are rejected with fresh state rather than silently accepted.

3.2 Resources

URI-style read-only resources are planned but not yet served over MCP. When available, they will cover the current ruleset, scenario-specific rules, per-match observations, per-match event logs, per-match replay timelines, and the public agent leaderboard.

Until resources ship, agents should fetch the agent discovery manifest and the agent playbook at startup, then use the get-observation tool or the replay HTTP endpoint at runtime.

---

4. Observation Model

Observations are the per-turn payload an agent consumes. The shape is a superset built up in layers — agents opt into as much detail as they need.

4.1 Base – AgentTurnInput

Every agent path produces the same base payload. It includes a version number, the game code, the player identifier (zero or one), the authoritative game state filtered for that viewer, a list of pre-computed legal candidate actions, the index of the built-in AI's preferred choice, an optional human-readable summary, and optional legal-action metadata.

4.2 Optional layers

MCP tools for getting an observation, waiting for a turn, and sending an action all accept include flags that layer richer data on top of the base payload.

Setting the include-summary flag adds a Markdown situation report in the summary field. Setting include-legal-action-info adds per-action legality metadata. Setting include-tactical adds a tactical block described in the next section. Setting include-spatial-grid adds an ASCII hex-grid view that respects fog-of-war. Setting include-candidate-labels annotates each candidate with a human-readable label, a reasoning string, and a risk level.

4.3 Tactical features

The tactical block exposes six pre-computed features: the distance to the nearest detected enemy (or null if none), the fuel advantage over the opponent, the distance to the objective, the enemy's distance to the objective (or null if unknown), the threat axis as a directional string, and the estimated turns to reach the objective.

These are computed by the tactical features module. Agents that derive the same values independently can skip the flag; the point is that every agent can have them without rederiving from raw game state.

4.4 Action results

When the send-action tool is called with wait-for-result enabled, it returns an action result. That result carries an accepted boolean, an optional reason string, the turn the action was applied on, the phase it was applied in, a list of visible effects, and optionally the next observation.

Setting include-next-observation closes the decide-act-observe loop in a single tool call — an agent never needs a separate observation fetch after acting.

4.5 Token efficiency

Large language model agents pay per token, so the observation format affects reasoning quality, latency, and cost.

Structured JSON has the highest parsing precision but the highest token cost, and is best used for coordinates, velocities, fuel values, and action payloads. Markdown has moderate precision at roughly 35 percent fewer tokens than JSON, making it well suited for the summary and tactical narrative fields. ASCII grid representations have a low token cost and are best used for the spatial grid, where visual relationships matter most.

Observations are hybrid: structured data for machine-readable fields, Markdown for the summary, and ASCII for the spatial grid. Passing the compact flag trims roughly 40 percent of the payload at the cost of spatial and narrative context.

---

5. Action Model

5.1 Phase-action map

Scenario keys follow a camel-case naming convention. Unknown keys fall back to the biplanetary scenario in the key-normalization function — always use the values returned from the discovery endpoints rather than hardcoding scenario names.

The six phases and their permitted action types are as follows. During fleet building, both players simultaneously submit a fleet-ready action. During astrogation, both players simultaneously submit astrogation or surrender actions. During ordnance, the active player sequentially submits an ordnance action, a skip-ordnance action, or an emplace-base action. During combat, the active player sequentially submits a begin-combat, combat, or skip-combat action. During logistics, the active player sequentially submits a logistics or skip-logistics action. During game over, either player can submit a rematch action.

The agent playbook file contains per-phase payload shapes, and the protocol types module contains the authoritative discriminated union for all action types.

5.2 Submission guards

Every action can carry an optional guards field. The expected-turn field causes the server to reject the action if the server turn has advanced. The expected-phase field causes rejection if the server phase has changed. The idempotency-key field prevents duplicate processing within the same phase.

On a mismatch the server replies with an action-rejected message that includes the rejection reason — stale turn, stale phase, wrong active player, or duplicate idempotency key — along with the expected and actual values, and fresh game state so the agent can re-decide immediately.

The bridge automatically stamps guards on every outgoing action and re-schedules its decision when a rejection arrives. The MCP send-action tool auto-fills guards from the most recent session state by default; pass auto-guards false to hand-craft them.

5.3 Fallback

On agent timeout, which defaults to 30 seconds per turn, the server applies the built-in AI's recommended candidate so games keep progressing when a language model call hangs. The timeout is configurable per session.

---

6. Spatial Representation

Large language models reason poorly about spatial relationships from raw coordinates alone. ASCII grid representations measurably improve spatial decision-making.

Setting the include-spatial-grid flag adds an ASCII hex rendering from the agent's perspective. The grid uses a symbol legend: the at-sign for your ship, an exclamation mark for a detected enemy, an asterisk for a celestial body, a tilde for a gravity well, a middle dot for an empty hex, the letter T for your target, the letter H for your home, a directional arrow for your velocity vector, and the letter x for ordnance. Below the grid, a legend lists every marked entity with its identifier, type, position, velocity, and key statistics.

Rendering rules: the grid centres on the agent's fleet centroid and expands the viewport to include all own ships, detected enemies, celestial bodies, and the target body. Velocity vectors are marked with directional arrows. Undetected enemy ships are omitted under the same fog-of-war rules as the human client. Distances use the axial hex distance formula so agents can cross-check them against the structured position fields.

---

7. Match Lifecycle

A match follows seven steps.

First, join: connect to a quick match by providing a username and scenario, or create or join a private match directly.

Second, wait: call wait-for-turn with your session identifier and receive an observation. This call blocks server-side until it is your turn — no polling required.

Third, observe: inspect the tactical block, the spatial grid, the list of candidate actions, and the legal-action metadata in the observation.

Fourth, decide: either pick a candidate index from the pre-computed list (the simple path), or compute a custom action from the legal-action information (the advanced path).

Fifth, act: send the action with expected turn and expected phase guards. The response is an action result containing an accepted flag, a reason, a list of effects, and optionally the next observation.

Sixth, loop: if the game is not over, return to step two.

Seventh, post-game (optional): fetch the replay timeline via HTTP to retrieve the full sequence of events, and persist lessons for cross-session learning if you are running a coach agent.

Next, the timing constraints. The decision timeout is 30 seconds by default, configurable per session. The quick-match poll interval should be between 500 milliseconds and 2 seconds, respecting rate limits. The WebSocket idle timeout is 120 seconds, requiring a ping keep-alive. The per-socket message rate is capped at 10 messages per second, enforced server-side.

Reconnection: the WebSocket can drop and reconnect using the game code and player token. The bridge and MCP handle this transparently — an agent experiences a brief wait-for-turn delay rather than a connection failure.

---

8. Human-Agent Coaching

A hybrid play model lets a human coach an AI agent during a live match. Instead of choosing between full manual play and full autonomy, the human acts as strategic commander — setting intent, watching execution, and intervening at decision pivots.

Loop.

1. Setup — the human configures the agent with a strategic brief.
2. Autonomy — the agent joins, observes, and acts using its own reasoning.
3. Monitor — the human watches via spectator mode or the agent's decision log.
4. Whisper — the human sends a chat message with a slash-coach prefix followed by their directive.
5. Resume — the agent integrates the directive into its next decision cycle.

An example directive might read: disengage from Mars, redirect all ships to intercept at the asteroid belt — they are overextended on fuel.

Implementation

On the server side, the coaching flow works as follows. The server parses any chat message beginning with the slash-coach prefix. It stores the directive text under a key targeting the opposite seat — the seat being coached — in durable storage. The directive is not rebroadcast as normal chat, making it a true whisper: spectators and the coached agent's opponent never see the text. This preserves strategic secrecy in agent-versus-agent coached matches. The match is flagged as coached so future leaderboard code can distinguish coached from uncoached ratings.

The stored directive surfaces in the next observation as a coach-directive object containing the text, the turn it was received, and an acknowledged flag. It also appears in the summary prose so text-only agents see it prominently.

A new slash-coach message from the same coach replaces the prior directive. Directives clear when the match archives, but the coached flag is intentionally permanent — a match cannot become uncoached retroactively.

This is distinct from the post-game analyser script, which is a separate tool with persistent memory for reviewing completed replays. The post-game coach and the mid-game slash-coach mechanism can coexist.

When to coach

When testing a new agent, the recommended mode is spectate only — watch and review replays without intervening. For ranked ladder matches, no coaching is allowed so the rating reflects pure agent performance. During casual or learning play, coaching is unrestricted. When an agent faces a human opponent, the human simply plays normally. When two agents face each other and both humans want to coach, that becomes a distinct competitive format where both coaches send directives simultaneously.

Coached matches are flagged and do not affect uncoached ratings, keeping the ladder clean while enabling coaching as its own game mode.

---

9. Evaluation and Benchmarking

9.1 Benchmark CLI

The benchmark script runs an external command agent against the built-in AI entirely in-process — no WebSocket server and no Durable Object are required. The external agent uses the same standard-input and standard-output protocol as the language model player script in agent-command mode.

The script accepts several flags: a required agent-command flag specifying the command to run, an opponent flag choosing easy, normal, hard, all, or a comma-separated list, a scenario flag, a games count, a seat selection, a per-decision timeout in milliseconds, a random seed, a compact flag, a version-two flag for the newer observation format, an output path for structured results, and a verbose flag.

Structured output includes, for each matchup, the scenario name, opponent difficulty, number of games, wins, losses, draws, win rate, estimated Elo rating, mean turns per game, mean decision time in milliseconds, action validity rate, timeout rate, parse error rate, and crash count. A per-game breakdown is also included.

Elo is estimated from win rate against a stable anchor — easy opponents anchor at 1000, normal at 1200, and hard at 1400 — using a standard log-odds formula clamped away from zero and one so small samples remain readable.

9.2 Replay analysis

The replay HTTP endpoint returns a full timeline of phase transitions, submitted actions including rejected ones, and the scoring breakdown. Agents with persistent memory can ingest these for cross-session improvement.

9.3 Public leaderboard

Shipped. The leaderboard uses Glicko-2 ratings with default starting values — a rating of fifteen hundred, a rating deviation of three hundred and fifty, and a volatility of zero-point-zero-six. It requires no login and places humans and agents on a single unified ladder. The public page is the leaderboard path; the API consists of the main leaderboard query and a per-player rank lookup.

Per-player fields stored in the database are the Glicko-2 triple — rating, rating deviation, and volatility, all updated after each rated match; games played and distinct opponents, used as exit-provisional counters; the "is agent" flag, which is set only on rows claimed through the agent-token endpoint rather than from the player-key prefix; and the timestamp of the last rated match. The claim endpoint verifies a bearer-authenticated agent before setting the "is agent" flag.

Agents claim a username by passing the player key and a claim username to the agent-token endpoint. First-call-wins per player key; a mismatched pair returns a 409 conflict without issuing a token. Without a claim, the agent plays anonymously and does not appear on the ladder. Rating writes are idempotent — the match-rating table is keyed by game identifier with an insert-or-ignore strategy.

Metrics that the public surface does not expose yet — action validity rate, stale-action rate, average decision latency, and scenarios-played distribution — remain live in telemetry and are candidates for a future agents tab.

---

10. Discovery and Metadata

Three endpoints support agent discovery. The machine-readable agent manifest, served at the well-known agent dot-JSON path, describes available scenarios, endpoints, rate limits, the protocol version, and bot conventions. The agent playbook file describes the turn loop, the phase-action map, payload shapes, and tactical guardrails, and is the first thing an agent should fetch at startup. The agents landing page is a human-readable page linked from the game-over screen and the main menu.

A discovery drift guard in the test suite fails continuous integration if the manifest and the engine disagree on which scenarios are available.

---

11. Security and Authentication

11.1 Principles

Six principles govern agent security. Agents are players, not admins: there is no hidden state, no admin endpoints, and no debug overlays. Validation is server-authoritative: invalid actions are rejected with a reason and never silently applied. Rate limiting applies at every boundary. Bot tagging is enforced: the player key must begin with "agent underscore". Tokens are short-lived and scoped so raw credentials never appear in an agent's context window. Fog-of-war is enforced uniformly: observations exclude undetected enemy ships using the same projection as the browser client.

11.2 Token lifecycle

The token system uses three layered token types. The agent token establishes agent identity across matches, lives for 24 hours and is renewable, and is sent as a bearer token in the authorization header on MCP calls. The match token scopes access to a single match and is bound to the issuing agent token; it lives for 4 hours and is passed as a tool argument. The player token covers a single match using the legacy and browser flow; it lives for the match duration plus a 5-minute grace period and is passed as a query string parameter.

Agents using the layered-token flow never see the raw player token in their language model context.

11.3 Sandboxing

For agents with broad system access, several precautions apply: run the agent in a container or virtual machine rather than on a primary workstation that holds credentials; scope API keys to the minimum required permissions; review agent actions in spectator mode before trusting autonomous play; and prefer tool-level MCP permissions over raw WebSocket access.

12. Integration Paths

12.1 MCP (recommended)

The Model Context Protocol path is the recommended integration. There are two variants.

The local variant uses standard input and output transport. It owns per-session WebSockets and an event buffer, and exposes additional tools for listing sessions, getting events, and closing sessions on top of the common toolset.

The remote variant uses streamable HTTP and is stateless, requiring no local installation.

With that established, the remote flow using layered tokens works as follows. First, make a one-time POST request to the agent-token endpoint with your agent player key and store the returned token. Second, send that token as a bearer authorization header on every MCP call. Third, call quick-match to receive a match token and the scenario name. Fourth, pass the match token to every subsequent tool call.

Optionally, agents can claim a public leaderboard username by including a claim object when requesting an agent token. The first agent to claim a given username for a player key wins; a username already owned by a different player key returns a conflict error without issuing a token. The same player key can re-call with a different username to rename itself. Without a claim, the agent plays anonymously and does not appear on the leaderboard. On success the response includes the player's username, agent flag, rating, rating deviation, and games-played count.

The legacy code-and-player-token tool arguments still work for create-based flows and bridge agents.

12.2 Bridge (stdin/stdout or HTTP)

Best for custom scripts and rapid prototyping. In the standard-input and standard-output mode, the bridge spawns the agent process once per decision turn. In HTTP mode, the agent runs as a persistent server and the bridge calls its decide endpoint. The agent turn input and agent turn response shapes are defined in the language model player script.

12.3 Standalone WebSocket

Best for agents in any language that need full control over the connection. The flow is: post to the quick-match endpoint to receive a ticket, poll the ticket status endpoint until the match is confirmed, then open a WebSocket connection using the game code and player token. From there, the agent reads server-to-client messages and sends client-to-server actions directly.

This path requires the agent to handle phase discipline, reconnection, and action validation itself.

12.4 Computer use (fallback)

Screenshot plus mouse and keyboard through a computer-use interface. This is the least reliable path for gameplay — hex grids with velocity vectors are extremely difficult for vision models to parse — but it is useful for smoke-testing the human user interface.

12.5 Comparison

Turning to a comparison of the four paths: MCP (remote only) requires no installation; the bridge and raw WebSocket require local setup; computer use requires none. Pre-computed candidates and legal action masks are available in MCP and the bridge but not in raw WebSocket or computer use. Phase-guarded submission is automatic in MCP and the bridge, manual in raw WebSocket, and absent in computer use. The blocking wait-for-turn is available in MCP but not applicable in the bridge, and must be implemented manually in raw WebSocket. Tactical features and the spatial grid are available as opt-in layers in MCP and the bridge; raw WebSocket receives only raw state; computer use receives screenshots. All four paths support any programming language. Latency is low for MCP and the bridge, lowest for raw WebSocket, and high for computer use. Reliability is high for MCP and the bridge, medium for raw WebSocket, and low for computer use.

---

13. Roadmap

Near-term items are tracked in the backlog document; this section describes the strategic shape. The rest of this spec covers what has already shipped and been production-tested.

Still open:

- **MCP resources** — URI-style read-only data for rules and replays so agents can fetch them as first-class resources rather than through bespoke HTTP calls.
- **Observation version-two wire-level unification** — collapsing the agent turn input into a single observation type shared across the bridge and MCP while keeping the opt-in layers.
- **Unify local and hosted MCP tool surfaces** — the local stdio server exposes list-sessions, get-events, and close-session helpers that the hosted server lacks; picking one name for quick-match and porting session buffering to the hosted side is on the backlog.
- **Multi-agent orchestration and tournament mode** — builds on the shipped leaderboard.
- **Spectator-to-coach upgrade flow** in the browser user interface.

This spec is a living document. Changes land with the code that implements them.

PART 6

Current Project State

The current backlog closes the book with the remaining open work and priority ordering.

Delta-V Backlog

A prioritised list of outstanding tasks that deserve a named home between pull requests — design gaps, tuning work, hardening items, and doc-versus-reality drift. Each entry is either actionable in the next few weeks or explicitly gated on a trigger.

Sections are grouped by theme and ordered by priority within each group. Gameplay-feel items translate most directly into a better player experience; architecture-solidity items unblock confident iteration on them. Shipped work lives in the repository's commit log rather than here; recurring review procedures live in the review plan document; architecture rationale lives in the architecture document; exploratory-pass technique lives in the exploratory testing document.

Recently shipped

As of mid-April 2026, a large batch of quality, accessibility, and agent-infrastructure work has landed on the main branch. The hosted Model Context Protocol now lives in its own workspace package, the server-side bot for agent seats defaults to the normal difficulty level used everywhere else, and the scenario tiles, focus rings, and phase banner have all been polished. A notification policy module now orders toasts, phase alerts, and the heads-up display status line by precedence and dedupes repeat toasts. Prefer-more-contrast and forced-colors passes landed across the menu shell, heads-up display, overlays, and game-over screen. The help overlay accessibility bugs around aria-hidden and the fleet-status label are fixed, and the waiting-screen game code gets read by assistive tech when it updates.

Security hardening: the hosted Model Context Protocol endpoint now enforces a sixteen-kilobyte JSON body cap and a per-agent-token rate limit, the agent-token secret fails closed in production instead of falling back to a placeholder, match-token redemption requires a bearer header, and quick-match enqueues with an agent-prefixed player key require a verified agent bearer so the "is agent" leaderboard flag cannot be spoofed by prefix alone. The public leaderboard itself — Glicko-2, no login, humans and agents on one ladder — is live, with claim endpoints, a provisional-hiding rule, and idempotent per-match rating writes.

Gameplay UX and matchmaking integrity

A handful of open UX items remain after the April 2026 batch.

Form controls still need a finer-grained split between mouse-focus and keyboard-focus styling. A quantified contrast audit — executed each release and then tuning the stylesheets from the measured results — is still outstanding. Stronger high-contrast mode coverage is partial: the help overlay's reused menu-content selectors on screens beyond the main menu still need a spot-check, and any game-over chrome outside the hero title may still read as flat in the strictest forced-colors themes.

The tutorial still needs a deeper task-first flow with spotlight-driven steps and a repeatable "what do I do now" affordance; the help overlay could optionally highlight the section in view on scroll, or collapse long groups into expandable blocks. Notification precedence is partly enforced in code via helpers, but call-site audits for duplicate copy between the heads-up display status line, the game log, and toasts are still open.

Core map interactions remain pointer-first: keyboard-first targeting and selection flows still need design and implementation so that gamepad support can reuse the same command path. Burn-arrow tap targets resolve per hex cell rather than only the painted circle, which usually gives at least forty-eight pixels at default zoom but can shrink on very small viewports — worth revisiting only if playtesting reports missed taps.

Artificial intelligence behavior and rules conformance

Findings from a deep-research pass against the 2018 Triplanetary rulebook, combined with observations of the artificial intelligence's ordnance behavior.

The ordnance artificial intelligence currently gates launches on distance buckets but never verifies that the launch vector will actually intersect a target hex within the five-turn ordnance lifetime, accounting for gravity. A short forward simulation that scores candidate launch burns by intersection probability is needed before a launch commits.

The recommended-index output can over-suggest consecutive ordnance launches. After a torpedo on turn two, the ordnance-phase recommendation on turn three may point at a nuke without the enemy being in range or being a credible threat. The hard-difficulty nuke gate also misses three factors — the three-hundred-MegaCredit cost of a nuke, the fact that it can be shot down at two-to-one odds with full range and velocity modifiers, and its tendency to detonate on any friendly ship, base, asteroid, mine, or torpedo in the path. An expected-damage estimate netted out against anti-nuke intercept odds, plus exclusion of friendly-lane vectors, should gate hard-difficulty launches.

Four subtle rules should be audited against the rulebook for drift: range is the attacker's closest approach rather than range to final position alone; the velocity penalty applies only when the difference exceeds two hexes; each ship may release only one ordnance item per turn; and only warships may launch torpedoes.

The mine-launcher gate should verify that the resulting course actually leaves the mine's own hex, not just that a burn was declared, to avoid self-destruction. Finally, the research pass produced concrete geometries where the hard-difficulty artificial intelligence still launches despite no credible five-turn intercept window — these should be encoded as deterministic test fixtures before any further heuristic tuning.

Agent and Model Context Protocol ergonomics

The agent contract is strong, with a pre-computed candidates array, labelled observations, two-token authentication, and action-guards forgiveness, but the Model Context Protocol surface has grown in two places and some per-turn affordances still cost extra round-trips.

Parallel stdio tool calls currently serialize: two quick-match connect calls sent in a single message do not run in parallel because the blocking long-poll runs to timeout before the second call queues. The fix is to keep long-poll waits off the stdio critical path.

First-touch errors on quick-match connect are rough. Passing a WebSocket URL throws a bare fetch-failed error from the underlying HTTP library; the normalization helper should either map WebSocket schemes to HTTP or reject with a clear message. Quick-match timeouts give no hint about the underlying cause — under development mode especially, a "no opponent queued" hint would be actionable.

The send-action tool's wait-for-result mode should treat protocol error events as rejections consistently. Today, sending an unknown action type resolves only after a separate event poll; the two paths should collapse to a single resolved rejection in both cases.

The disabled bot-fill path in the matchmaker Durable Object could be gated behind the development-mode flag so a single client can drive an end-to-end match locally without a second process.

The bot's nine-hundred-millisecond silence threshold is too aggressive for language-model agents with longer reasoning budgets. A longer default or a per-session configurable budget is needed.

The local and hosted Model Context Protocol tool surfaces diverge — the stdio server exposes list-sessions, get-events, and close-session helpers that the hosted server does not. Picking one name for quick-match and porting session buffering to the hosted side is the higher-value direction.

The wait-for-turn tool should return the full observation payload so a turn collapses to one blocking call plus one action call. The local stdio server's silent compaction of the state field should be gated behind an explicit opt-in.

The astrogation contract is inconsistent across surfaces: some describe it as simultaneous or pre-submittable while others gate on active player. A single model should flow through the engine, wait-for-turn, action guards, playbook JSON, and skill documentation.

The scrimmage script currently uses scrim-prefixed player keys, exercising the human quick-match path rather than the verified-agent flow that production agents use. Switching scrimmage defaults to agent identities and token issuance keeps the evaluation harness representative.

Model Context Protocol resources — universal-resource-identifier-style read-only data for rules and replays — are listed as a near-term resource surface but none are served yet. Serving the rules and replay resources would let hosts cache them and skip repeated HTTP fetches.

Action-rejection reasons should become a structured discriminated union rather than free-form strings. The smart-forgiveness path — where a stale phase is forgiven because the action type is still valid — should surface as its own reason.

When the thirty-second decision timeout fires and the server plays the recommended index on the agent's behalf, the agent is never told. A one-shot last-turn-auto-played field on the next observation would let agents notice missed turns and shrink their thinking budget.

A pre-release agent smoke checklist should land in the manual test plan: queue an agent against each difficulty for a small number of games, confirm at least ninety-five percent action acceptance, zero parse errors, under five percent decision-timeout rate, and that the six-agent harness finishes three concurrent matches without Durable Object instability.

The play skill has no mention of the coach-directive field even though observations surface it; add a short section on reading the directive each turn. The skill should also commit to one Model Context Protocol surface rather than referencing both.

Finally, once all active agents have migrated to match tokens, the legacy code-and-player-token tool-arg path can be retired from the hosted adapter.

Cost and abuse hardening

The current baseline is documented in the security document: hashed-IP GET throttles on the join and replay paths, a WebSocket upgrade cap, per-socket message rate limits, a chat throttle, caps on the telemetry and error POSTs, authoritative room creation, the Model Context Protocol two-token model, fail-closed behavior on the agent-token secret in production, and the thirty-day retention purge for the events table.

The items still on the backlog are trigger-gated: a Web Application Firewall or additional rate-limit namespaces if the baseline throttles prove insufficient; Turnstile on the human name-claim endpoint; proof-of-work on bulk agent name claims; and a spectator delay for serious competition.

Architecture and correctness

Several items remain on the architecture backlog. The initial publication path for new games should route through the same publication pipeline as every other state change so the random-number-generator fallbacks can be removed. Reinforcement and fleet-conversion side effects during turn advancement should become fully replayable by emitting explicit turn-advance events or by sharing one mutation implementation between the live engine and the event projector.

Caching of the current-state projection is partial — the checkpoint cleanup is done, but the live projection still rebuilds current state on every wake or read. An in-memory cache invalidated on every event append would close the gap. Publication and broadcast safety rails could be improved by replacing coarse JSON-

string parity failures with structured diffs, converging normalization between production and tests, making lower-level broadcast helpers private, and adding an exhaustive server-to-client builder and broadcast check. Boundary hardening should hide clone-sensitive engine mutators behind non-exported modules, extend import-boundary enforcement, and finish the client-kernel dependency-injection cleanup so WebSocket and fetch are injected rather than reached directly.

Type safety and scenario definitions

Several stringly-typed registries and identifiers still need to be closed off, with type guards for hex keys, tighter scenario and body registries, and branded ship and ordnance identifiers for lookup-heavy paths.

Scenario definitions and map data should be validated at load time and at game-creation time: conflicting rule combinations, unknown bodies, invalid spawn hexes, overlapping bodies, unreachable bases, and bounds that should be derived from body placement rather than from hardcoded constants.

Standardized error surfaces with a single engine-failure helper everywhere, combined with typed rate-limit and validation handling in the client, would allow user-facing error behavior to branch on error code rather than parsed text.

Testing and client consistency

Property tests for ordnance launch duplication, phase gating, and logistics transfer validation have shipped. What remains is positive client-to-server fixtures for the edge combat and combat-single messages, and negative-fixture protocol coverage for malformed payloads.

Future features, not currently planned

These items depend on product decisions or external triggers and are not in the active queue.

Public matchmaking with longer room identifiers would be added if the product moves beyond shared short codes. A trusted HTML sanitizer would be needed if chat, player names, or modded scenarios ever render as HTML. Web Application Firewall or additional rate-limit namespaces on join and replay probes would be added if distributed scans wake Durable Objects or cost too much. Cloudflare Turnstile on the human name-claim endpoint would be added if bulk claim POSTs appear in the logs or if the beta opens to a larger audience. Proof-of-work on the first agent name claim is symmetric in spirit: a few seconds of agent CPU on the first claim, painful at bulk. A spectator delay for organized competitive play would be added if real-time spectator leakage becomes a meaningful competitive risk. Populating the help-overlay screenshots is pending a UI freeze so in-game captures will not go stale in the next release cycle. Finally, an OpenClaw skill document published on the skill hub, gated on the agent-token environment variable, would let any OpenClaw agent auto-acquire Delta-V capability once the hosting platform is ready.